Introduction to R for Data Science: A LISA 2020 Guidebook

Jacob D. Holster

2022-07-10

Contents

1 R Foundations **7**

1.1 Getting Started with R and RStudio 11

1.2 Coding in R 13

1.3 Case Study 17

1.4 Review 18

1.5 References 18

2 Working with Data in R **21**

2.1 Installing and Calling Packages 21

2.2 Importing and Exporting Data 22

2.3 Working With Datasets 24

2.4 Case Study 35

2.5 Review 36

2.6 References 36

3 Data Visualization **39**

3.1 Overview 39

3.2 Plotting in R 45

3.3 Case Study 67

3.4 Review 74

3.5 References 74

4 Foundations of Statistical Analysis 77

4.1 Descriptive Statistics 78

4.2 Correlational Statistics 87

4.3 Review 92

4.4 References 93

5 Exploring and Modeling Data 95

5.1 Quick Review 95

5.2 Correlational Models 98

5.3 Simple and Multiple Regression 102

5.4 Inferential Statistics 112

5.5 Review 126

5.6 References 126

6 Structural Equation Modeling 129

6.1 SEM Syntax 129

6.2 Case Study 153

6.3 Review 155

6.4 References 156

7 Network Analysis 157

7.1 Zacharies Karate Club Case Study 159

7.2 Community Detection 169

7.3 Network Simulation 172

7.4 Advanced Case Study 180

7.5 Datasets for Network Analysis 183

7.6 Review 184

7.7 References 184

8 Text Analysis **187**

8.1 R and Text Data 188

8.2 Cleaning and Processing Text 191

8.3 Quantitative Text Analysis 197

8.4 Sentiment Analysis 200

8.5 Networks with Hamilton Lyrics 211

8.6 Topic Modeling 213

8.7 Review 220

8.8 References 220

Chapter 1

R Foundations

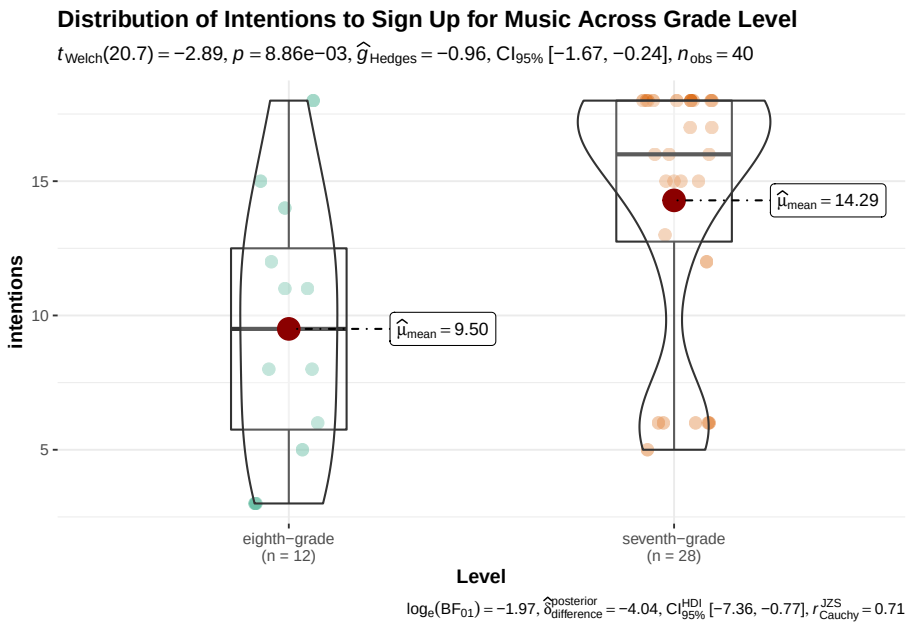
Data science is emerging as a vital skill for researchers, analysts, librarians, and others who deal with data in their personal and professional work. In essence, data science is the application of the scientific method to data for the purpose of understanding the world we live in. More specifically, data science tasks emerge from an interdisciplinary amalgam of statistical analysis, computer science, and social science research conventions. Although other programming languages such as python exceed R in general popularity, R remains as one of the most popular programming languages for data scientists and researchers due to a focus on statistical programming. As such, R and RStudio are invaluable tools to the data scientist, statistician, researcher, and many others.

This guidebook aims to provide readers an opportunity to make a start towards learning R for a variety of data science tasks, include (a) data cleaning and preparation, (b) statistical analysis, (c) data visualization, (d) natural language processing, (e) network analysis, and (f) Structural Equation Modeling to name a few. In Chapters 1 and 2 we invite readers to install R and RStudio and to start manipulating data for analysis. Chapter 3 and Chapter 4 include introductory exercises to teach data visualization and statistical analysis in R. In Chapter 5 and beyond, you will explore basic analytic concepts (e.g., correlation and regression) and more advanced approaches to data modeling through the lenses of Structural Equation Modeling, Network Analysis, and Text Analysis.

By the end of this guidebook, among several other skills, you will be able to use R to visualize distributions of data across categorical groups, such as in the example below:

```
library(ggstatsplot)
library(tidyverse)
data <- read_csv('musicclassintentions.csv')
```

```
ggbetweenstats(
  data = data,
  x = Level,
  y = intentions,
  title = "Distribution of Intentions to Sign Up for Music Across Grade Level"
)
```



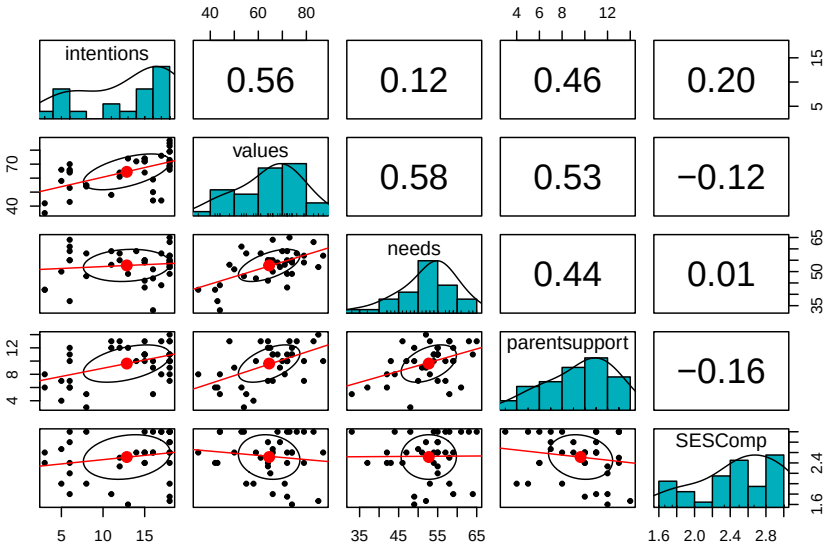
Calculate correlation coefficients, and visualize relationships between your data:

```
library(psych)
library(tidyverse)

variablelist <- data %>% select(intentions, values, needs, parentsupport, SESComp)
psych::pairs.panels(variablelist,
             method = "pearson", # correlation method
             hist.col = "#00AFBB",
             density = TRUE,  # show density plots
             ellipses = TRUE,
             lm = TRUE# show correlation ellipses
             )
```

Table 1.1: Estimates for Variation in Music Elective Intentions.

Predictor	B	SE	t	p
(Intercept)	-5.42	6.261	-0.87	0.394
SESComp	3.87	1.639	2.36	0.025
parentsupport	0.52	0.306	1.69	0.101
needs	-0.28	0.115	-2.40	0.023
values	0.28	0.072	3.90	0.001



Fit General Linear Models (GLM) and produce publishable tables:

```
library(tidyverse)
library(knitr)
library(broom)

tidy(lm(intentions ~ SESComp + parentsupport + needs + values, data=data)) %>%
  kable(caption = "Estimates for Variation in Music Elective Intentions.",
    col.names = c("Predictor", "B", "SE", "t", "p"),
    digits = c(0, 2, 3, 2, 3))
```

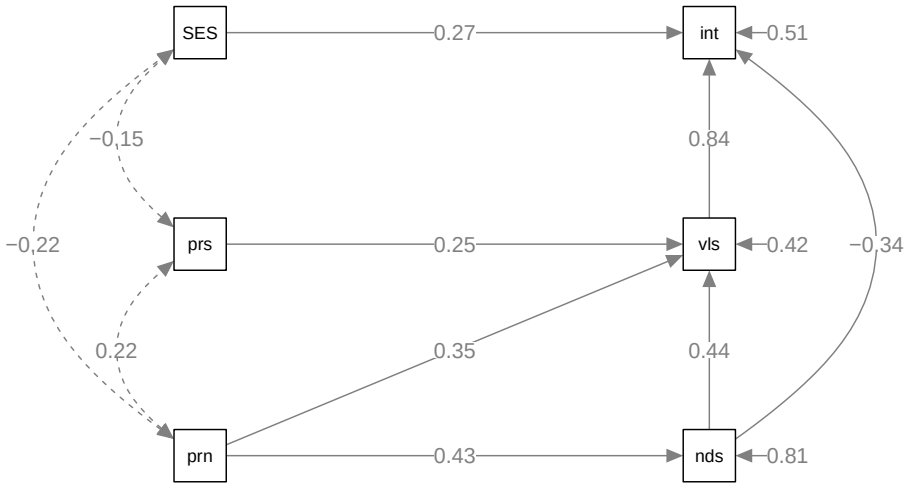
Fit, interpret, and visualize Structural Equation Models (SEM):

```
library(lavaan)
library(semPlot)

model <- '
  # regressions
    needs ~ parentsupport
    values ~ needs + parentsupport + peersupport
    intentions ~ values + needs + SESComp'

fit <- sem(model, data=data)

semPaths(fit, what = "path", whatLabels = "std", style = "lisrel", edge.label.cex = .9
```



As well as clean text data and analyze large corpora through the lenses of machine learning concepts such as sentiment analysis and topic modeling:

```
library(sentimentr)
sentiment('Sentiment analysis is super fun. I hate sentiment analysis. Sentiment analys
```

```
##    element_id sentence_id word_count  sentiment
## 1:          1           1          5  0.6708204
## 2:          1           2          4 -0.3750000
## 3:          1           3          4  0.0000000
```

Each concept is presented with unique datasets, including qualitative data from Harry Potter, Hamilton, and Donald Trump Tweets, and quantitative data from `baseR` datasets such as `mtcars`, elective intentions data for middle school students, and other real world data.

Additionally, there will be an informal assessment tied to each chapter so you can test and apply your skills as you move through the book.

1.1 Getting Started with R and RStudio

This chapter contains an introduction to the installation of R, how to install packages, and an introduction to object-based coding concepts. If you are having trouble downloading R and installing your first packages, please view the optional check in assessment at https://jayholster.shinyapps.io/RLevel0Assessment/ Take a few minutes to download and install both R and RStudio. They are both free and easy to download.

- Download R from <https://cran.r-project.org/> Choose your operating system (Windows, MacOS, or Linux) and download as you would any other program.
- Download the free version of RStudio for your OS from <https://www.rstudio.com/products/rstudio/download/ Follow prompts to install.

According to Rstudio's documentation, <https://cran.r-project.org/doc/contrib/Paradis-rdebuts_en.pdf>): "when R is running, variables, data, functions, results, etc, are stored in the active memory of the computer in the form of `objects` which have a name. The user can do actions on these objects with operators (arithmetic, logical,comparison, . . .) and functions (which are themselves objects)." We will explore the concept and capabilities of the `obect` throughout this text.

1.1.1 The Integrated Development Environment

Where R is a programming language, RStudio is an integrated development environment (IDE) which enables users to efficiently access and view most facets of the program in a four pane environment. These include the source, console, environment and history, as well as the files, plots, packages and help. The console is in the lower-left corner, and this is where commands are entered and output is printed. The source pane is in the upper-left corner, and is a built in text editor. While the console is where commands are entered, the source pane includes executable code which communicates with the console. The environment tab, in the upper-right corner displays an list of loaded R

objects. The history tab tracks the user's keystrokes entered into the console. Tabs to view plots, packages, help, and output viewer tabs are in the lower-right corner.

Where SPSS and other menu based analytic software are limited by user input and installed software features, R operates as a mediator between user inputs and open source developments from our colleagues all over the world. This affords R users a certain flexibility. However, it takes a few extra steps to appropriately launch projects. Regardless of your needs with R, you will likely interact with the following elements of document set up.

1.1.2 Learning to Read Code

> "Numquam ponenda est pluralitas sine necessitate. Plurality is never to be posited without necessity."
>
> William of Occam, circa 1495

The most formidable challenge many new R users face is learning to code. While coding can seem daunting at first, it is important to remember that all coding tasks simply involve solutions to problems the user identifies. No matter how difficult the problem, there are always a lot of solutions to each problems, and someone else has always encountered the solution, and likely has posted it to a forum. Occams Razor (i.e., the solution with the least amount of assumptions is the best) helps you identify the problems to solve as you interact with code. For new users, your breakthrough moment where you start to feel like a programmer might come from a well-worded google search and a focused effort to solve an issue with the R programming language. It is, indeed, a language, so half of the battle is learning to read code in a manner that is meaningful to you. Throughout this guidebook, you will be provided with tutorials and suggestions for reading the code you interact with. In this first chapter, you will start to encounter objects, functions, arguments, and operators. It is important to develop code reading fluency. As such, I encourage readers to speak code out loud to themselves as they interact with this guidebook.

1.1.3 Practices in Reproducibility

It is important—whether you are working alone or with others—to adopt a collaboration mindset. This value is clearly important when working with other statistical collaborators or with domain experts who do not have experience in R. Even experienced users might become confused when examining a peer's code. The same effect may occur if you return to a project after many months, and find yourself lost in your own code. As such, I recommend utilizing R markdown files (`File -> New File -> R Markdown`)and comments `#` to provide notes to yourself and others who might interact with your code. For instance, this is

an R Markdown document (`.Rmd` for file extensions). R Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents that can include blocks of code, as well as space for narrative to describe the code. For more details on using R Markdown see http://rmarkdown.rstudio.com.

The `#` symbol is used to change the size of headers `### = small, ## = medium, # = large`. There several options for formatting R markdown documents. For a introductory and comprehensive cheatsheet, see https://rstudio.com/wp-content/uploads/2015/02/rmarkdown-cheatsheet.pdf

There are no rules for how you should format your document. If you only need to write R code and have no need for whitespace or printing out your work, you can use an R script (`File -> New File -> R Script`). However, clean coding practices often require annotation, and R markdown makes that easy.

When you click the `Knit` button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can quickly insert chunks into your R Markdown file with the keyboard shortcut `Cmd + Option + I (Ctrl + Alt + I for Windows users)`. Comments can be utilized within code chunks, and are not considered a functioning part of the code to R.

```
x <- 10 # This is an example of a comment.
```

No matter what coding format you choose, insert narrative into your document in a way that makes sense to you. It may be helpful to split your code up into small, easy to digest chunks as to not become overwhelmed when examining your work.

1.2 Coding in R

1.2.1 Set Your Working Directory

It is helpful to use a project specific directory and to frequently save your work. When you first use R follow this procedure: Create a sub-directory (folder), "R" for example, in your "Documents" folder, or somewhere on your machine that you can easily access. This sub-folder, also known as working directory, will be used by R to read and save files. Think of it as a download and upload folder for R only.

You can specify your working directory to R in a few ways. One option is to click the `"Session"` drop down menu at the top of your screen and then click `"Set Working Directory."` It might also be useful to change the directory using code. To do this, use the function `setwd()`, and then enter the path of your directory into the parenthesis. Your working directory path will look something like what you see below. If you are unsure about the path you should input here,

find the folder using your machine's finder function, right click the folder, and examine the details of the folder's path. Make sure that you are using forward slashes in quotes as the example indicates.

You will need to run the code chunk in order to process the change in working directory. To run a code chunk, you can select the code you want to run and hit command and enter simultaneously on Mac, or control and enter on a Windows machine. Alternatively, you can click the green play button in the top right corner of the code chunk to run the entire cell.

```
setwd('/Users/username/Desktop/R/')
```

1.2.2 Operators

If this is your first foray into coding, you might think of it as a conversation you are having with R about a problem you are trying to solve. To start, you might consider simple arithmetic as the problem, and the code you write as the conversation. You can talk to R using numerical digits and text. Operators are the symbols that connect your numbers or words with mathematical (e.g., addition), relational (e.g., >=), and other logical or conditional manipulations

To start coding with mathematical operators, enter a number in the code box below, then click the run button.

```
7
```

```
## [1] 7
```

Now, pick a set of two numerals to sum, placing an addition sign between them. Then click the run button. Considering operators alone, R can be utilized as a simple calculator.

```
7+2
```

```
## [1] 9
```

Base R comes with working mathematical operators for addition `+`, subtraction `-`, multiplication `*`, division `/`, and exponents `^`. Although I've left an example for you below, you might try making your own.

```
7+2-10*40
```

```
## [1] -391
```

1.2.3 Functions

Functions run specific tasks based on the arguments within parenthesis. For example, the `sum()` function adds a specified number set. You can read the code below to say "sum the numbers 1 and 10."

```
sum(1, 10)
```

```
## [1] 11
```

There are some grammatical conventions that R requires to successfully run code. When you try to run this function below without the comma, R returns an error. Fix this example by including a comma directly after the first number. It does not matter whether you have a space after the comma before the next numeral. If any part of your code is not correct, it will produce an error message like you see below. The "unexpected numeric constant" in the error message refers to the space after the 1 that was not preceded by a comma.

```
sum(1 10)
```

```
## Error: <text>:1:7: unexpected numeric constant
## 1: sum(1 10
##            ^
```

Instead of a comma, you can use a colon—another operator—to sum a sequence of numbers. You can read this code aloud saying "sum all numbers from 1 to 10."

```
sum(1:10)
```

```
## [1] 55
```

There are a plethora of pre-installed functions in R. R also has the capability to find the right function for you. For instance, you might encounter a new function which you are not familiar with, like `seq()` below. To investigate what this function does, place your cursor after 'seq' but before the first parenthesis, and press tab. Hover over the function `seq` in the dropdown list to see a full description. If you need more information, you might use the `help()` function, with the name of the function you are curious about inside the parenthesis. The description indicates that the function is used to generate regular sequences. The information that emerges from using the `help()` function describes a list of arguments, including from, to, and by. This pane will populate on the bottom right side of RStudio when you run the `help()` function. Based on this information, what do you think the output of the first line in the following chunk will be?

```
seq(0, 20, 4)
```

```
## [1]  0  4  8 12 16 20
```

```
help(seq)
```

Was the output what you expected? The `seq()` function generates a sequence of numbers. In this example, 0 and 20 are the upper and lower limits of this sequence. The 4 indicates that the sequence should list numbers from 0 to 20 in increments of 4. Now, look to the bottom right side RStudio. Since you ran the entire cell, the command 'help(seq)' launched a search in the R documentation for the seq function in addition to running the seq function. Here, we can ascertain that this function takes a set of arguments `(from = 0, to = 20, by = 4)`. When you paste that exact code into the `seq` function, it generates the same result. Try it!

1.2.4 Objects

You may have heard the phrase "object-oriented programming." This phrase is accurate as all coding in R relies on the assignment of objects. When you assign an object in R, you are indicating that you want R to remember this assignment so it can be used as part of other code. `pi` is an example of an object built into base R. The input below is not numeric, but still represents a number. Run the code, and you will see that the word pi has been assigned the numeric value of pi. This is one of the few predefined objects in R.

```
pi
```

```
## [1] 3.141593
```

You might also assign a number pi to an object you name yourself.

```
x <- 3.141593
```

To assign values to objects, as the numerical content of pi was to the object x, use the `<-` operator. For example, the code segment below assigns the value 50 to the object 'a', and 14 to the object 'b' using the `<-` operator. It may be helpful to read the code out loud, saying a is 50, and b is 14.

```
a <- 50
b <- 14
```

There are five basic types of objects in R. The objects a, b, and x are each examples of **vectors**, or homogeneous data made up of characters, logical, numerical, or integer values. Additionally, a **list** of heterogeneous data (i.e., involving multiple data types) can be assigned to an object. **Matricies**, or two-dimensional data with undefined column headers, and **data frames**, a matrix with multiple specified columns that represent a certain type of observation seen in corresponding rows, can also be assigned to objects. Moreover, many objects are typically involved in coding and they tend to interact with each other.

See the code chunk below for a simple example of the interaction of two objects that were assigned in the previous code chunk.

```
a + b
```

```
## [1] 64
```

Notice that R held the object assignments from the previous cell. You can also assign a function to an object, and call that object to execute the function. For instance:

```
addvalues <- a + b
addvalues
```

```
## [1] 64
```

The product is not reached because R understands the input `addvalues`, but because the object add values calls the newly defined function 'a + b'. Try switching the values of a and b three chunks ago, and running the subsequent chunks. Remember that objects are case sensitive and cannot contain spaces. If you ran the code 'A+B', what would happen?

1.3 Case Study

Now it is time for our first case study.

You are approached by a colleague who wants create code that sums numerical values associated with letters in others' names (e.g., a = 1, b = 2,... z = 26). To start on this project, create 26 objects, one for each letter of the alphabet. Then sum your own name using mathematical expressions and the objects you created.

```
a <- 1
b <- 2
c <- 3 # and so on
```

1.4 Review

In this chapter, we explored how to download R and R studio, set up our working directory, learned about object based programming, and began to work with a few functions. To make sure you understand the material, there is a practice assessment to go along with this chapter at https://jayholster.shinyapps.io/RLevel0Assessment/.

1.5 References

Duignan, B. (2021). *Occam's razor*. *Encyclopedia Britannica*. https://www.britannica.com/topic/Occams-razor

Epskamp, S. (2015). semPlot: Unified visualizations of structural equation models. *Structural Equation Modeling, 22*(3), 474–483. <https://doi.org/10.1080/10705511.2014.937847>

Patil, I. (2021). Visualizations with statistical details: The 'ggstatsplot' approach. *Journal of Open Source Software*, *6*(61), 3167. [https://doi.org/[10.21105/joss.03167](https://doi

Robinson D, Hayes A, & Couch, S. (2022). *broom: Convert Statistical Objects into Tidy Tibbles*. https://broom.tidymodels.org/, https://github.com/tidymodels/broom.

Revelle, W. (2017) psych: Procedures for Personality and Psychological Research, Northwestern University, Evanston, Illinois, USA, <https://CRAN.R-project.org/package=psych> Version = 1.7.8.

Rinker, T.W. (2021). *sentimentr: Calculate Text Polarity Sentiment*. version 2.9.0, https://github.com/trinker/sentimentr.

Rosseel, Y. (2012). lavaan: An R Package for Structural Equation Modeling. *Journal of Statistical Software*, *48*(2), 1–36. https://doi.org/[10.18637/jss.v048.i02](https://doi.org/%5B

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L.D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T.L., Miller, E., Bache, S.M., Müller, K., Ooms, J., Robinson, D., Seidel, D.P., Spinu, V., Takahashi, K., Vaughan, D., Wilke, C., Woo, K., & Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686.

Xie, Y. (2021). *knitr: A General-Purpose Package for Dynamic Report Generation in R*. https://yihui.org/knitr/

1.5.1 R Short Course Series

Video lectures of each guidebook chapter can be found at https://osf.io/6jb9t/. For this chapter, find the follow the folder path R Level Zero -> AY 2021-2022

Spring and access the video files, r markdown documents, and other materials for each short course.

1.5.2 Acknowledgements

This guidebook was created with support from the Center for Research Data and Digital Scholarship and the Laboratory for Interdisciplinary Statistical Analaysis at the University of Colorado Boulder, as well as the U.S. Agency for International Development under cooperative agreement #7200AA18CA00022. Individuals who contributed to materials related to this project include Jacob Holster, Eric Vance, Michael Ramsey, Nicholas Varberg, and Nickoal Eichmann-Kalwara.

Chapter 2

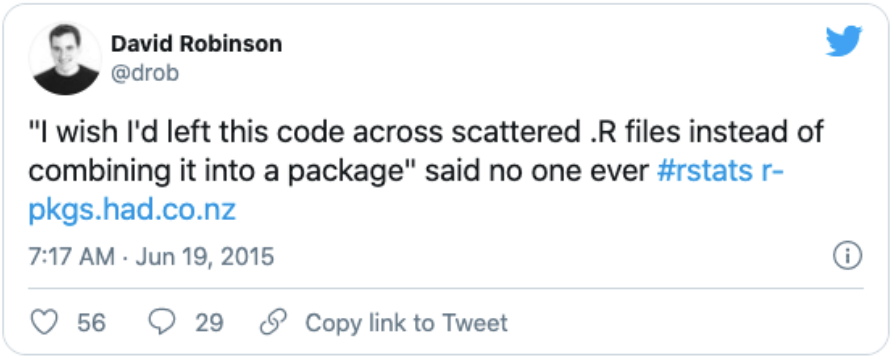
Working with Data in R

R would be almost useless without data. Therefore, it is important to become familiar with conventions that allow users to import, examine, and edit data. In this chapter, we will cover (a) how to install packages, where many open-source datasets are hosted; (b) how to import your own data; (c) how to identify and examine important characteristics of datasets, regardless of their point of origination; and (d) wrangle and modify datasets without leaving the IDE.

2.1 Installing and Calling Packages

BaseR comes with a lot of power. However, packages make R even more powerful. In essence, packages are the fundamental unit of shareable code in R. Packages help standardize tools and conventions and save time as they are often created by data professionals around the world to solve specific problems. For example, this guidebook relies heavily on the `tidyverse` package, which contains a set of R packages that "share an underlying design philosophy, grammar, and data structure" for the purpose of data wrangling, analysis, and visualization (Wickham et al., 2019). These include (a) `ggplot2` for data visualization, (b) `dplyr` for data manipulation, (c) `tidyr` for cleaning data, (d) `purrr` for consistent treatment of functions and vectors, (e) `tibble` for fast and accurate data uploading, (e) `stringr` for string or text manipulation, and (f) `forcats`, which supports categorical variables such as race/ethnicity or gender. For more information on `tidyverse`, type the function `help(tidyverse)` into a code chunk or script file.

There is a package for almost anything you can think of. You can even make your own packages if you find yourself using R for a unique work flow.



In this chapter, we will be using data from Nate Silver's statistical analysis website—`fivethirtyeight` (Kim et al., 2018). The `fivethirtyeight` package includes 128 callable datasets. For a full list of datasets, follow this link: <https://cran.r-project.org/web/packages/fivethirtyeight/vignettes/fivethirtyeight.html>. To see all datasets that are available to you as callable objects, run the function `data()`

To install packages, use the code below. Remember to wrap the package in quotes when you are installing it. Then, you will need to call the packages using the library() function. You do not need to use quotes when calling packages you have already installed.

```
install.packages("tidyverse")
install.packages("fivethirtyeight")
```

```
library(tidyverse)
library(fivethirtyeight)
```

2.2 Importing and Exporting Data

We will be using a built-in dataset from the `fivethirtyeight` package that contains information on US Births from 1994 to 2003. Call the dataset with the object named `'US_births_1994_2003` and assign it to the object `data` with the `<-` operator. This code can be read as "create a new object 'data' identical to the dataset 'US_births_1994_2003'." The purpose of renaming this data object is to simplify subsequent coding practices. That said, you can choose to rename your dataset anything you'd like, or keep the name as it was originally stated.

2.2.1 R-based Data

```
data <- US_births_1994_2003
```

Practice calling another dataset. Try calling the `data()`function or visiting the `fivethirtyeight` website and select a dataset that you are interested in examining. Once you have chosen a dataset, you can assign that data object to another name, or simply type in the name of the dataset into a code chunk. Alternatively, you can use the `data()` funciton with the name of the dataset inside the parenthesis to specify which dataset you are calling in.

If you would like to import your own data, use the `tidyverse` conventions below. Remember, you must have the `tidyverse` library installed and called for this function to work.

2.2.2 CSV Data

```
data <- read_csv('filename.csv')
```

Use caution with xlsx file types that contain multiple sheets. CSVs are utilized more frequently than xlsx files as `tidyverse` handles them quickly and simply. To load an xlsx file, you will need to explicitly install the "readxl" package. You can then call the specific sheet you want to load using the `sheet` argument. You will need to assign that sheet to an object to save that particular sheet for further examination and manipulation.

2.2.3 XLSX Data

```
install.packages('readxl')
library(readxl)
data <- read_excel(spreadsheet, sheet = "sheet1")
```

`Haven` is a package that is associated with, but not inherently a part of, the `tidyverse` package for SPSS, SAS, and STATA filetypes. You will need to follow similar conventions as with the readxl library. You can either define the path using an object, or explictly state the path in the ultimate function's parenthesis.

2.2.4 SPSS ".sav" Data

```
install.packages('haven')
library(haven)
path <- file.path("example.sav")
data <- read_sav(path)
```

2.2.5 SAS ".sas7bdat" or ".sas7bcat" Data

```
dataset <- read_sas("example.sas7bdat")
```

2.2.6 For Stata ".dta" Data

```
data <- read_dta("example.dta")
```

2.2.7 Exporting Data

You can export data similarly to how you imported data. See the code chunk below for examples of export functions for each data filetype. It is important to note that each of these are exported from the data object, as it is assumed you will have edited that data object at some point while using R. Each of these functions will save a dataset in your working directory based on the characteristics of the object "data" at the time the function was run.

```
write_csv(data)
write_excel(data)
write_sav(data)
write_sas(data)
write_dta(data)
```

2.3 Working With Datasets

Whether you are using your own dataset or conducting secondary data analysis, it is important to understand the data that are available to you. There are several ways to go about this in R.

2.3.1 Viewing the Dataset

First, consider the `head()` and `tail()` functions which display the first and last six rows of a dataframe, respectively. Place your data object within the function's parenthesis, and then run the code to get a snapshot of the dataset. Here, you might notice the output is a **tibble**. Although there are other distinctions, the key difference between a tibble and a dataframe is that tibbles display data types along with data (e.g., integer, date, numerial). You can see an example of this under the each column name below:

```
head(data)
```

```
## # A tibble: 6 x 6
##    year month date_of_month date       day_of_week births
##   <int> <int>         <int> <date>     <ord>        <int>
## 1  1994     1             1 1994-01-01 Sat           8096
## 2  1994     1             2 1994-01-02 Sun           7772
## 3  1994     1             3 1994-01-03 Mon          10142
## 4  1994     1             4 1994-01-04 Tues         11248
## 5  1994     1             5 1994-01-05 Wed          11053
## 6  1994     1             6 1994-01-06 Thurs        11406
```

In some cases a dataset contains dozens of variables in columns, which may be difficult to assess using the data viewer. To investigate the names of all columns, run the function `colnames()`

```
colnames(data)
```

```
## [1] "year"          "month"         "date_of_month" "date"
## [5] "day_of_week"   "births"
```

For a summary of each column of data, call the `summary()` function.

```
summary(data)
```

```
##       year          month        date_of_month        date
##  Min.   :1994   Min.   : 1.000   Min.   : 1.00   Min.   :1994-01-01
##  1st Qu.:1996   1st Qu.: 4.000   1st Qu.: 8.00   1st Qu.:1996-07-01
##  Median :1998   Median : 7.000   Median :16.00   Median :1998-12-31
##  Mean   :1998   Mean   : 6.524   Mean   :15.73   Mean   :1998-12-31
##  3rd Qu.:2001   3rd Qu.:10.000   3rd Qu.:23.00   3rd Qu.:2001-07-01
##  Max.   :2003   Max.   :12.000   Max.   :31.00   Max.   :2003-12-31
##
```

```
##  day_of_week     births
##  Sun  :522   Min.   : 6443
##  Mon  :522   1st Qu.: 8844
##  Tues :522   Median :11615
##  Wed  :522   Mean   :10877
##  Thurs:521   3rd Qu.:12274
##  Fri  :521   Max.   :14540
##  Sat  :522
```

You can also see the entire dataset in a separate pane within R using `view()`

```
view(data)
```

2.3.2 Data Types

Most R workflows involve one of five basic data types. These include (a) integers (e.g., 2); (b) numeric values, which include decimals (e.g., 2.5); (c) factors, or categorical variables with more than one level (e.g., gender); (d) logical, or Boolean values (i.e., True/False); and (e) characters or strings (e.g., "Thanks for all the fish"). Some functions will only work when a certain data type is present in the data set. For instance, if you plan to conduct an analysis of variance (ANOVA), you will need a factor to be present in your data. Similarly, a logistic regression typically requires Boolean values, and text analysis tends to require string data.

The following functions are useful to examine different facets related to data types present in the dataset. The `length()` function returns the number of elements or components in a dataset.

```
length(data)
```

```
## [1] 6
```

The `str()` function returns the structure of the dataset. The output of this function includes example data from each column, the data type, as well as the class of the data object.

```
str(data)
```

```
## tibble [3,652 x 6] (S3: tbl_df/tbl/data.frame)
##  $ year         : int [1:3652] 1994 1994 1994 1994 1994 1994 1994 1994 1994 1994 ..
##  $ month        : int [1:3652] 1 1 1 1 1 1 1 1 1 ...
##  $ date_of_month: int [1:3652] 1 2 3 4 5 6 7 8 9 10 ...
```

```
##  $ date         : Date[1:3652], format: "1994-01-01" "1994-01-02" ...
##  $ day_of_week  : Ord.factor w/ 7 levels "Sun"<"Mon"<"Tues"<..: 7 1 2 3 4 5 6 7 1 2 ...
##  $ births       : int [1:3652] 8096 7772 10142 11248 11053 11406 11251 8653 7910 10498 ...
```

If you are only interested in the class of the object, the `class()` function refers to the object itself, which in this case is a tibble.

```
class(data)
```

```
## [1] "tbl_df"     "tbl"        "data.frame"
```

The `names()` function, much like `col_names()` returns a list of the column names.

```
names(data)
```

```
## [1] "year"          "month"         "date_of_month" "date"
## [5] "day_of_week"   "births"
```

The `nrow()` function returns the number of rows in the dataset.

```
nrow(data)
```

```
## [1] 3652
```

While the `dim()` function returns the number of rows and columns simultaneously.

```
dim(data)
```

```
## [1] 3652    6
```

Sometimes when your data is loaded, R will recognize a column with an incorrect structure. For example, you can change the data type of the month column in US Births dataset 'month' integer, which tends to include decimals, to numeric using the function as.numeric(). The $ sign signifies that you are attempting to access a column based on its name in the dataset.

The code below explicitly converts the month column in the data object to a numeric data type using the function "as.numeric()." Follow up this conversion with the `str()` function to assess whether the change was made.

```
data$month <- as.numeric(data$month)
str(data)
```

```
## tibble [3,652 x 6] (S3: tbl_df/tbl/data.frame)
##  $ year         : int [1:3652] 1994 1994 1994 1994 1994 1994 1994 1994 1994 1994 ..
##  $ month        : num [1:3652] 1 1 1 1 1 1 1 1 1 ...
##  $ date_of_month: int [1:3652] 1 2 3 4 5 6 7 8 9 10 ...
##  $ date         : Date[1:3652], format: "1994-01-01" "1994-01-02" ...
##  $ day_of_week  : Ord.factor w/ 7 levels "Sun"<"Mon"<"Tues"<..: 7 1 2 3 4 5 6 7 1 2
##  $ births       : int [1:3652] 8096 7772 10142 11248 11053 11406 11251 8653 7910 10
```

When operationalizing factors for quantitative data analysis, you might want to convert a string based factor to numeric values (e.g., creating dummy variables out of categorical data). The code below converts the string data in the day of the week column into a numeric vector. When running factorial/categorical-based analyses (such as ANOVA variants or Chi-Square tests), R will create dummy variables to factors of data type "string." Each factor will be labeled in the order it uniquely appears in the dataset, row-wise.

In this example, we assigned the function to a new data object. As a result, we can manipulate and analyze the new data object while retaining the original. The creation of new objects will be useful as we begin to subset data for analysis.

```
num_day_of_Week <- as.numeric(data$day_of_week)
```

2.3.3 Examining Vectors and Variables

Object-oriented programming and statistics both tend to rely on the analysis of vectors and variables. The `unique()` function displays unique values in a column. Use this function to answer the question: Are all 12 months of the year represented in this dataset? This is also a great way to troubleshoot extant factor levels that are actually misspellings, or extra white space in the cell. For instance, if a cell in the month column mistakenly included 13, it would be visible in the output below.

```
# Get the unique months present in the data
unique(data$month)
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12
```

Furthermore, you can use `max()` and `min()` to identify the largest and smallest values in a column. Other functions help with descriptive statistics, such as `mean()` and `sd()` Descriptive statistics will be covered in more depth in Chapter Four.

```
max(data$births)
```

```
## [1] 14540
```

```
min(data$births)
```

```
## [1] 6443
```

```
mean(data$births)
```

```
## [1] 10876.82
```

```
sd(data$births)
```

```
## [1] 1858.567
```

2.3.4 Filtering Data

To subset the data for births from 1995 alone, use the `filter()` function to create a new object with the data you specify. The data are subset by the argument "year == 1995" which indicates that only observations that contain 1995 in the year column should be retained. In the code below, the filtered dataset is assigned to a new data object "births_1995." We then use the `head()` function to examine the first six instances in the dataset.

```
births_1995 <- filter(data, year == 1995)
head(births_1995)
```

```
## # A tibble: 6 x 6
##    year month date_of_month date       day_of_week births
##   <int> <dbl>         <int> <date>     <ord>        <int>
## 1  1995     1             1 1995-01-01 Sun           7828
## 2  1995     1             2 1995-01-02 Mon           7883
## 3  1995     1             3 1995-01-03 Tues          9999
## 4  1995     1             4 1995-01-04 Wed          11315
## 5  1995     1             5 1995-01-05 Thurs        11243
## 6  1995     1             6 1995-01-06 Fri          11506
```

Base R allows you to find the index of one column where a maximum occurs in another. For example, the following code uses the `max()` and `which.max()` functions to identify the date when the maximum number of births took place.

```
# Find the date of the maximum births
maxindex <- which.max(data$births)
maxdate <- data$date[maxindex]
maxdate
```

```
## [1] "1999-09-09"
```

However, you can use the `filter()` function from the `tidyverse` to achieve the same result in one line of code. You can read this aloud as "filter the object data where the births variable is equal to the maximum number present in the births column."

```
filter(data, births == max(births))
```

```
## # A tibble: 1 x 6
##    year month date_of_month date       day_of_week births
##   <int> <dbl>         <int> <date>     <ord>        <int>
## 1  1999     9             9 1999-09-09 Thurs        14540
```

Imagine that you want to identify the max number of births for a Sunday in particular in any year. This code uses the `filter()` function to create a new data object called sundays, which only includes data for observations where the day of the week was the string 'Sun.' In the next line of code, the `filter()` function is utilized on the newly created data object "sundays" to identify the observation with the maximum number of births.

```
# this is what filter is doing on groups
sundays <- filter(data, day_of_week == 'Sun')
filter(sundays, births==max(births))
```

```
## # A tibble: 1 x 6
##    year month date_of_month date       day_of_week births
##   <int> <dbl>         <int> <date>     <ord>        <int>
## 1  1994     8            14 1994-08-14 Sun           8926
```

If you are only interested in specific variables or vectors within a data frame you can use the `select()` function and list the data object, then the column names that you are interested in seeing. Additionally, notice how the `head()` function is wrapped around the `select()` function in this example to limit the output to the first six observations.

```
head(select(data, date, births))
```

```
## # A tibble: 6 x 2
##   date       births
##   <date>      <int>
## 1 1994-01-01   8096
## 2 1994-01-02   7772
## 3 1994-01-03  10142
## 4 1994-01-04  11248
## 5 1994-01-05  11053
## 6 1994-01-06  11406
```

To remove a column from the data object, place a minus sign before it in the column argument space within the `select()` function.

```
head(select(data, -date))
```

```
## # A tibble: 6 x 5
##    year month date_of_month day_of_week births
##   <int> <dbl>         <int> <ord>        <int>
## 1  1994     1             1 Sat           8096
## 2  1994     1             2 Sun           7772
## 3  1994     1             3 Mon          10142
## 4  1994     1             4 Tues         11248
## 5  1994     1             5 Wed          11053
## 6  1994     1             6 Thurs        11406
```

2.3.5 Grouping Data

Alternatively, you can use the `group_by()` function while filtering the dataset by max births to display the maximum number of births on each day of the week for any year. The `group_by()` function groups the data object by the observations in the called column, which in this case is "day_of_week." Then, the grouped data are filtered to with the same `max()` function. Grouping the data retains the factor structure of the "day_of_week" vector in subsequent functions. The grouped and filtered dataset are assigned to a new object "maxbirths" for future reference.

```
data_grouped <- group_by(data, day_of_week)
maxbirths <- filter(data_grouped, births == max(births))
maxbirths
```

```
## # A tibble: 7 x 6
## # Groups:   day_of_week [7]
##    year month date_of_month date       day_of_week births
##   <int> <dbl>         <int> <date>     <ord>        <int>
## 1  1994     8            14 1994-08-14 Sun           8926
## 2  1994     9            17 1994-09-17 Sat           9779
## 3  1999     9             9 1999-09-09 Thurs        14540
## 4  2001    12            28 2001-12-28 Fri          13918
## 5  2003     9             3 2003-09-03 Wed          14119
## 6  2003    12            22 2003-12-22 Mon          12967
## 7  2003    12            30 2003-12-30 Tues         14438
```

You can also group by more than one column. This code expands on the previous example to include the maximum number of births on each day of the week within each year present in the dataset.

```
# Add year as another grouping variable
data_grouped <- group_by(data, day_of_week, year)
data_max_day_year <- filter(data_grouped, births==max(births))
head(data_max_day_year)
```

```
## # A tibble: 6 x 6
## # Groups:   day_of_week, year [6]
##    year month date_of_month date       day_of_week births
##   <int> <dbl>         <int> <date>     <ord>        <int>
## 1  1994     7             6 1994-07-06 Wed          13086
## 2  1994     7             7 1994-07-07 Thurs        13049
## 3  1994     8            14 1994-08-14 Sun           8926
## 4  1994     9            16 1994-09-16 Fri          12884
## 5  1994     9            17 1994-09-17 Sat           9779
## 6  1994    11            21 1994-11-21 Mon          11807
```

Arguments within functions are oftentimes objects themselves. As such, you can replace the "data_grouped" object in the previous chunk with its assigned `group_by()` function to simplify your code while achieving the same result.

```
data_max_day_year <- filter(group_by(data, day_of_week, year), births==max(births))
head(data_max_day_year)
```

```
## # A tibble: 6 x 6
## # Groups:   day_of_week, year [6]
##    year month date_of_month date       day_of_week births
##   <int> <dbl>         <int> <date>     <ord>        <int>
## 1  1994     7             6 1994-07-06 Wed          13086
```

```
## 2  1994     7             7 1994-07-07 Thurs        13049
## 3  1994     8            14 1994-08-14 Sun          8926
## 4  1994     9            16 1994-09-16 Fri         12884
## 5  1994     9            17 1994-09-17 Sat          9779
## 6  1994    11            21 1994-11-21 Mon         11807
```

2.3.6 Altering Data

There are many useful functions for altering your data frame. In this section you will start to see the %>%operator. You can read this operator as the word 'then' or 'where' when you are reading code to yourself (e.g., take the original dataset, then arrange it by births in descending order)

You can use this operator to apply functions to your dataset. For instance, the code below reorders the observations in the data by births in descending order. To arrange in ascending order simply remove the additional `desc()` function within the `arrange()` function as the default arrangement is ascending order.

```
# Order by number of births
data <- data %>% arrange(desc(births))
head(data)
```

```
## # A tibble: 6 x 6
##    year month date_of_month date       day_of_week births
##   <int> <dbl>         <int> <date>     <ord>        <int>
## 1  1999     9             9 1999-09-09 Thurs        14540
## 2  2003    12            30 2003-12-30 Tues         14438
## 3  2003     9            16 2003-09-16 Tues         14145
## 4  2003     9             3 2003-09-03 Wed          14119
## 5  2003     9            23 2003-09-23 Tues         14036
## 6  2002     9            12 2002-09-12 Thurs        13982
```

You can also rename specific columns of your data without opening and editing the file using the `rename()` function. You can read the code below aloud saying "create a new object data from the old object data where you rename date_of_month as day, and day_of_week as weekday."

```
# Rename day_of_month and day_of_week columns
data <- data %>% rename(day = date_of_month, weekday = day_of_week)
head(data)
```

```
## # A tibble: 6 x 6
##    year month   day date       weekday births
##   <int> <dbl> <int> <date>     <ord>    <int>
```

```
## 1  1999     9     9 1999-09-09 Thurs    14540
## 2  2003    12    30 2003-12-30 Tues     14438
## 3  2003     9    16 2003-09-16 Tues     14145
## 4  2003     9     3 2003-09-03 Wed      14119
## 5  2003     9    23 2003-09-23 Tues     14036
## 6  2002     9    12 2002-09-12 Thurs    13982
```

The examples below utilize the %>% operator to conditionally select observations within the data object. The first example reads, "create a new object july_data" based on the old object data where you filter for observations where the month column value is 7." The second example extends this concept by including a second condition. The ampersand in the `filter()` function allows for the identification of observations only in the month of January, 2000.

```
# Select days only in the month of July
july_data <- data %>% filter(month == 7)

# Select days only in the month of january in the year 2000
jan_2000_data <- data %>% filter(month == 1 & year == 2000)
```

Recall our use of sequences in Chapter One. In this example, we filter out the data for the first half of the year (January through June) using a colon in the subsetting argument rather than individual numbers. Then, use the `max()` function to check whether the the filtering function worked.

```
janthrujune <- data %>% filter(month == 1:6)
max(janthrujune$month)
```

```
## [1] 6
```

You can also add an additional column to the data frame based on conditional values of other columns using the `mutate()` function. The example below creates a vector Boolean values which indicates whether the observation occurred during a warmer month. The subsequent plot serves as a preview of the next chapter on Data Visualization, but also demonstrates a practical use of the mutate function in statistical modeling. The grouped violin plot answers the question: Were there more births in warmer months than in colder months?

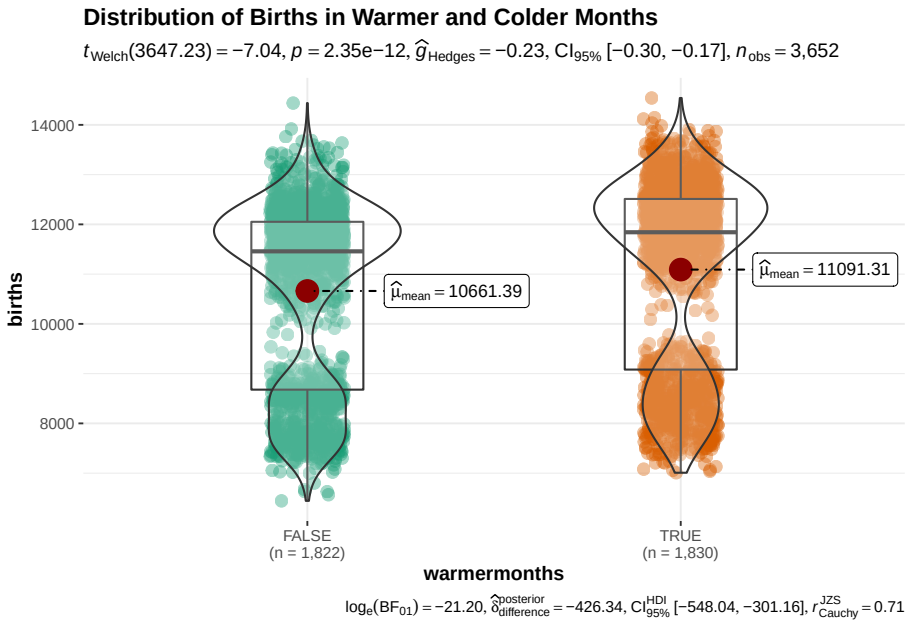
```
data <- data %>% mutate(warmermonths = between(month, 4, 9))
str(data)
```

```
## tibble [3,652 x 7] (S3: tbl_df/tbl/data.frame)
##  $ year        : int [1:3652] 1999 2003 2003 2003 2003 2002 2001 2003 2002 2002 ...
```

```
##  $ month       : num [1:3652] 9 12 9 9 9 9 12 9 9 9 ...
##  $ day         : int [1:3652] 9 30 16 3 23 12 28 10 24 17 ...
##  $ date        : Date[1:3652], format: "1999-09-09" "2003-12-30" ...
##  $ weekday     : Ord.factor w/ 7 levels "Sun"<"Mon"<"Tues"<..: 5 3 3 4 3 5 6 4 3 3 ...
##  $ births      : int [1:3652] 14540 14438 14145 14119 14036 13982 13918 13908 13884 13883 ...
##  $ warmermonths: logi [1:3652] TRUE FALSE TRUE TRUE TRUE TRUE ...
```

```
library(ggstatsplot)

ggbetweenstats(
  data = data,
  x = warmermonths,
  y = births,
  title = "Distribution of Births in Warmer and Colder Months"
)
```



2.4 Case Study

Use the following exercises to learn more about the US_births_1994_2003 dataset and practice the skills you acquired in this chapter.

- Using the `filter()`, `min()`, and/or `group_by()` functions, identify the Wednesday and Friday in the dataset with the least total number of births.

- Find the number of births for January 1st across the dataset.
- Subset the dataset to only include observations that occur in winter (December, January, February, and March in North America)
- Subset the previous winter births dataset further by only allowing instances after 2001 to remain in the dataset.

2.5 Review

In this chapter, we covered how to install and use libraries, how to import data from several formats, and how packages support several basic data science tasks. To make sure you understand this material, there is a practice assessment to go along with this chapter at https://jayholster.shinyapps.io/IntrotoRAssessment/.

2.6 References

Kim A.Y., Ismay C., & Chunn, J. (2018). "The fivethirtyeight R Package: 'Tame Data' Principles for Introductory Statistics and Data Science Courses." *Technology Innovations in Statistics Education*, *11*. https://escholarship.org/uc/item/0rx1231m.

Patil, I. (2021). Visualizations with statistical details: The 'ggstatsplot' approach. *Journal of Open Source Software*, *6*(61), 3167. https://doi.org/10.21105/joss.03167

Wickham H., Bryan, J. (2022). readxl: Read Excel Files. https://readxl.tidyverse.org, https://github.com/tidyverse/readxl.

Wickahm, H., & Miller, E., (2022) haven. https://cran.r-project.org/web/packages/haven/index.html

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L.D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T.L., Miller, E., Bache, S.M., Müller, K., Ooms, J., Robinson, D., Seidel, D.P., Spinu, V., Takahashi, K., Vaughan, D., Wilke, C., Woo, K., & Yutani, H. (2019). Welcome to the tidyverse. Journal of Open Source Software, 4(43), 1686. https://doi.org/10.21105/joss.01686.

2.6.1 R Short Course Series

Video lectures of each guidebook chapter can be found at https://osf.io/6jb9t/. For this chapter, find the follow the folder path Intro to R -> AY 2021-2022 Spring and access the video files, r markdown documents, and other materials for each short course.

2.6.2 Acknowledgements

This guidebook was created with support from the Center for Research Data and Digital Scholarship and the Laboratory for Interdisciplinary Statistical Analysis at the University of Colorado Boulder, as well as the U.S. Agency for International Development under cooperative agreement #7200AA18CA00022. Individuals who contributed to materials related to this project include Jacob Holster, Eric Vance, Michael Ramsey, Nicholas Varberg, and Nickoal Eichmann-Kalwara.

Chapter 3

Data Visualization

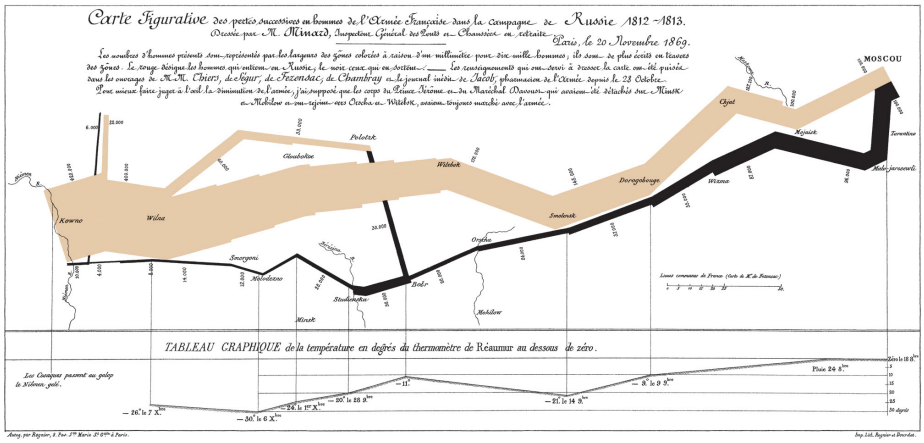


Figure 3.1: Charles Minard's Napolean's Disastrous Russia Campaign of 1812

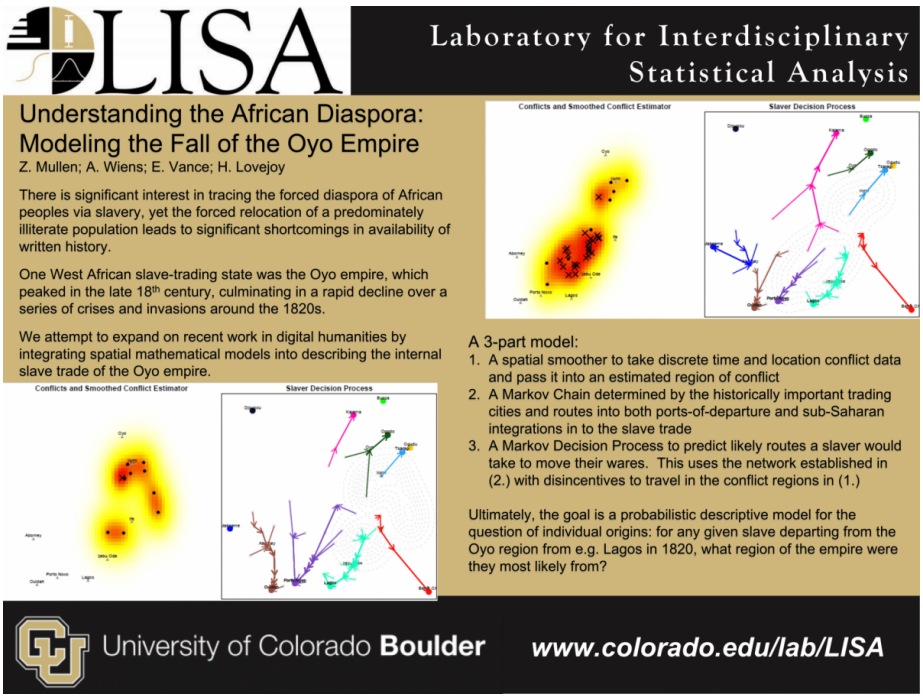
3.1 Overview

Socrates once argued that explanations for phenomena should be *teleological*. Teleological thinkers tend to place the purpose, or function, of at the center of any explanation—asking not what things are, but what things do. Data visualizations *are* most often the representation of found or collected data. They are often presented through the use of figures (e.g., charts, conceptual maps, scatter plots) in two-dimensional space.

The answer to "what does data visualization do" says a great deal more about both the value of effective data visualization and the capabilities of R to support

that goal. High quality data visualizations, created in R or not, tend to both answer a research question and tell a story. For instance, Charles Minard visualized Napoleon's troop movements to and from Moscow in the war of 1812. This chart displays the geography, temperature, as well as the number and direction French troops were moving. More importantly, the visualization demonstrates a significant loss of life—over 300,000 troops would not make the journey back home (as signified by the width of the black line).

Or see the slide below which includes a visualization of forced relocation of African peoples in the 19th century (more details on the slide).



Or the example below which uses qualitative data to illustrate trending google searches in 2021.

In this chapter you will become familiar with several data visualization techniques in R, with special attention paid to the `qplot` and `ggplot` libraries.

3.1.1 Common Data Visualizations

The R programming language is popular among statisticians, quantitative researchers, and data scientists. As such, data visualization packages in R tend to best support the creation of statistical displays. These displays might provide insight, if not answers, to research questions. All examples in this section are

created using the `ggstatsplot` library, which provides support for creating robust visualizations with minimal code. Additionally, the `ggstatsplot` follows most conventions that are used in `ggplot2`, which will be covered in depth later in this chapter.

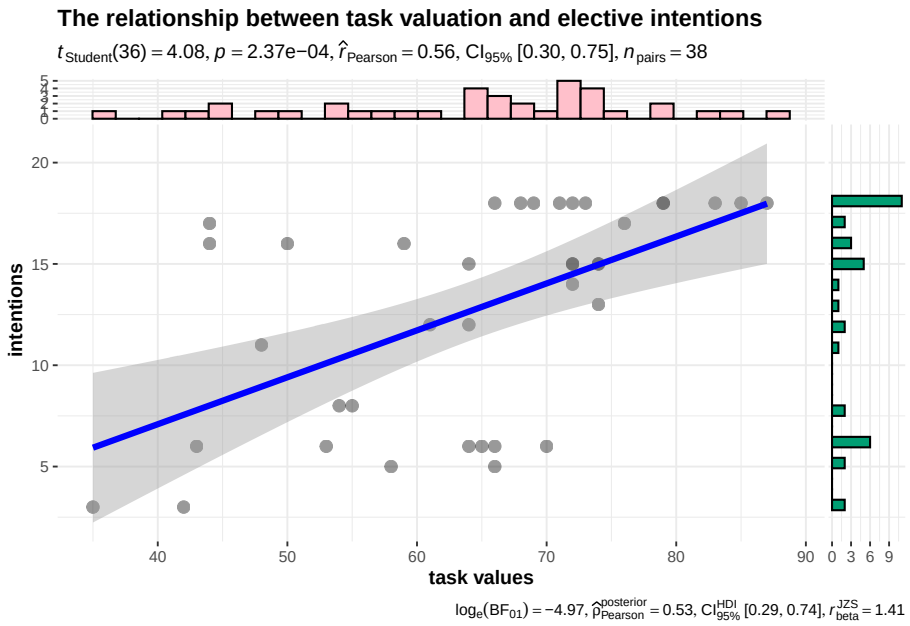
3.1.1.1 Scatter Plots

Many statistical visualizations are axis-based. For instance, *scatter plots* use the x and y axis to pinpoint the exact measurement of similar cases across two variables. A scatter plot can demonstrate the extent to which variables are correlated, or whether relationships are linear. Observations—rows in datasets—are plotted on the x and y axis simultaneously. Patterns can sometimes be visible when multiple data points are plotted together. The plot below includes a line of best fit, a visualization of a 95% confidence interval, and violin plots for the distributions of values on the x and y axis. More specifically, the plot indicates a moderate, positive, and significant relationship between task values (e.g., interest in a task) and elective intentions for the surveyed instrumental adolescents.

```
library(tidyverse)
library(ggstatsplot)

data <- read_csv('musicclassintentions.csv')

ggscatterstats(
  data = data,
  x = values,
  y = intentions,
  marginal.type = "violin",
  xlab = "task values",
  ylab = "intentions",
  title = "The relationship between task valuation and elective intentions",
  xfill = "pink",
  yfill = "#009E73"
)
```

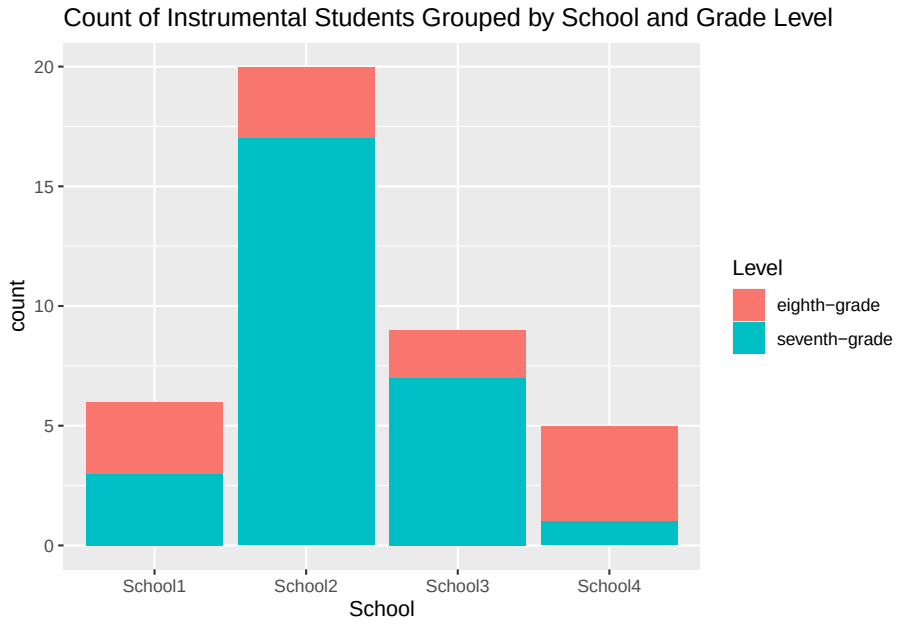


3.1.1.2 Histograms

At times, the y-axis only represents the count, and sometimes density, of single variables. You might use a **bar chart** or **histogram** to display univariate descriptive statistics. The underlying purpose of this visualization is simple. It answers the question, "how many students in the sample come from each school?" The fill argument answers a second question, "how many students are in seventh- or eighth-grade within each school?"

```
ggplot(data = data, aes(x = School, fill = Level)) +
  geom_histogram(stat = "count") +
  ggtitle("Count of Instrumental Students Grouped by School and Grade Level") +
  xlab("School")
```

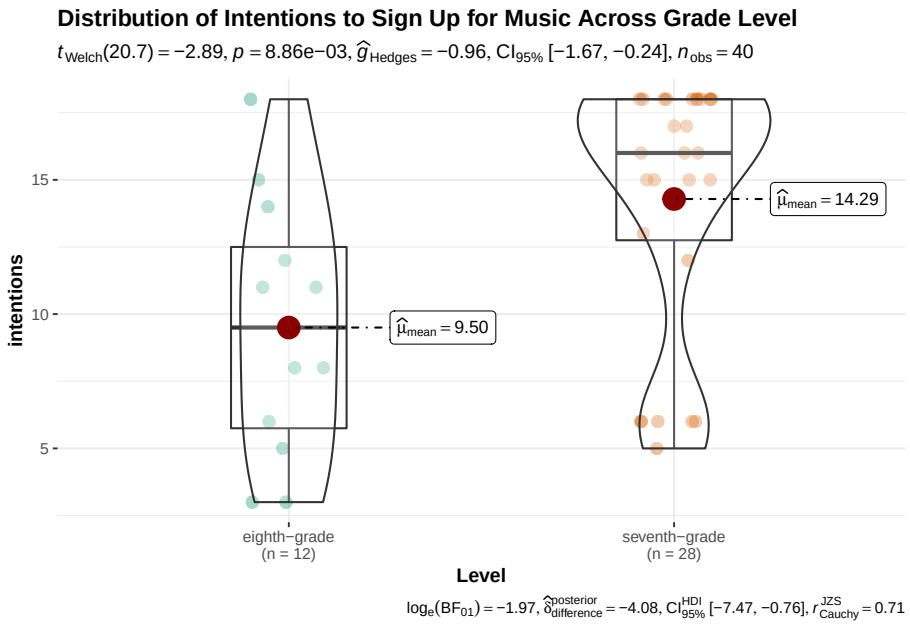
```
## Warning: Ignoring unknown parameters: binwidth, bins, pad
```



3.1.1.3 Violin Plots

Violin plots can contextualize group differences found in data analysis. You may recall this example from Chapter 1. The red dot represents the mean of each group, and the gray horizontal line represents each median. These plots demonstrate that there is a statistical and practical significant difference in how seventh- and eighth-grade band and orchestra students in a United States sample are considering future elective music studies. The plot also indicates a potential ceiling effect for seventh-grade students, as many responded with the highest possible score. There are several other plots that tend to be utilized for illustrating group differences.

```
ggbetweenstats(
  data = data,
  x = Level,
  y = intentions,
  title = "Distribution of Intentions to Sign Up for Music Across Grade Level"
)
```



3.1.2 Visualization as the Research Process

Data visualization lends itself as responsive to demands of the research processes. For instance, you might assess the normality of variables with a histogram, or you might visualize the strength and direction of a correlation with a scatter plot. Visualizations can answer or contribute to the answers of research questions.

To exemplify this concept, we are going to start with data collected on Bechdel test. The Bechdel test measured the representation of women in 1,794 fictional films released between 1971 and 2013. Films that pass the Bechdel test would feature at least two women who talk to each other about something other than a man. Roughly half of films pass the test. The dataset also includes financial information, including the domestic and international earnings. The inclusion of these social and financial variables allows for some consideration of bias in the film industry.

3.1.3 Getting to Know the Packages

The Bechdel dataset is called from the `fivethirtyeight` package. If you do not have the packages already, install and call the `tidyverse`, `fivethirtyeight`, and `ggpubr` packages.

As discussed in the first chapter, the `tidyverse` installation contains multiple R packages for the purpose of data wrangling, analysis, and visualization. Descriptions of the attendant packages are listed in the table below.

Table 3.1: tidyverse packages

pac kage	purpose
`ggpl ot2`	data visualization (e.g., histogram, scatter plot)
`ti dyr`	produces tables where rows are observation
`re adr`	imports data from working directory
`pu rrr`	mapping functions across multiple variables
`tib ble`	allows tables to keep column names with no row names, includes and displays data type in output
`stri ngr`	tools for string manipulation and identification (e.g., "text data")
`forc ats`	considers whether data are factors and creates a specific data type in the tibble
`dp lyr`	tools for data manipulation (e.g., changing column names, subsetting data)

`ggpubr` includes tools for creating publishable plots, making it easy to annotate and arrange multiple plots. Install any packages that you need, and then load each library.

3.2 Plotting in R

Now that you have loaded the `fivethirtyeight` package, load the bechdel dataset. Here it is assigned to the object `bech_data` Then examine the dataset using the `View()` and `str()` functions. You might also use the `psych::describe()`function for quick descriptive analysis of any quantitative data, including skewness and kurtosis value, which are metrics of the normality of data distributions. Statistical concepts will be introduced in further detail in the next chapter.

```
## tibble [1,794 x 15] (S3: tbl_df/tbl/data.frame)
##  $ year         : int [1:1794] 2013 2012 2013 2013 2013 2013 2013 2013 2013 2013 ...
##  $ imdb         : chr [1:1794] "tt1711425" "tt1343727" "tt2024544" "tt1272878" ...
##  $ title        : chr [1:1794] "21 & Over" "Dredd 3D" "12 Years a Slave" "2 Guns" ...
##  $ test         : chr [1:1794] "notalk" "ok-disagree" "notalk-disagree" "notalk" ...
##  $ clean_test   : Ord.factor w/ 5 levels "nowomen"<"notalk"<..: 2 5 2 2 3 3 2 5 5 2 ...
##  $ binary       : chr [1:1794] "FAIL" "PASS" "FAIL" "FAIL" ...
```

```
##  $ budget       : int [1:1794] 13000000 45000000 20000000 61000000 40000000 2250000
##  $ domgross     : num [1:1794] 25682380 13414714 53107035 75612460 95020213 ...
##  $ intgross     : num [1:1794] 4.22e+07 4.09e+07 1.59e+08 1.32e+08 9.50e+07 ...
##  $ code         : chr [1:1794] "2013FAIL" "2012PASS" "2013FAIL" "2013FAIL" ...
##  $ budget_2013  : int [1:1794] 13000000 45658735 20000000 61000000 40000000 2250000
##  $ domgross_2013: num [1:1794] 25682380 13611086 53107035 75612460 95020213 ...
##  $ intgross_2013: num [1:1794] 4.22e+07 4.15e+07 1.59e+08 1.32e+08 9.50e+07 ...
##  $ period_code  : int [1:1794] 1 1 1 1 1 1 1 1 1 1 ...
##  $ decade_code  : int [1:1794] 1 1 1 1 1 1 1 1 1 1 ...
```

```
## # A tibble: 1,794 x 15
##     year imdb      title  test  clean_test binary budget domgross intgross code
##    <int> <chr>     <chr>  <chr> <ord>      <chr>   <int>    <dbl>    <dbl> <chr>
##  1  2013 tt1711425 21 & ~ nota~ notalk     FAIL   1.3 e7 25682380   4.22e7 2013~
##  2  2012 tt1343727 Dredd~ ok-d~ ok         PASS   4.5 e7 13414714   4.09e7 2012~
##  3  2013 tt2024544 12 Ye~ nota~ notalk     FAIL   2   e7 53107035   1.59e8 2013~
##  4  2013 tt1272878 2 Guns nota~ notalk     FAIL   6.1 e7 75612460   1.32e8 2013~
##  5  2013 tt0453562 42     men   men        FAIL   4   e7 95020213   9.50e7 2013~
##  6  2013 tt1335975 47 Ro~ men   men        FAIL   2.25e8 38362475   1.46e8 2013~
##  7  2013 tt1606378 A Goo~ nota~ notalk     FAIL   9.2 e7 67349198   3.04e8 2013~
##  8  2013 tt2194499 About~ ok-d~ ok         PASS   1.2 e7 15323921   8.73e7 2013~
##  9  2013 tt1814621 Admis~ ok    ok         PASS   1.3 e7 18007317   1.80e7 2013~
## 10  2013 tt1815862 After~ nota~ notalk     FAIL   1.3 e8 60522097   2.44e8 2013~
## # ... with 1,784 more rows, and 5 more variables: budget_2013 <int>,
## #   domgross_2013 <dbl>, intgross_2013 <dbl>, period_code <int>,
## #   decade_code <int>
```

```
##                vars    n          mean           sd     median      trimmed
## year              1 1794       2002.55         8.98     2005.0      2003.88
## imdb*             2 1794        897.50       518.03      897.5       897.50
## title*            3 1794        883.82       510.14      885.5       883.69
## test*             4 1794          6.65         2.62        7.0         6.89
## clean_test*       5 1794          3.53         1.48        4.0         3.64
## binary*           6 1794          1.45         0.50        1.0         1.43
## budget            7 1794  44826462.61  48186026.12 28000000.0  35815614.85
## domgross          8 1777  69132048.28  80367309.51 42194060.0  53837590.51
## intgross          9 1783 150385700.05 210335267.83 76482461.0 106338373.03
## code*            10 1794        63.57        17.95       68.0        66.22
## budget_2013      11 1794  55464608.45  54918635.60 36995786.0  46262685.87
## domgross_2013    12 1776  95174783.58 125965348.89 55993640.5  70822783.23
## intgross_2013    13 1783 197837984.97 283507948.20 96239640.0 138312058.71
## period_code      14 1615          2.42         1.19        2.0         2.33
## decade_code      15 1615          1.94         0.69        2.0         1.92
##                       mad  min        max      range  skew kurtosis         se
## year                 7.41 1970       2013         43 -1.27     1.34       0.21
```

```
## imdb*              664.95    1       1794       1793  0.00    -1.20      12.23
## title*             656.05    1       1768       1767  0.00    -1.21      12.04
## test*                2.97    1         10          9 -0.52    -0.93       0.06
## clean_test*          1.48    1          5          4 -0.27    -1.56       0.03
## binary*              0.00    1          2          1  0.21    -1.96       0.01
## budget         30393300.00 7000  425000000  424993000  2.04     5.56 1137653.20
## domgross       47480054.47    0  760507625  760507625  2.45     8.95 1906495.20
## intgross       91800310.28  828 2783918982 2783918154  3.54    22.95 4981228.08
## code*                14.83    1         85         84 -1.26     1.27       0.42
## budget_2013    38745808.79 8632  461435929  461427297  1.73     3.95 1296607.47
## domgross_2013  62679691.17  899 1771682790 1771681891  3.96    28.95 2989025.45
## intgross_2013 115351802.68  899 3171930973 3171930074  3.71    23.23 6714127.24
## period_code          1.48    1          5          4  0.47    -0.76       0.03
## decade_code          0.00    1          3          2  0.08    -0.90       0.02
```

To create pragmatic visualizations of financial data, it will be useful to rescale the columns such that the values are displayed in millions of dollars, rather than dollars, which would eliminate multiple zeros on the display. To achieve this, use mathematical operators (e.g., data$column/20) on the columns of interest, and call back the edited columns to their original names. Make sure you do not run this chunk more than once. Consider to yourself, what would happen if these objects were assigned twice?

```
bech_data$budget <- bech_data$budget/1000000
bech_data$domgross <- bech_data$domgross/1000000
bech_data$intgross <- bech_data$intgross/1000000
```

3.2.1 Quick Plots

Now that we are more familiar with our data, it is sometimes useful to quickly visualize data using qplots. The 'q' stands for quick, whereas ggplot is more flexible and customize-able.

There are four key arguments to consider when creating a `qplot`. These are data, where you define your dataset, x and y, where you define which variables will map onto the x and y axes, and geom, which dictates the geometry or type of your plot. Geom options include `point`, `line`, `smooth`, `dotplot`, `boxplot`, `violin`, `histogram`, and `density`.

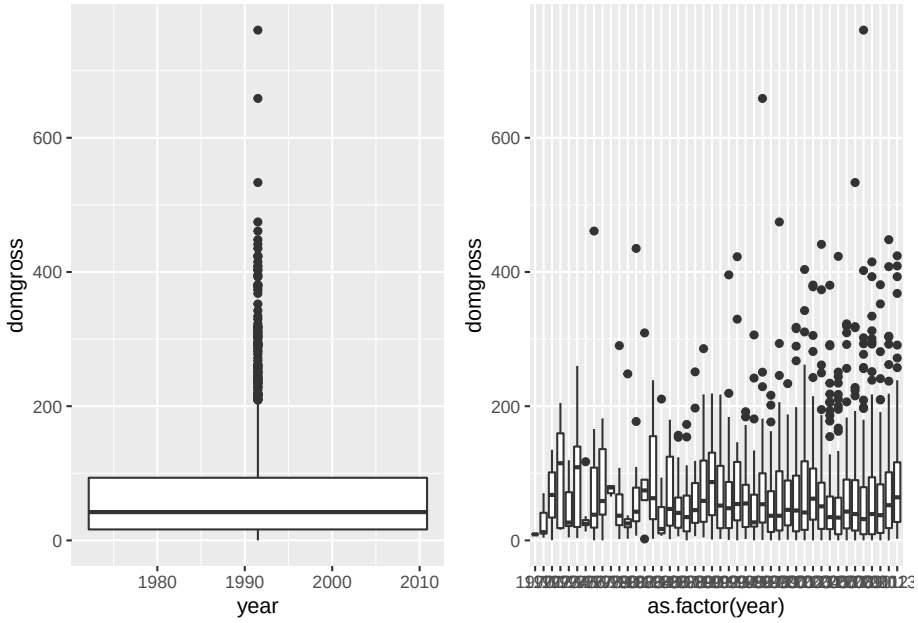
```
  qplot(data = "data_frame", x = "x_variable", y = "y_variable", geom = "whatever_plot_you_want")
```

Boxplots are a focal point in these exercises. Boxplots are like violin plots in that they display the distribution of data. The key difference is that boxplots

include displays of a few descriptive statistics, where violin plots are derived from each individual data point.

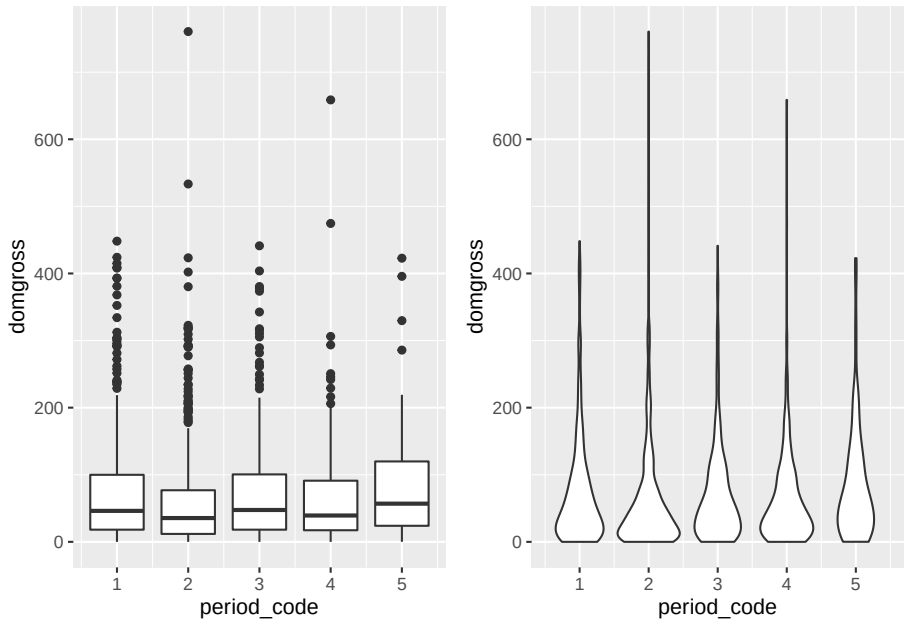
The following code will produce a set of boxplots describing the distributions of movie budgets disaggregated by year. R treats column data, such as the year column, as a single vector. Set the year column as a factor within the x argument using the `as.factor()` function. What does the second plot tell you that the first one fails to communicate?

```
p <- qplot(data = bech_data, x = year, y= domgross, geom = 'boxplot')
p1 <- qplot(data = bech_data, x = as.factor(year), y= domgross, geom = 'boxplot')
ggarrange(p, p1)
```



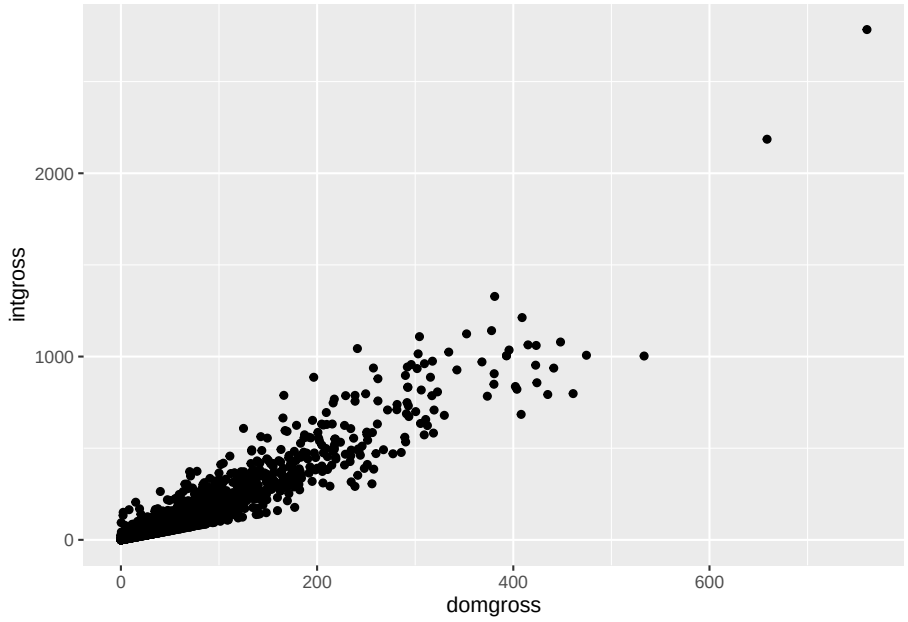
The data were also categorized into five period codes to provide time-based groups. Let's group the domestic gross values by period codes to compare the distributions of film's earnings over time. Also, here is a chance to directly compare visualization of descriptive data using boxplots and violin plots.

```
p2 <- qplot(data = bech_data, x = period_code, y = domgross, group = period_code, geom
p3 <- qplot(data = bech_data, x = period_code, y = domgross, group = period_code, geom
ggarrange(p2, p3)
```

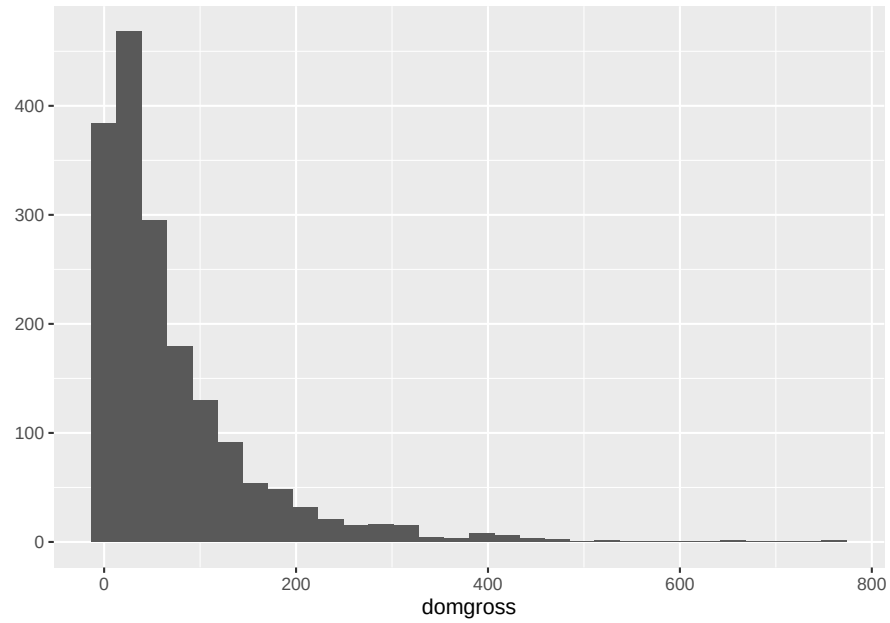


If you are not sure which plot you need to use, you might rely on qplot's assumptions. If you include both x and y arguments, qplot will assume you want a scatterplot. If only the x variable is provided, a histogram is assumed. If only the y variable is provided, a scatter plot will be created where the x axis will represent each individual case in a sequence. Additionally, you can add multiple geoms using a vector of geoms instead of a single geom argument (e.g., line four in this code chunk).

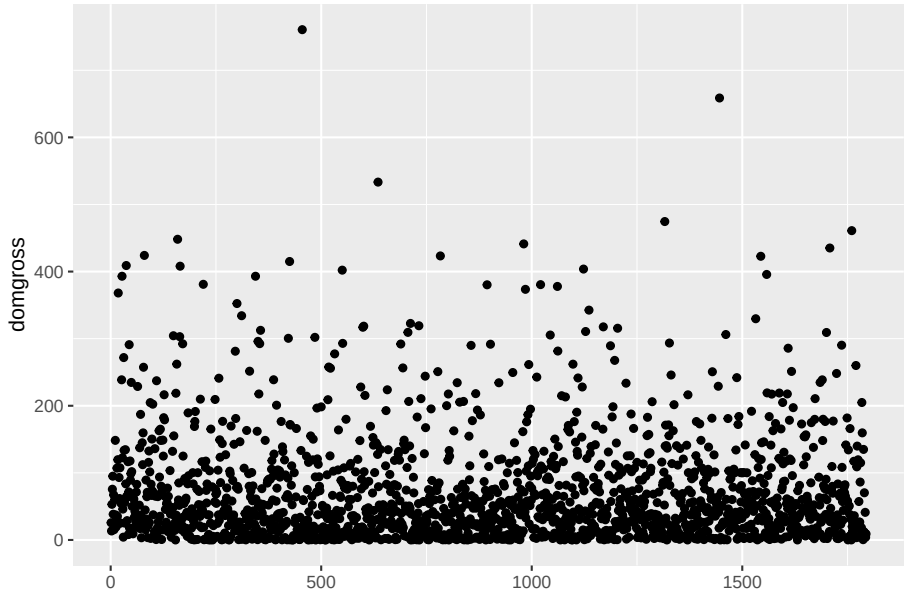
```
qplot(x = domgross, y = intgross, data = bech_data) #scatterplot
```



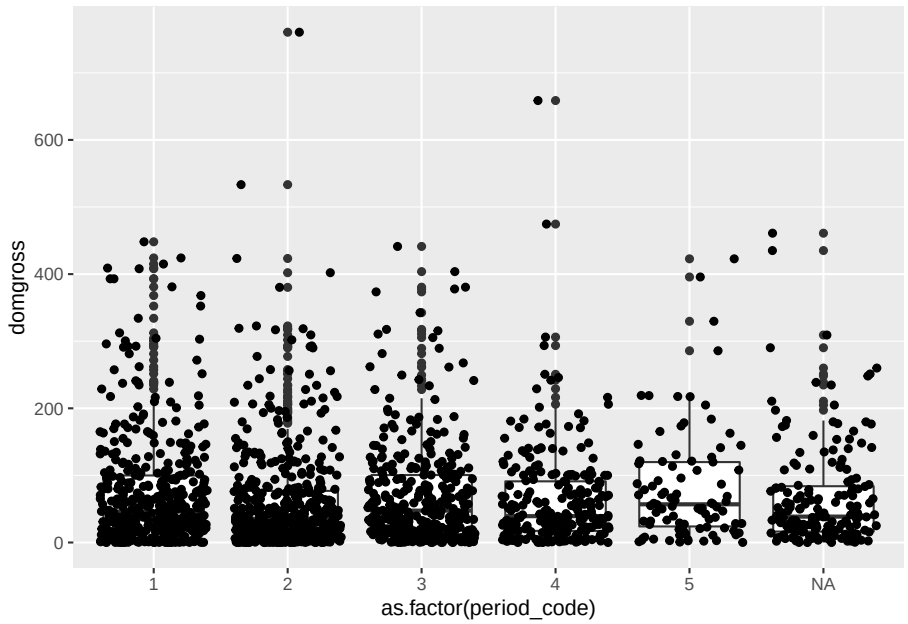

```
qplot(x = domgross, data = bech_data) #histogram
```



```
qplot(y = domgross, data = bech_data) #casewise univariate scatterplot
```




```
qplot(as.factor(period_code), domgross, data = bech_data, geom = c("boxplot", "jitter"))
```



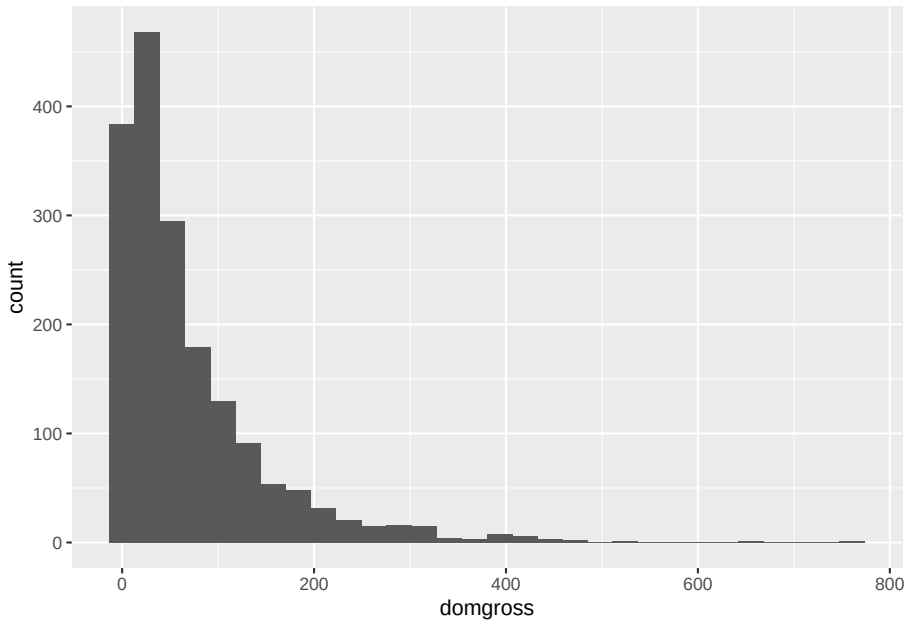
3.2.2 Plotting with ggplot

Onto `ggplot`. We have much more flexibility within the `ggplot` framework, mostly due to a focus on additive layers. To start, make the same violin plot for the Bechdel data period codes using ggplot. The `qplot` code is commented out below. The `aes()` function suggests aesthetic considerations to the plotting function in the context of the data that are loaded in. The geom is added as an additional layer, and itself is a function rather than an argument.

```
#qplot(data = bech_data, x = period_code, y = domgross, group = period_code, geom = 'v
ggplot(data = bech_data, aes(x = period_code, y = domgross, group = period_code)) +
  geom_violin()
```

Using ggplot, we can dive deep into histograms and density plots. With `ggplot`, these particular visualizations are most effective when they provide insight at a glance for the distributions of data across several groups. To start, let's create a histogram of the domestic gross revenue. Like before, we only need an x variable for histograms.

```
ggplot(data = bech_data, aes(x = domgross)) +
  geom_histogram()
```



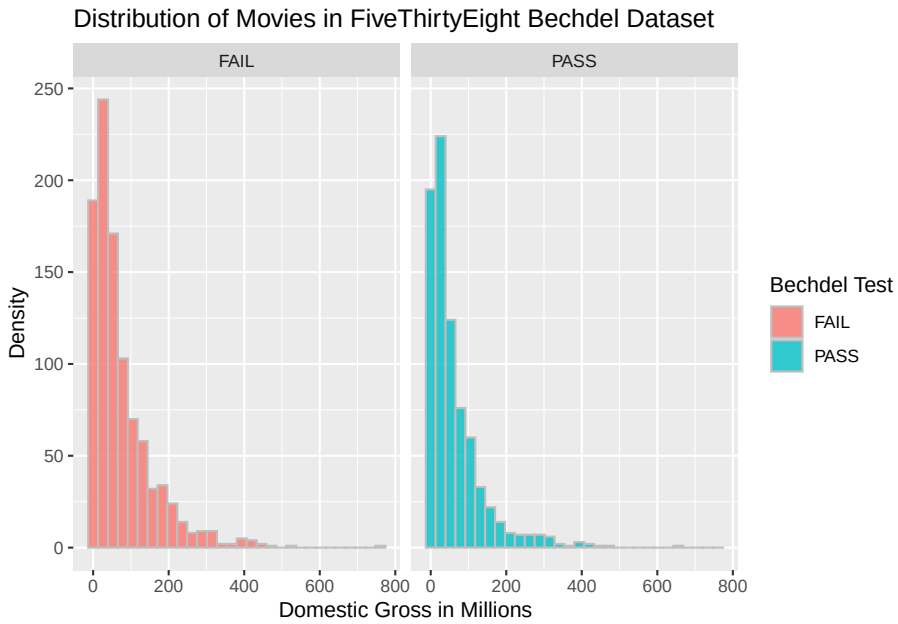
3.2.2.1 Using Layers

Now let's add some bells and whistles using ggplot's layers-based coding to make this visualization more readable and appealing. Remember the plus sign creates a new layer which culminates in the final plot.

There are several considerations for this plot which can be used to customize other data. The fill argument within the `aes()` function indicates that cases should be filled in based on their pass/fail status, which was represented by the `binary` variable. The alpha argument in `geom_histogram()` function adjusts the opacity of the filled colors. ggtitle adds the title, and the `xlab()` and `ylab()` functions create labels on each axis. The `labs()` function creates a legend title, seen at the bottom right of the figure. Lastly, the `facet_wrap()` function creates multiple plots based on the status of a called variable, which is in this case the binary pass/fail variable.

Try running each line of this code chunk independently, adding on a layer each time to see how this code creates the final product.

```
ggplot(data = bech_data, aes(x = domgross, fill = binary)) +
  geom_histogram(alpha = 0.8, colour = 'grey') +
  ggtitle("Distribution of Movies in FiveThirtyEight Bechdel Dataset") +
  xlab('Domestic Gross in Millions') +
  ylab('Density') +
  labs(fill = 'Bechdel Test') +
  facet_wrap(~ binary)
```

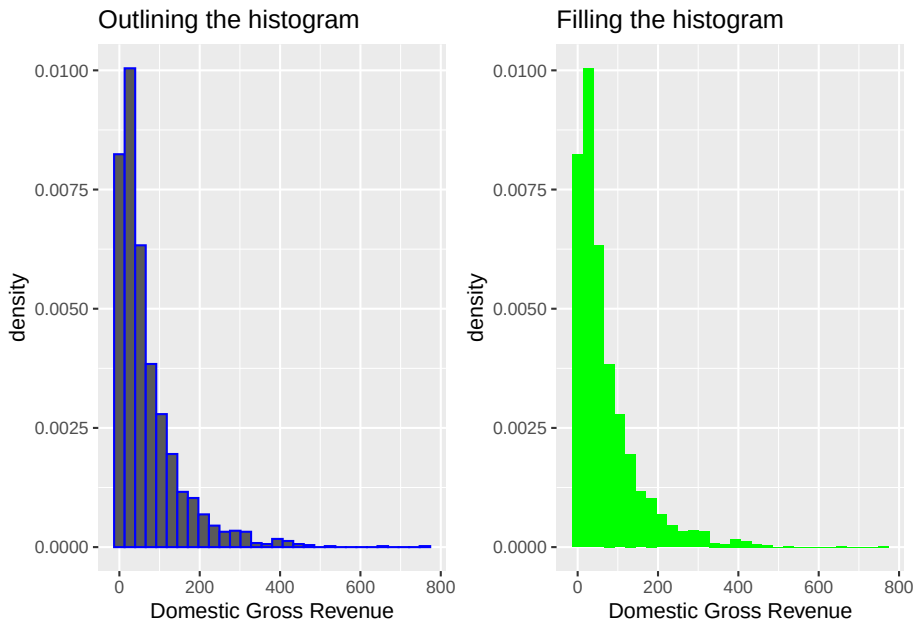


You can specify the color (outline) and fill color of plots. You can also assign ggplots to objects. The `ggpubr` function `ggarrange()` allows you to easily present multiple plots in the same output.

```
color <- ggplot(data = bech_data, aes(x = domgross, y = ..density..)) +
  geom_histogram(color = 'blue') +
  ggtitle("Outlining the histogram") +
  xlab("Domestic Gross Revenue")

fill <- ggplot(data = bech_data, aes(x = domgross, y = ..density..)) +
  geom_histogram(fill = 'green') +
  ggtitle("Filling the histogram") +
  xlab("Domestic Gross Revenue")

ggarrange(color, fill)
```

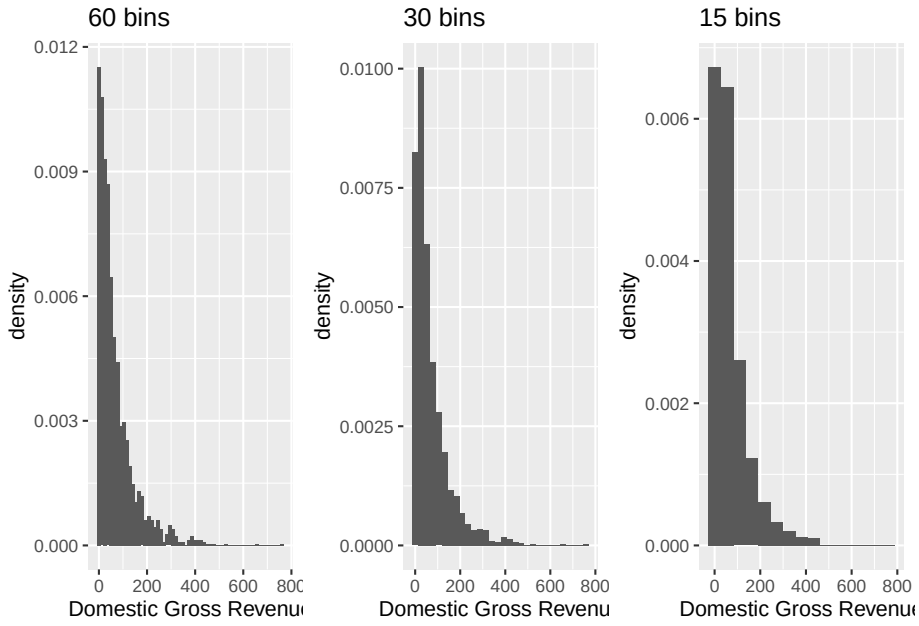


You can also vary the number of bins in each histogram. Run the following chunk to see the implications of the `bin` argument. The more bins there are, the more specific the visualization.

```
# Vary the number of bins per histogram
bin60 <- ggplot(data = bech_data, aes(x = domgross, y = ..density..)) +
  geom_histogram(bins = 60) +
  ggtitle("60 bins") +
  xlab("Domestic Gross Revenue")
```

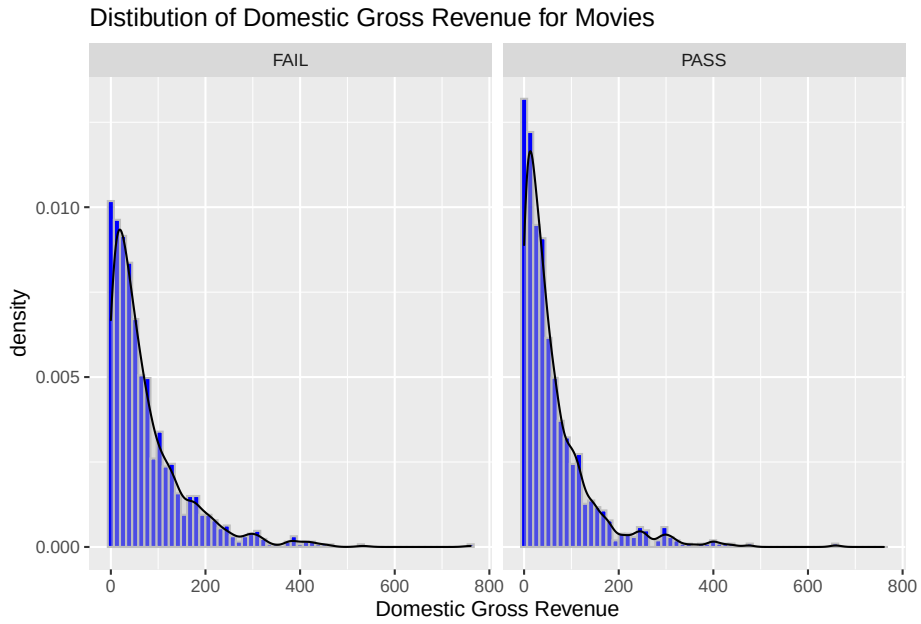
```
bin30 <- ggplot(data = bech_data, aes(x = domgross, y = ..density..)) +
  geom_histogram(bins = 30) +
  ggtitle("30 bins") +
  xlab("Domestic Gross Revenue")
bin15 <- ggplot(data = bech_data, aes(x = domgross, y = ..density..)) +
  geom_histogram(bins = 15) +
  ggtitle("15 bins") +
  xlab("Domestic Gross Revenue")

ggarrange(bin60, bin30, bin15, ncol = 3)
```



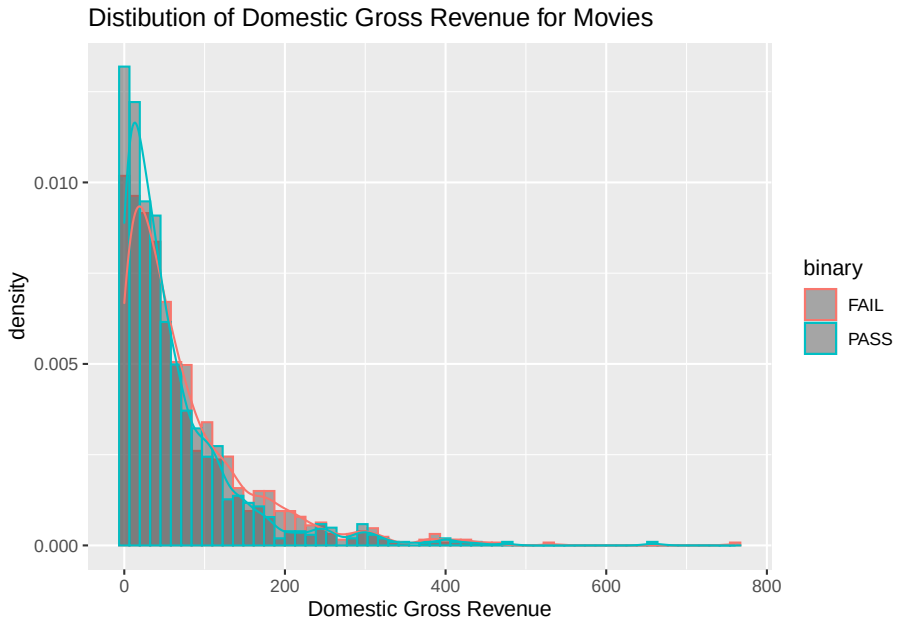
The density plot is a variation of the histogram, where values in columns are smoothed to be equally distributed allowing for the overall distribution of the data to become easy to assess. You can use `geom_density()` to add a density plot on top of a histogram.

```
# Add a density plot
ggplot(data = bech_data, aes(x = domgross, y = ..density..)) +
  geom_histogram(bins = 60, color = 'grey', fill = 'blue') +
  ggtitle("Distibution of Domestic Gross Revenue for Movies") +
  xlab("Domestic Gross Revenue") +
  geom_density(alpha = .4, fill = 'grey') +
  facet_wrap(~ binary)
```



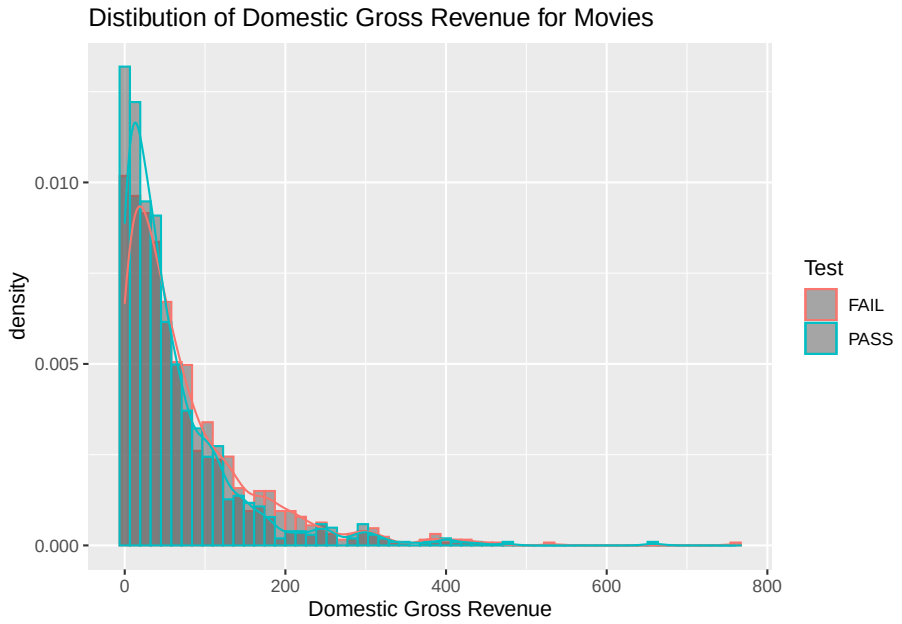
You can also create histograms to compare multiple groups within a single plot. Try switching the `color` argument with the fill argument, keeping the object binary in place. See if you can apply what you have learned so far to improve the appeal of the plot.

```
ggplot(data = bech_data, aes(x = domgross, y = ..density.., color = binary)) +
  geom_histogram(position = "identity", bins = 60, alpha = .5) +
  ggtitle("Distibution of Domestic Gross Revenue for Movies") +
  xlab("Domestic Gross Revenue") +
  geom_density(alpha = .4)
```



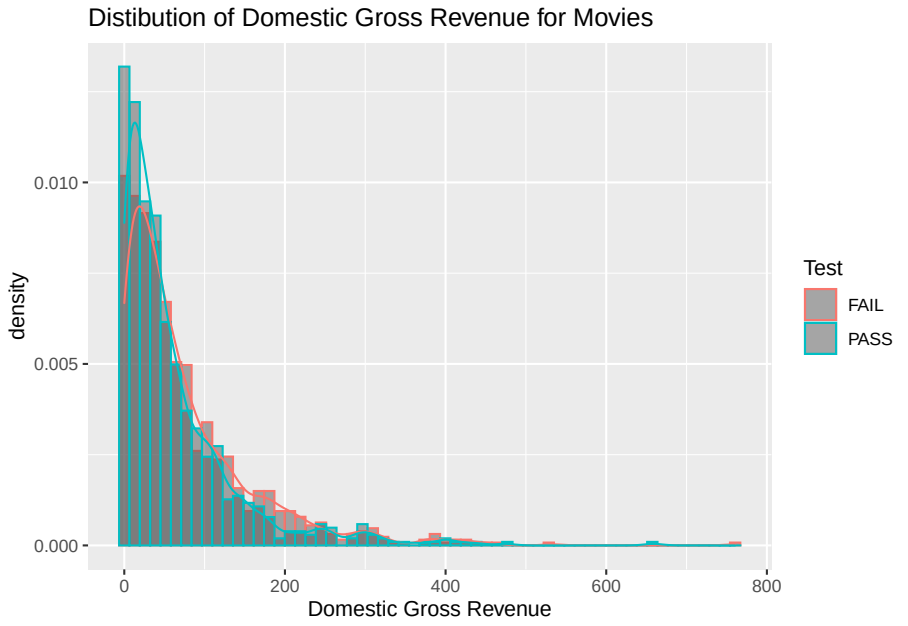
The function `scale_color_discrete()` also allows you to change the legend title.

```
# Change legend title
ggplot(data = bech_data, aes(x = domgross, y = ..density.., color = binary)) +
  geom_histogram(position = "identity", bins = 60, alpha = .5) +
  ggtitle("Distibution of Domestic Gross Revenue for Movies") +
  xlab("Domestic Gross Revenue") +
  geom_density(alpha = .4) +
  scale_color_discrete(name = "Test")
```



Use `ggsave()` and set the desired filename, size, dpi, and other parameters for saving your plot.

```
# Save your plot
ggplot(data = bech_data, aes(x = domgross, y = ..density.., color = binary)) +
  geom_histogram(position = "identity", bins = 60, alpha = .5) +
  ggtitle("Distibution of Domestic Gross Revenue for Movies") +
  xlab("Domestic Gross Revenue") +
  geom_density(alpha = .4) +
  scale_color_discrete(name = "Test")
```



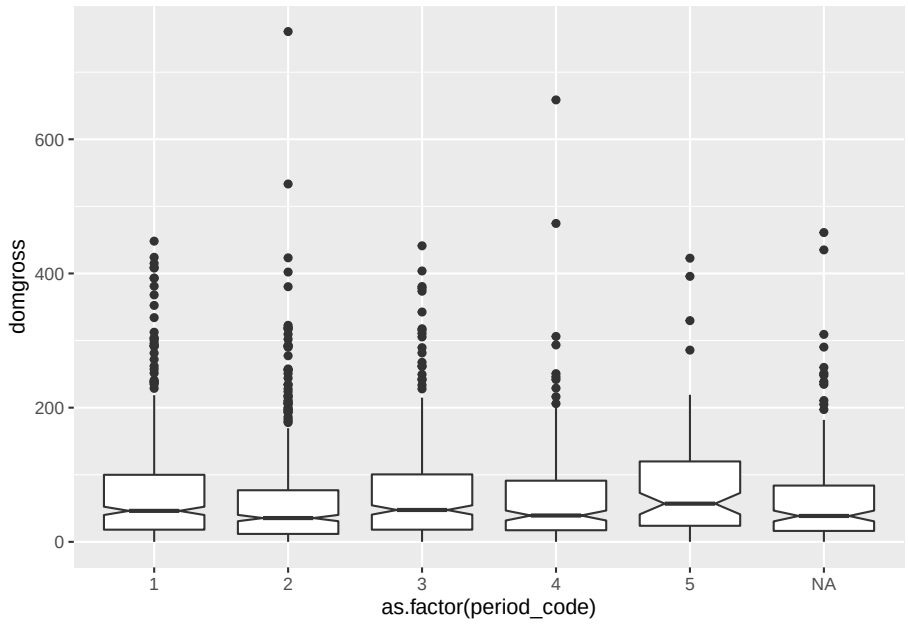

```
ggsave("Hist_Dens.png", width = 5, height = 5)
```

3.2.2.2 Other Useful Functions and Arguments

Let's create some more box plots and violin plots using the period code column. As these plots are used to compare distributions of data, they are especially useful for comparing different groups of continuous data. You can use the `mutate()` function to set period_code as a factor, as to make your ggplot coding simpler and cleaner. In this iteration, let's set try a notched box plot, which emphasize the median with notches.

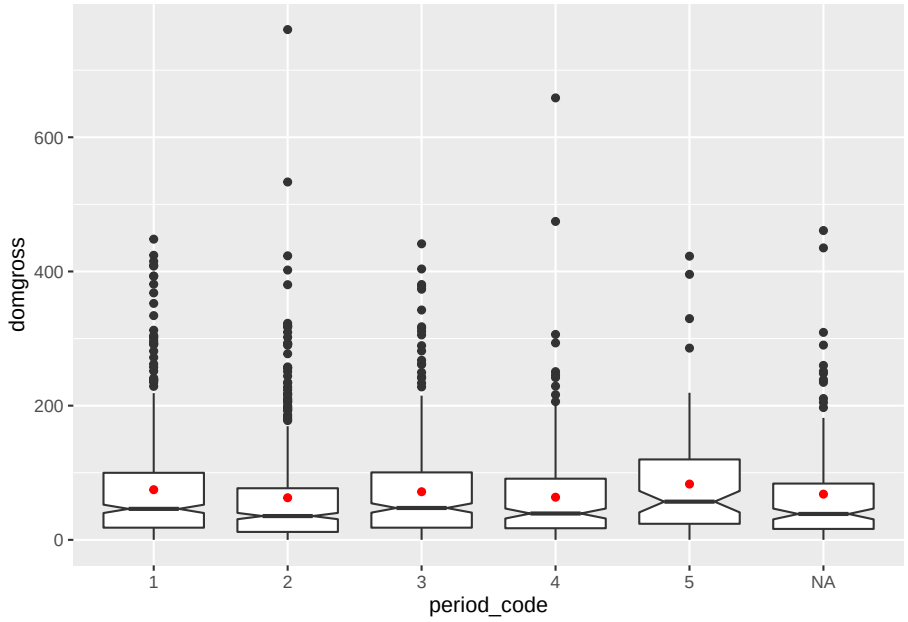
```
bech_data <- bech_data %>% mutate(period_code = as.factor(period_code))

ggplot(data = bech_data, aes(x = as.factor(period_code), y = domgross)) +
  geom_boxplot(notch = T)
```



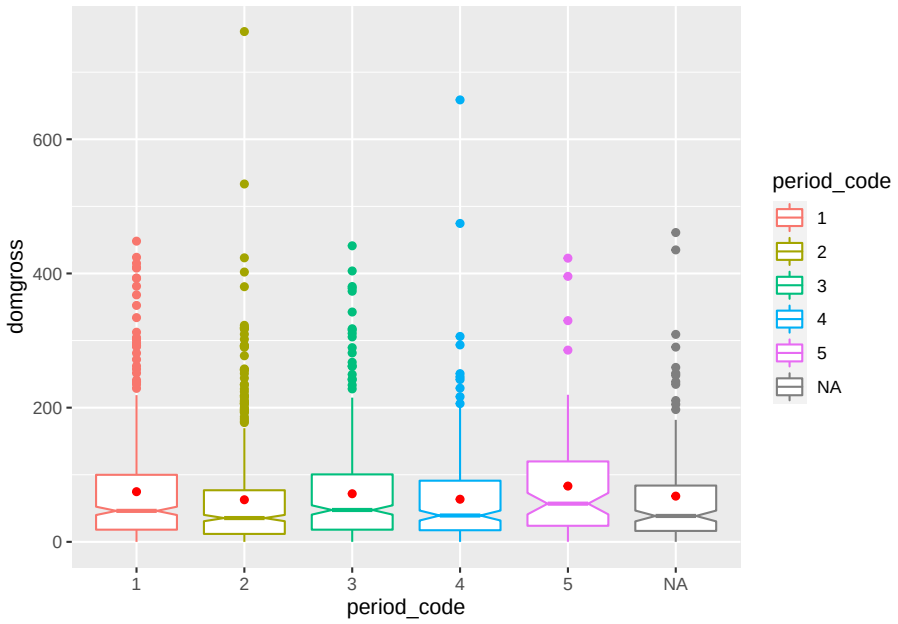
You can add a dot to represent the mean domestic revenue for each group of movies using `stat_summary(fun.y = mean, geom = "point", color = "anycolor")`

```
ggplot(data = bech_data, aes(x = period_code, y = domgross)) +
  geom_boxplot(notch = T) +
  stat_summary(fun.y = mean, geom = "point", color = 'red')
```



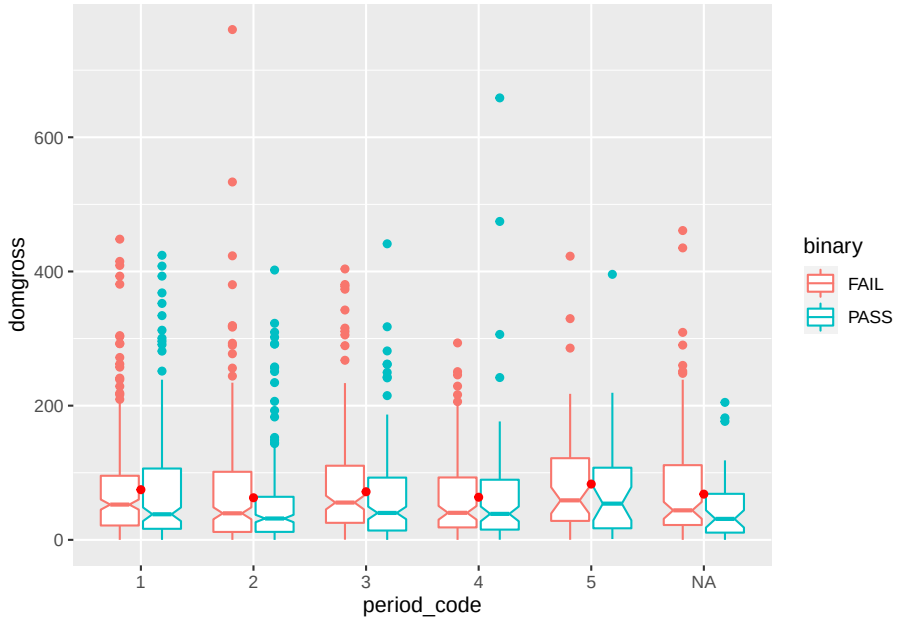
Assign period_code to the argument `color` to assign each period code a unique color.

```
ggplot(data = bech_data, aes(x = period_code, y= domgross, color = period_code)) +
  geom_boxplot(notch = T) +
  stat_summary(fun.y=mean, geom="point", color = 'red')
```



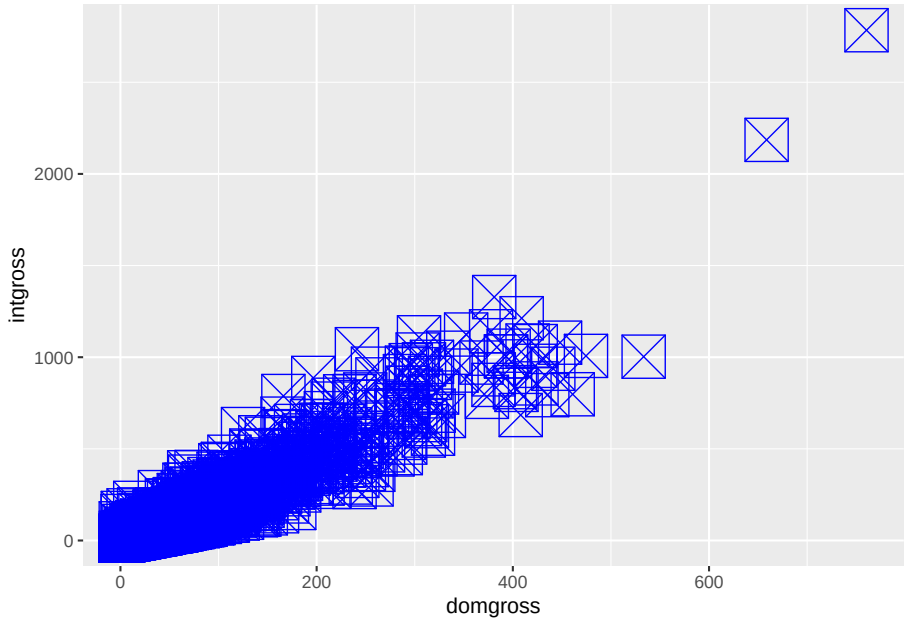
You can create make group comparisons using box plots. Use the `aes()` function to specify the group using the `color` or `fill` argument, and the variables of interest using the `x` and `y` arguments. Instead of outlining this boxplot in color, fill the boxplot in color. Add a legend title. Use the layer "scale_fill_discrete" when "fill" is used in the aes. Use the layer "scale_color_discrete" when "color" is used in the aes.

```
ggplot(data = bech_data, aes(x = period_code, y = domgross, color = binary)) +
  geom_boxplot(notch = TRUE) +
  stat_summary(fun.y=mean, geom="point", color = 'red')
```



You can change the size, shape, and color of points in your scatter plot.

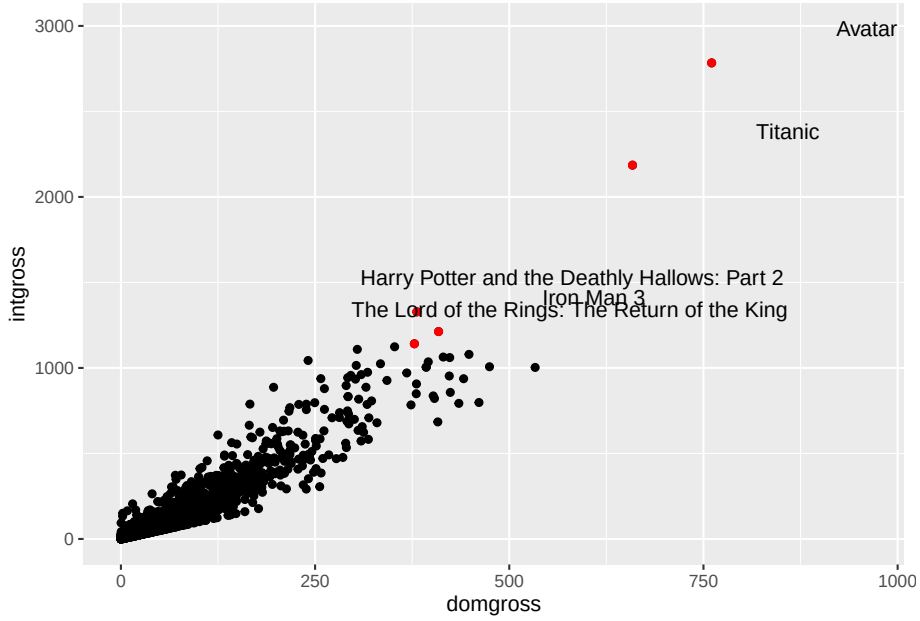
```
ggplot(data = bech_data, aes(x = domgross, y = intgross)) +
  geom_point(size = 10, shape = 7, color = 'blue')
```



To label the points that represent top five movies based on international gross revenue, first arrange the international gross revenue column in descending order, using the `slice()` function to limit the dataframe to the top five movies. Then, add an addition `geom_point()` layer calling the topfive data, as shown below. `nudge_x` and `nudge_y` allow you to move the labels in reference to the datapoint.

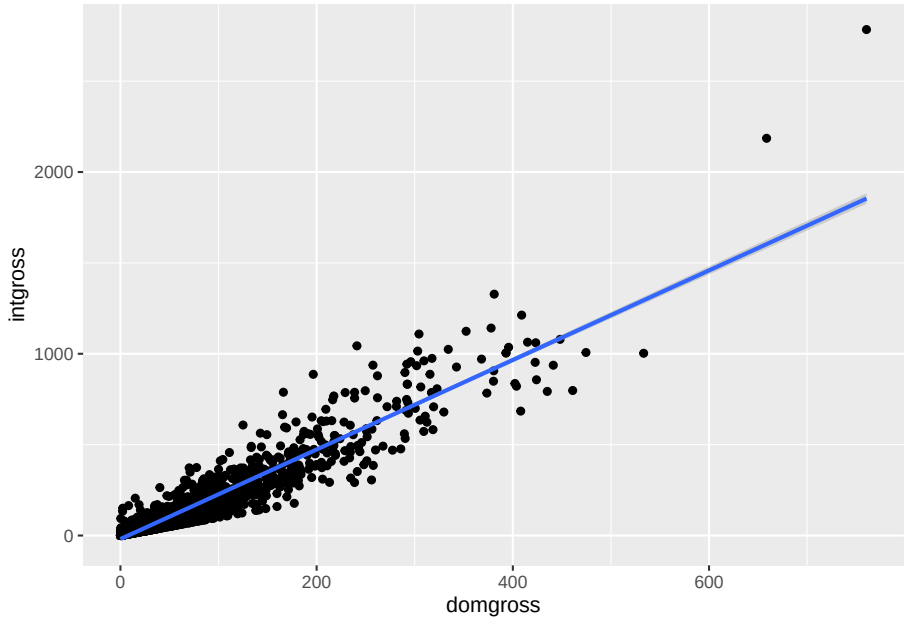
```
topfive <- bech_data %>%
  arrange(desc(intgross)) %>%
  slice(1:5)

ggplot(data = bech_data, aes(x = domgross, y = intgross)) +
  geom_point() +
  geom_point(data = topfive, aes(x = domgross, y = intgross), color = 'red') +
  geom_text(data = topfive, label = topfive$title, nudge_x = 200, nudge_y = 200)
```



Add a regression line to any scatter plot with `geom_smooth(method = 'lm')`

```
ggplot(data = bech_data, aes(x = domgross, y = intgross)) +
  geom_point() +
  geom_smooth(method = 'lm')
```



You can also display scatter plots by groups, distinguished by color or shape, as seen below. See if you can add a regression line to one of these plots.

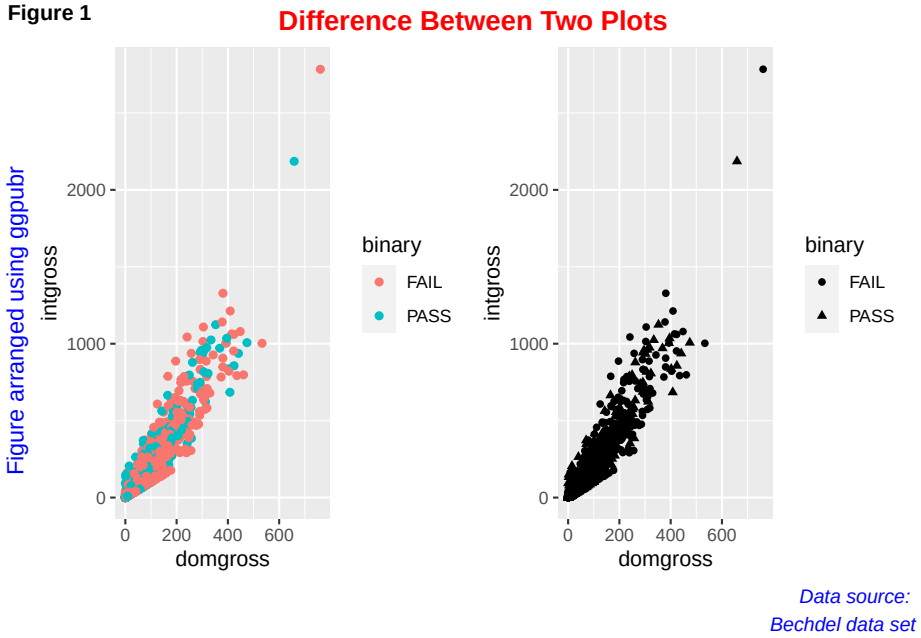
```
# Scatter plot by group
plot1 <- ggplot(data = bech_data, aes(x = domgross, y = intgross, color = binary)) +
  geom_point()

plot2 <- ggplot(data = bech_data, aes(x = domgross, y = intgross, shape = binary)) +
  geom_point()

plot3 <- ggarrange(plot1, plot2, ncol=2)

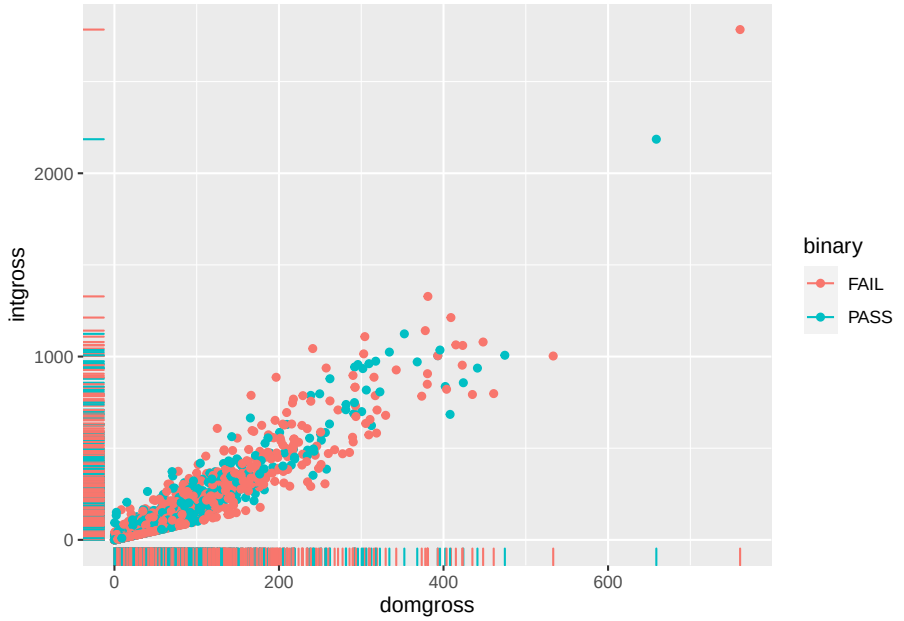
annotate_figure(plot3,
  top = text_grob("Difference Between Two Plots", color = "red", face = "bold", size = 14),
  bottom = text_grob("Data source: \n Bechdel data set", color = "blue", hjust = 1, x = 1, face =
  left = text_grob("Figure arranged using ggpubr", color = "blue", rot = 90), fig.lab = "Figure 1
```

Figure 1



You can also add rugs, or lines across the x-axis that are tied to single points of data, to the plot using `geom_rug()`

```
# Add rugs to scatter plot
ggplot(data = bech_data, aes(x = domgross, y = intgross, color = binary)) +
  geom_point() +
  geom_rug()
```



`ggedit()` allows you to create plots with an interface for editing aspects of each layer.

```
#install.packages('ggedit')
library(ggedit)
p <- ggplot(data = bech_data, aes(x = domgross, y = intgross, color = binary)) + geom_point()
ggedit(p)
```

3.3 Case Study

This case study was derived from a real-life data visualization problem. The resulting visualizations were eventually published in an academic journal. This case study does not exemplify how to make the "right" choices for data visualization, but is written to demonstrate how data visualizations can drive and change a research process. With this in mind, it is beneficial for the R user to be consistently interpreting their findings visually.

A researcher was conducting a content analysis where they found 10 themes in a text-based corpus. The researcher has quantified the number of topics across fourteen years (1958-1972), and needed to find an effective way to present this information for potential publication.

If you are following along in R, install the necessary packages for this case study.

```
#install.packages('gifski')
#install.packages('gganimate')
#install.packages('transformr')
#install.packages('janitor')
#install.packages('stringr')
#install.packages('forcats')
#install.packages('lubridate')
#install.packages('ggrepel')
```

Then load the libraries and dataset. This dataset is available at https://osf.io/6jb9t/ under the Data Visualization folder. The dataset includes three columns—a topic name designation, a year between 1958 and 1972, and the number of times that topic occurred in that year.

```
library(stringr)
library(gganimate)
library(gifski)
library(transformr)
library(janitor)
library(lubridate)
library(ggrepel)

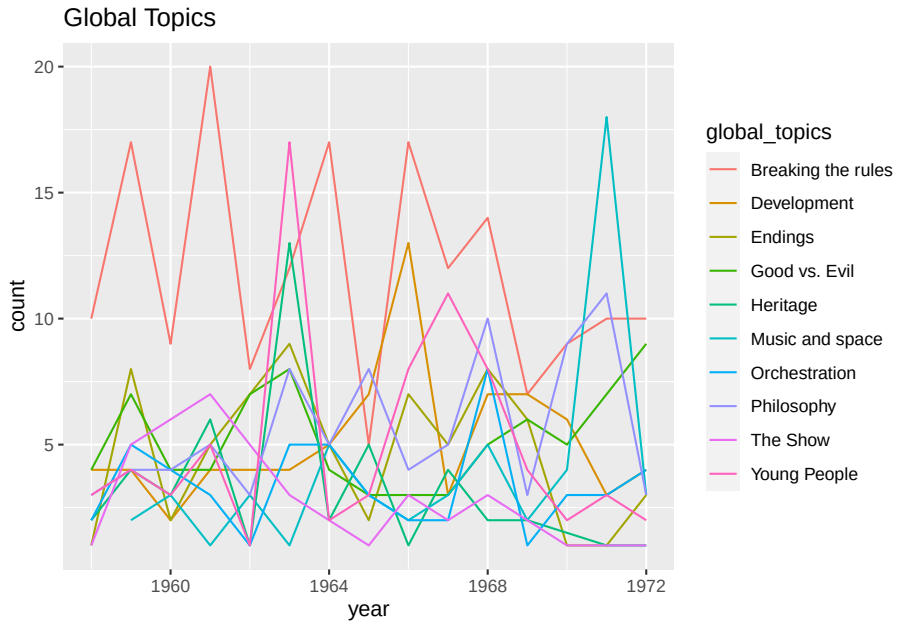
global <- read_csv("globalperyear.csv")
```

The researcher had a meeting with coauthors, where it was decided to change the names of six of the ten topics. The `grepl` package function, `str_replace_all()` was used to update each instance of the incorrect topic labels in the appropriate column.

```
global$global_topics <- str_replace_all(global$global_topics, 'Themes', 'Development')
global$global_topics <- str_replace_all(global$global_topics, 'Stories in music', 'Phil
global$global_topics <- str_replace_all(global$global_topics, 'Musical choices', 'Orche
global$global_topics <- str_replace_all(global$global_topics, 'Musical heritage', 'Heri
global$global_topics <- str_replace_all(global$global_topics, 'Young people in music hi
global$global_topics <- str_replace_all(global$global_topics, 'The show', 'The Show')
```

Since the visualization presenting the counts of topics over time, the researcher tries a line graph to display the topics.

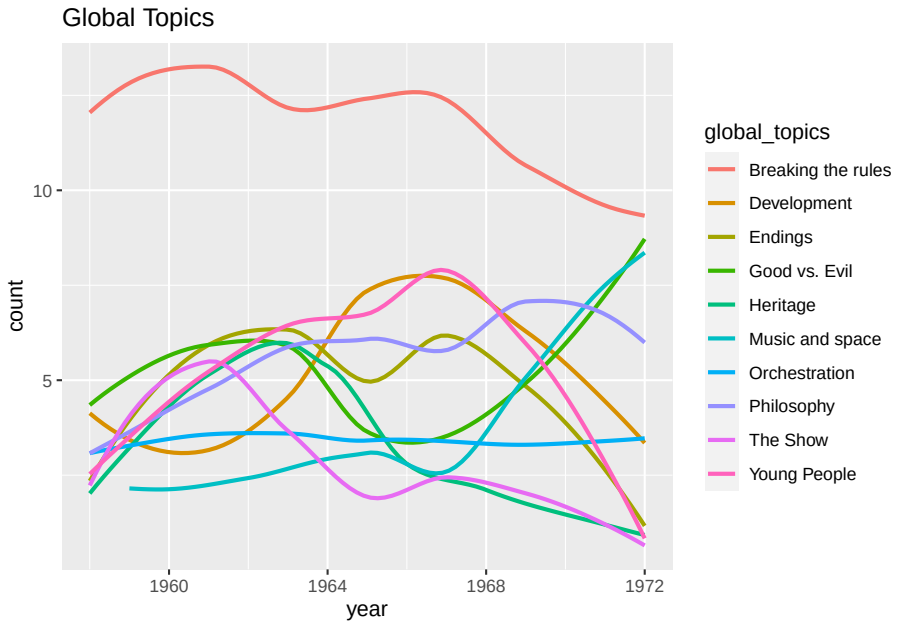
```
ggplot(global, aes(x = year, y = count, color = global_topics)) +
  geom_line() +
  labs(
    title = paste("Global Topics")
  )
```



This plot was chaotic and difficult to read, so the researcher tried the geom_smooth() function as an additional layer. Set the `se` argument to `FALSE` to avoid extra noise in the plot. FYI, the `se` argument, when true, plots the standard error of each smoothed distribution, which in this case tended to be quite large.

```
ggplot(global, aes(x = year, y = count, color = global_topics)) +
  geom_smooth(se = FALSE) +
  labs(title = paste("Global Topics")
  )
```

```
## `geom_smooth()` using method = 'loess' and formula 'y ~ x'
```



This output was still a bit chaotic, so the researcher decided to present ten small plots in one figure instead of one large plot. A new dataset needs to be created for each small plot, which is done using the filter() function.

```
rules <- global %>%
  filter(global_topics == "Breaking the rules")
development <- global %>%
  filter(global_topics == "Development")
show <- global %>%
  filter(global_topics == "The Show")
space <- global %>%
  filter(global_topics == "Music and space")
philosophy <- global %>%
  filter(global_topics == "Philosophy")
goodvsevil <- global %>%
  filter(global_topics == "Good vs. Evil")
endings <- global %>%
  filter(global_topics == "Endings")
heritage <- global %>%
  filter(global_topics == "Heritage")
young <- global %>%
  filter(global_topics == "Young People")
Orchestration <- global %>%
  filter(global_topics == "Orchestration")
```

Ten small plots were created, each calling the unique datasets. The `coord_cartesian()` layer was also added. This function sets a limit to the x and y-axes (in this case the limit was 0, 20). This layer allowed for the plots to be framed without additional white space.

```
rulesplot <- ggplot(rules, aes(x = year, y = count)) +
  geom_smooth(aes(), se = TRUE) +
  coord_cartesian(ylim = c(0, 20)) +
  labs(
    title = paste("Breaking the Rules")
  )

developmentplot <- ggplot(development, aes(x = year, y = count)) +
  geom_smooth(aes(), se = TRUE) +
  coord_cartesian(ylim = c(0, 20)) +
  labs(
    title = paste("Development")
  )

showplot <- ggplot(show, aes(x = year, y = count)) +
  geom_smooth(aes(), se = TRUE) +
  coord_cartesian(ylim = c(0, 20)) +
  labs(
    title = paste("The Show")
  )

spaceplot <- ggplot(space, aes(x = year, y = count)) +
  geom_smooth(aes(), se = TRUE) +
  coord_cartesian(ylim = c(0, 20)) +
  labs(
    title = paste("Music and Space")
  )

philosophyplot <- ggplot(philosophy, aes(x = year, y = count)) +
  geom_smooth(aes(), se = TRUE) +
  coord_cartesian(ylim = c(0, 20)) +
  labs(
    title = paste("Philosophy")
  )

goodvsevilplot <- ggplot(goodvsevil, aes(x = year, y = count)) +
  geom_smooth(aes(), se = TRUE) +
  coord_cartesian(ylim = c(0, 20)) +
  labs(
    title = paste("Good vs. Evil")
  )
```

```
endingsplot <- ggplot(endings, aes(x = year, y = count)) +
  geom_smooth(aes(), se = TRUE) +
  coord_cartesian(ylim = c(0, 20)) +
  labs(
    title = paste("Endings")
  )

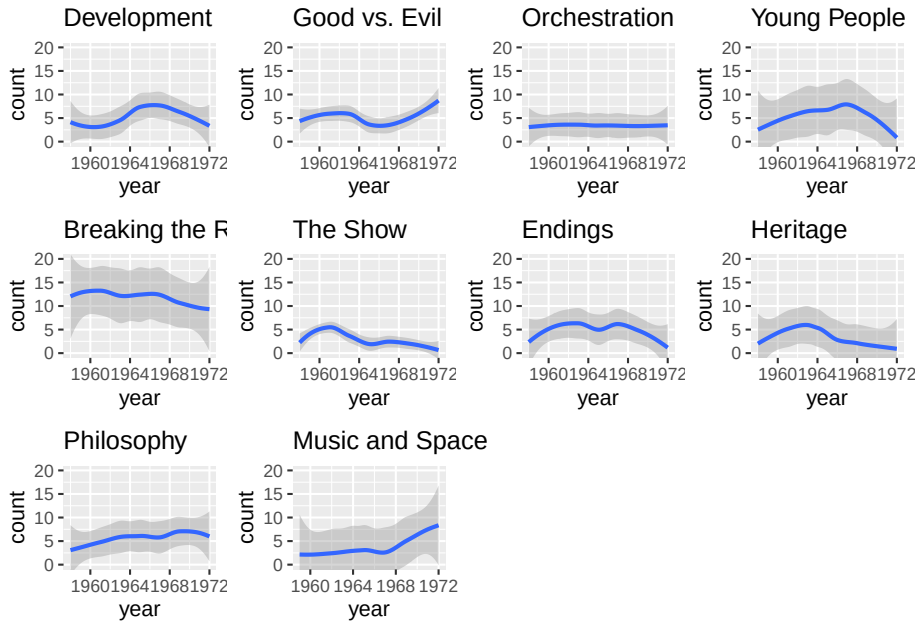
heritageplot <- ggplot(heritage, aes(x = year, y = count)) +
  geom_smooth(aes(), se = TRUE) +
  coord_cartesian(ylim = c(0, 20)) +
  labs(
    title = paste("Heritage")
  )

youngplot <- ggplot(young, aes(x = year, y = count)) +
  geom_smooth(aes(), se = TRUE) +
  coord_cartesian(ylim = c(0, 20)) +
  labs(
    title = paste("Young People")
  )

orchestrationplot <- ggplot(Orchestration, aes(x = year, y = count)) +
  geom_smooth(aes(), se = TRUE) +
  coord_cartesian(ylim = c(0, 20)) +
  labs(
    title = paste("Orchestration")
  )
```

Then, the researcher used the `ggarrange()` function to display each of the plots together at once.

```
ggarrange(developmentplot, goodvsevilplot, orchestrationplot, youngplot, rulesplot, sho
```



There was an opportunity to publish a digital file that includes animated data along with the manuscript. The researcher then used the following code to create an animated gif using the same dataset. The last two lines of code are commented out as they would not render in this document, but you can try it for yourself with the same code.

```
#install.packages('plotly')
library(plotly)
```

```
##
## Attaching package: 'plotly'

## The following object is masked from 'package:sentimentr':
##
##     highlight

## The following object is masked from 'package:ggplot2':
##
##     last_plot

## The following object is masked from 'package:stats':
##
##     filter
```

```
## The following object is masked from 'package:graphics':
##
##     layout
```

```
p <- global %>%
  arrange(count) %>%
  mutate(topics = factor(global_topics, levels=c("The Show", "Young People", "Music and
  ggplot(global, mapping = aes(x = topics, y = count)) +
  geom_histogram(stat = 'identity', aes(fill = topics)) +
  coord_flip() +
  labs(title = 'Year: {frame_time}',
       x = 'Global Categories',
       y = "Category Mentions",
       caption="Global Categories: Trends Over Time") +
  transition_time(year) +
  ease_aes('linear')

#animate(p, nframes = 600, fps = 24, width = 600, height = 400, end_pause = 30)
#p
```

3.4 Review

In this chapter we reviewed some common data visualizations and practices for creating several visualizations in R. For more examples, check out ggplot's gallery for fantastic and inspiring visualizations at this link: [<https://exts.ggplot2.tidyverse.org/gallery>](https://exts.ggplot2.tidyverse.org/gallery/). To make sure you understand this material, there is a practice assessment to go along with this chapter at https://jayholster.shinyapps.io/DataVisualizationAssessment/

3.5 References

Firke, S. (2021) janitor. https://cran.r-project.org/web/packages/janitor/index.html

Grolemund, G., & Wickham, H. (2011). Dates and times made easy with lubridate. *Journal of Statistical Software, 40*(3), 1–25. https://www.jstatsoft.org/v40/i03/.

Holster, J. D. (2022) Music's Monarch Speaks: A Content Analysis of Leonard Bernstein's Young People's Concerts, *Qualitative Research in Music Education, 4*(1), 62–104. https://vpa.uncg.edu/wp-content/uploads/2022/02/QRME-v4-i1-R2.pdf

Iturbide, M., Bedia, S., Herrera, J., Baño-Medina, J. Fernández, M.D., Frías, R. Manzanas, D., San-Martín, E., Cimadevilla, A.S., Cofiño, A.S., & Gutiérrez, J.M (2019) The R-based climate4R open framework for reproducible climate data access and post-processing. *Environmental Modelling & Software*, *111*, 42-54. https://[doi.org](https://%5Bdoi.org){.uri} /10.1016/j.envsoft.2018.09.009

Kassambara, A., (2020). ggpubr. https://cran.r-project.org/web/packages/ggpubr/index.html

Kim A.Y., Ismay C., & Chunn, J. (2018). "The fivethirtyeight R Package: 'Tame Data' Principles for Introductory Statistics and Data Science Courses." *Technology Innovations in Statistics Education*, *11*. <https://escholarship.org/uc/item/0rx1231m>

Lyttle I (2021). *vembedr: Embed Video in HTML*. R package version 0.1.5, https://github.com/ijlyttle/vembedr.

Mindard, C. J. (1869). Napolean's disastrous Russian campaign of 1812. https://www.masswerk.at/minard/

Mullen, Z., Wiens, A., Vance, E., & Lovejoy, H. (n.d.) Understanding the African diaspora: Modeling the fall of the Oyo empire. https://www.colorado.edu/lab/lisa/

Patil, I. (2021). Visualizations with statistical details: The 'ggstatsplot' approach. *Journal of Open Source Software*, *6*(61), 3167. [https://doi.org/[10.21105/joss.03167](https://doi.org/[10.211](10.21105/joss.03167)

Revelle, W. (2017) psych: Procedures for Personality and Psychological Research, Northwestern University, Evanston, Illinois, USA, <https://CRAN.R-project.org/package=psych> Version = 1.7.8.

Ooms, J. (2022). gifski. https://cran.r-project.org/web/packages/gifski/index.html

Pederson, T. L. (2020). gganimate. https://cran.r-project.org/web/packages/gganimate/gganimate.pdf

Sievert, C. (2020). *Interactive Web-Based Data Visualization with R, plotly, and shiny*. Chapman and Hall/CRC. ISBN 9781138331457, <https://plotly-r.com>.

Sidi, J. (2020). ggedit. https://cran.rstudio.com/web/packages/ggedit/index.html

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L.D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T.L., Miller, E., Bache, S.M., Müller, K., Ooms, J., Robinson, D., Seidel, D.P., Spinu, V., Takahashi, K., Vaughan, D., Wilke, C., Woo, K., & Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software, 4*(43), 1686. https://doi.org/10.21105/joss.01686.

3.5.1 R Short Course Series

Video lectures of each guidebook chapter can be found at https://osf.io/6jb9t/. For this chapter, find the follow the folder path Data visualization in R -> AY 2021-2022 Spring and access the video files, r markdown documents, and other materials for each short course.

3.5.2 Acknowledgements

This guidebook was created with support from the Center for Research Data and Digital Scholarship and the Laboratory for Interdisciplinary Statistical Analaysis at the University of Colorado Boulder, as well as the U.S. Agency for International Development under cooperative agreement #7200AA18CA00022. Individuals who contributed to materials related to this project include Jacob Holster, Eric Vance, Michael Ramsey, Nicholas Varberg, and Nickoal Eichmann-Kalwara.

Chapter 4

Foundations of Statistical Analysis

Quantitative methods suggest that rules of probability can determine whether observed differences within or between groups in a sample represents a real difference in a population. These differences are communicated in terms of statistics, which are essentially numbers that describe characteristics of data. These characteristics are measured, described, compared, and modeled in varied ways. However, all statistical analysis share a similar aim—to make meaning from quantitative data or to explain some observed phenomena (e.g, current status of a variable, relationships between variables, effects between variables).

As statistical analysis is embedded in the scientific method, the R user might consider the research process at large before interacting with R. When analyzing data it is helpful to hold a genuine desire to understand a topic, be aware of and control your biases, resist simplistic or convenient explanations for relationships, and employ systematic peer-reviewed methods. When asking and investigating research questions, it is suggested to set aside your personal beliefs and experiences regarding the data. Additionally, be willing to adopt rational perspectives on your data while suspending disbelief about findings that are contrary to your expectations or beliefs.

In this and the following chapter, you will be guided through methods that will support many types of statistical analyses. For instance, you will learn how to determine the prevalence of a phenomenon, explore underlying factors, test interventions, test group differences, and explore multivariate approaches to statistical analysis.

4.1 Descriptive Statistics

Descriptive statistics provide a (for lack of a better term) description of what is observed in collected data—whereby measures of centrality and variability are our main tools for understanding. These are oftentimes referred to as summary statistics. We use descriptive statistics to assess the characteristics of participants, which supports the ability to make statements regarding make the generalizability of the sample to the overall population or other context.

Measures of **central tendency** are used to describe data with a single number. For instance, the mean represents the average of all of each of the individual data within a set.

```
mean(mtcars$mpg)
```

```
## [1] 20.09062
```

The median describes the point at which 50% of cases in a distribution fall below and 50% fall above.

```
median(mtcars$mpg)
```

```
## [1] 19.2
```

Variability is the amount of spread or dispersion in a set of scores. Knowing the extent to which data varies allows researchers to compare groups within data effectively and accurately.

Assessments of variability provide a representation of the amount of dispersion within a distribution from a measure of central tendency. For instance, range displays the lowest and highest score in a vector.

```
range(mtcars$mpg)
```

```
## [1] 10.4 33.9
```

Standard deviation represents the average score deviation from the mean.

```
sd(mtcars$mpg)
```

```
## [1] 6.026948
```

Variance is the square of the standard deviation.

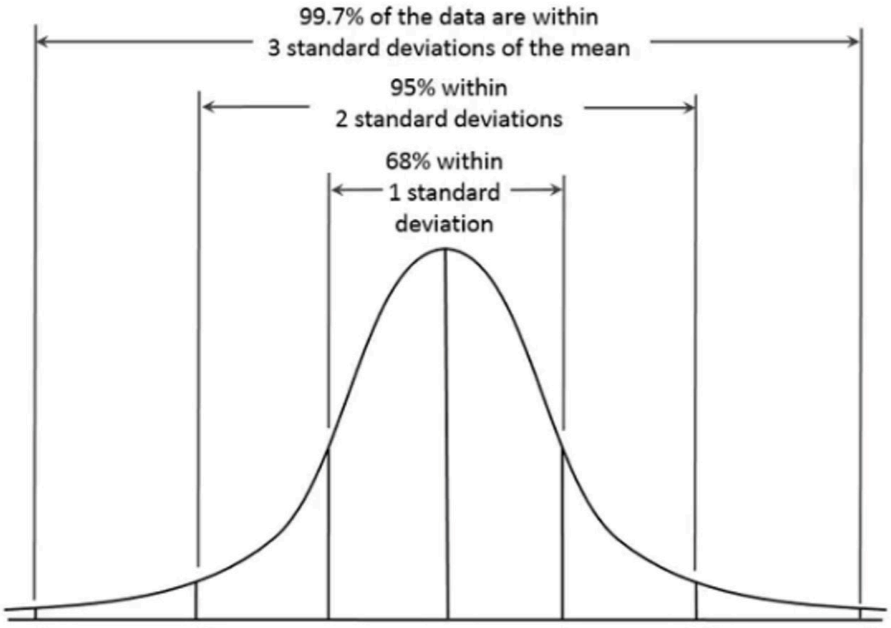
```
var(mtcars$mpg)
```

```
## [1] 36.3241
```

In reference to the distribution of the data, standard deviations tell us where scores generally fall. SD is often reported alongside the mean (e.g., $M = 7$, $SD = 1.5$). With these numbers, we can ascertain that 68% of scores will fall between 5.5 and 8.5, and 95% of scores fall between 4 and 10. Furthermore, we can ascertain with a normal distribution that about 99% of scores might fall between 2.5 and 12.5.

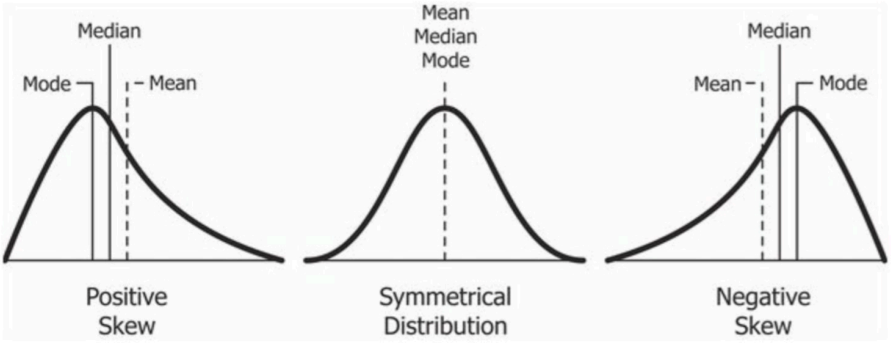
These association between means, standard deviations, and normal distributions parameterize the probability of future observations on a similar measure. Many of the most common statistical tests we use are conceived of assuming this parametric concept—rationalizing the assumption that observations will be normally distributed.

As exemplified above, data that meet this assumption tend to have around 68% of data points falling between +-1 standard deviation of the mean, and 95% of data falling between +-2 standard deviations of the mean. Furthermore, normality tends to be met when the distribution curve is peaked at the mean and has only one mode. Additionally, the curve in a normal distribution should be symmetrical on both sides of the mean, median, and mode while having asymptotic tails (tails curve but do not approach the axis; no 100s or zeros).

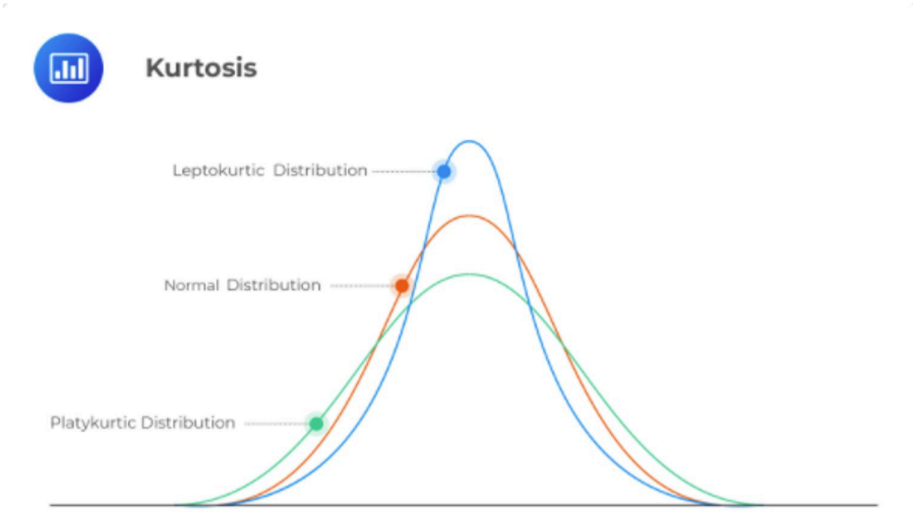


A distribution is skewed, or has skewness, positively or negatively when there is longer tail on one of the distribution. A longer tail on the right tends to indicate a smaller number of data points at the higher end of the distribution—a positive skew. A shorter tail on the right indicates a larger number of data points on the high end of the distribution—a negative skew.

One helpful rule of thumb is that if the mean is greater than the median the distribution likely has a positive skew. Conversely, if the mean is less than the median, the distribution likely has a negative skew. See the figure below for an illustration of each distribution.



The smaller the standard deviation and variance, the higher the level of agreement, or similarity, in response or scores. When agreement or similarity in scores is very high a distribution may be leptokurtic—having low variance (i.e., small standard deviations). When scores are very broad high variance may create a platykurtic distribution. These are represented within the concept of kurtosis. Kurtosis will be assessed alongside skewness in a few paragraphs.



There are several ways to assess for the assumption of normality. Some researchers prefer to visually assess histograms. Some like to compare means and medians within the context of standard deviations. The most reliable, and mathematically-based, may be a Shapiro-Wilk test. A non-significant Shapiro-Wilk outcome will provide support of a normal distribution as test significance can be practically interpreted to suggest that the distribution is significantly different than a vector that is normally distributed. For instance, as the p value exceeds .05 when applied to the mtcars mpg column the miles per gallon variable can be interpreted as normally distributed across the vector. Use the function `shapiro.test()`

```
shapiro.test(mtcars$mpg)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  mtcars$mpg
## W = 0.94756, p-value = 0.1229
```

You can also examine the influence of kurtosis and skewness values using the `psych::describe()` function. Interpretations of skewness and kurtosis values differ from field to field, but generally you can assume a normal distribution if these values fall within -3 and +3.

```
psych::describe(mtcars$mpg)
```

```
##    vars  n  mean   sd median trimmed  mad  min  max range skew kurtosis   se
## X1    1 32 20.09 6.03   19.2    19.7 5.41 10.4 33.9  23.5 0.61    -0.37 1.07
```

4.1.1 Case Study Part One

We are going to utilize the mtcars dataset to apply descriptive statistical concepts using R. The dataset includes information regarding fuel consumption and 10 other aspects of automobile design and performance across 32 automobiles.

Put otherwise, `mtcars` is a data frame with 32 observations of 11 (numeric) variables. These include:

- Miles/(US) gallon (mpg)
- Number of cylinders (cyl)
- Displacement (disp)
- Gross horsepower (hp)
- Rear axle ratio (drat)
- Weight in 1000 lbs (wt)
- 1/4 mile time (qsec)
- Engine (0 = V-shaped, 1 = straight) (vs)
- Transmission (0 = automatic, 1 = manual) (am)
- Number of forward gears (gear)
- Number of carburetors (carb)

First, load the `tidyverse` library and examine the dataset.

```
library(tidyverse, quiet = T)
data(mtcars) # bring mtcars dataset into environment
str(mtcars) #use the str() function to examine the structure of the data frame.
```

```
## 'data.frame':    32 obs. of  11 variables:
##  $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
##  $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
##  $ disp: num  160 160 108 258 360 ...
##  $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
##  $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
##  $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
##  $ qsec: num  16.5 17 18.6 19.4 17 ...
```

```
##  $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
##  $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
##  $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
##  $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

Let's do some dataset manipulation before diving into analysis. Remember our pipe operator %>%? It's going to help us get rid of a column using the `select()` function, and create a new column using the `mutate()` function. You can read the following code as: "Assign new object mtcars as old dataset mtcars where everything but drat is selected. Then wt is divided by qsec using the mutate function to create a new column called wtqsec_ratio."

```
mtcars <- mtcars %>% select(-drat) %>% mutate(wtqsec_ratio = wt/qsec)
```

When examining the structure of the data frame, note that the columns vs, am, cyl, gear, and carb are listed as numeric when in actuality they are factor/categorical variables. Use `mutate()` and `as.ordered()` to set numerical vectors as ordered factors. Then, use the `recode()` function to label each factor level with the appropriate string for engine type. Finally, reexamine the structure to see evidence of the updates.

```
mtcars <- mtcars %>%
  mutate(cyl = as.ordered(cyl), gear = as.ordered(gear), carb = as.ordered(carb))

mtcars <- mtcars %>%
  mutate(vs = recode_factor(vs, `0` = "v-shaped", `0.5` = 'v-shaped', `1` = "straight"),
         am = recode_factor(am, `0` = "automatic", `1` = "manual"))

str(mtcars)
```

```
## 'data.frame':    32 obs. of  11 variables:
##  $ mpg         : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
##  $ cyl         : Ord.factor w/ 3 levels "4"<"6"<"8": 2 2 1 2 3 2 3 1 1 2 ...
##  $ disp        : num  160 160 108 258 360 ...
##  $ hp          : num  110 110 93 110 175 105 245 62 95 123 ...
##  $ wt          : num  2.62 2.88 2.32 3.21 3.44 ...
##  $ qsec        : num  16.5 17 18.6 19.4 17 ...
##  $ vs          : Factor w/ 2 levels "v-shaped","straight": 1 1 2 2 1 2 1 2 2 2 ...
##  $ am          : Factor w/ 2 levels "automatic","manual": 2 2 2 1 1 1 1 1 1 1 ...
##  $ gear        : Ord.factor w/ 3 levels "3"<"4"<"5": 2 2 2 1 1 1 1 2 2 2 ...
##  $ carb        : Ord.factor w/ 6 levels "1"<"2"<"3"<"4"<..: 4 4 1 1 2 1 4 2 2 4 ...
##  $ wtqsec_ratio: num  0.159 0.169 0.125 0.165 0.202 ...
```

Base R comes with a variety of functions for descriptive statistics, including `mean`, `sd`, `var`, `min`, `max`, `median`, `range`, and `quantile`. These can be used on

specific variables in a data frame, or to all columns or rows in the data frame using the `apply` function. The X argument below represents the dataset or array to which functions will be applied. The 2 tells R to look at the columns, whereas 1 would indicate that a function would be applied to rows. The `FUN` argument calls the specific function to apply, and `na.rm` denotes whether missing data should be removed or not.

```
apply(X = mtcars, MARGIN = 2, FUN = min, na.rm = TRUE)
```

```
##          mpg          cyl         disp           hp           wt         qsec
##       "10.4"          "4"      " 71.1"        " 52"      "1.513"      "14.50"
##           vs           am         gear         carb wtqsec_ratio
##   "straight"  "automatic"          "3"          "1" "0.08720302"
```

The `psych` library includes helpful descriptive statistics functions. Use psych's `describe()` function to see the descriptive statistics we previously examined in one table.

```
#install.packages('psych')
library(psych)
psych::describe(mtcars)
```

```
##              vars  n   mean     sd median trimmed    mad   min    max  range
## mpg             1 32  20.09   6.03  19.20   19.70   5.41 10.40  33.90  23.50
## cyl*            2 32   2.09   0.89   2.00    2.12   1.48  1.00   3.00   2.00
## disp            3 32 230.72 123.94 196.30  222.52 140.48 71.10 472.00 400.90
## hp              4 32 146.69  68.56 123.00  141.19  77.10 52.00 335.00 283.00
## wt              5 32   3.22   0.98   3.33    3.15   0.77  1.51   5.42   3.91
## qsec            6 32  17.85   1.79  17.71   17.83   1.42 14.50  22.90   8.40
## vs*             7 32   1.44   0.50   1.00    1.42   0.00  1.00   2.00   1.00
## am*             8 32   1.41   0.50   1.00    1.38   0.00  1.00   2.00   1.00
## gear*           9 32   1.69   0.74   2.00    1.62   1.48  1.00   3.00   2.00
## carb*          10 32   2.72   1.37   2.00    2.65   1.48  1.00   6.00   5.00
## wtqsec_ratio   11 32   0.18   0.06   0.18    0.18   0.07  0.09   0.31   0.22
##                skew kurtosis    se
## mpg            0.61    -0.37  1.07
## cyl*          -0.17    -1.76  0.16
## disp           0.38    -1.21 21.91
## hp             0.73    -0.14 12.12
## wt             0.42    -0.02  0.17
## qsec           0.37     0.34  0.32
## vs*            0.24    -2.00  0.09
## am*            0.36    -1.92  0.09
## gear*          0.53    -1.07  0.13
```

```
## carb*         0.35    -0.95  0.24
## wtqsec_ratio  0.27    -0.70  0.01
```

Use the psych function `describeBy()` and the argument `group` to see summary statistics grouped by any variable. The code below groups by transmission, where automatic = 0 and manual = 1). The use of double colons `::` tells R to use the function from this specific package and not any other `describeBy()` function from packages other than `psych`.

```
psych::describeBy(mtcars, group='am') # summary statistics by grouping variable
```

```
##
##  Descriptive statistics by group
## am: automatic
##              vars  n   mean     sd median trimmed    mad    min    max  range
## mpg             1 19  17.15   3.83  17.30   17.12   3.11  10.40  24.40  14.00
## cyl*            2 19   2.47   0.77   3.00    2.53   0.00   1.00   3.00   2.00
## disp            3 19 290.38 110.17 275.80  289.71 124.83 120.10 472.00 351.90
## hp              4 19 160.26  53.91 175.00  161.06  77.10  62.00 245.00 183.00
## wt              5 19   3.77   0.78   3.52    3.75   0.45   2.46   5.42   2.96
## qsec            6 19  18.18   1.75  17.82   18.07   1.19  15.41  22.90   7.49
## vs*             7 19   1.37   0.50   1.00    1.35   0.00   1.00   2.00   1.00
## am*             8 19   1.00   0.00   1.00    1.00   0.00   1.00   1.00   0.00
## gear*           9 19   1.21   0.42   1.00    1.18   0.00   1.00   2.00   1.00
## carb*          10 19   2.74   1.15   3.00    2.76   1.48   1.00   4.00   3.00
## wtqsec_ratio   11 19   0.21   0.05   0.21    0.21   0.04   0.12   0.31   0.18
##                skew kurtosis    se
## mpg            0.01    -0.80  0.88
## cyl*          -0.95    -0.74  0.18
## disp           0.05    -1.26 25.28
## hp            -0.01    -1.21 12.37
## wt             0.98     0.14  0.18
## qsec           0.85     0.55  0.40
## vs*            0.50    -1.84  0.11
## am*             NaN      NaN  0.00
## gear*          1.31    -0.29  0.10
## carb*         -0.14    -1.57  0.26
## wtqsec_ratio   0.38    -0.68  0.01
## ------------------------------------------------------------
## am: manual
##              vars  n   mean    sd median trimmed   mad   min    max  range
## mpg             1 13  24.39  6.17  22.80   24.38  6.67 15.00  33.90  18.90
## cyl*            2 13   1.54  0.78   1.00    1.45  0.00  1.00   3.00   2.00
## disp            3 13 143.53 87.20 120.30  131.25 58.86 71.10 351.00 279.90
## hp              4 13 126.85 84.06 109.00  114.73 63.75 52.00 335.00 283.00
```

```
## wt              5 13   2.41  0.62   2.32    2.39  0.68  1.51   3.57   2.06
## qsec            6 13  17.36  1.79  17.02   17.39  2.34 14.50  19.90   5.40
## vs*             7 13   1.54  0.52   2.00    1.55  0.00  1.00   2.00   1.00
## am*             8 13   2.00  0.00   2.00    2.00  0.00  2.00   2.00   0.00
## gear*           9 13   2.38  0.51   2.00    2.36  0.00  2.00   3.00   1.00
## carb*          10 13   2.69  1.70   2.00    2.55  1.48  1.00   6.00   5.00
## wtqsec_ratio   11 13   0.14  0.05   0.13    0.14  0.05  0.09   0.24   0.16
##                skew kurtosis    se
## mpg            0.05    -1.46  1.71
## cyl*           0.87    -0.90  0.22
## disp           1.33     0.40 24.19
## hp             1.36     0.56 23.31
## wt             0.21    -1.17  0.17
## qsec          -0.23    -1.42  0.50
## vs*           -0.14    -2.13  0.14
## am*             NaN      NaN  0.00
## gear*          0.42    -1.96  0.14
## carb*          0.54    -1.25  0.47
## wtqsec_ratio   0.62    -0.90  0.01
```

Analyze frequency or count data for various groups using the `table()` function. The first line of code will create a table of counts disaggregated by transmission type and number of gears. The second line of code modifies the `mtcars` object to be grouped by transmission and gear, then summarizes each into a proportion of the entire population of cars as column 'prop' The `spread()` function is used to display the data of interest based on established groups.

```
table(mtcars$am, mtcars$gear)
```

```
##
##              3  4  5
##   automatic 15  4  0
##   manual     0  8  5
```

```
mtcars %>% group_by(am, gear) %>% summarise(prop = n()/nrow(mtcars)) %>% spread(gear, p
```

```
## `summarise()` has grouped output by 'am'. You can override using the `.groups`
## argument.
```

```
## # A tibble: 2 x 4
## # Groups:   am [2]
##   am           `3`   `4`    `5`
##   <fct>      <dbl> <dbl>  <dbl>
## 1 automatic  0.469 0.125 NA
## 2 manual    NA     0.25   0.156
```

4.2 Correlational Statistics

Using contingency tables such as the first in the previous code chunk, we can run a Chi Square or Fisher's exact test to see whether these categorical variables are independent. The null hypothesis for each of these tests is that there is no relationship between transmission type and the number of gears in a car. What do our results suggest?

```
tb2 <- table(mtcars$am, mtcars$gear)
chisq.test(tb2)
```

```
##
##  Pearson's Chi-squared test
##
## data:  tb2
## X-squared = 20.945, df = 2, p-value = 2.831e-05
```

```
fisher.test(tb2)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  tb2
## p-value = 2.13e-06
## alternative hypothesis: two.sided
```

Both tests indicate that the null hypothesis should be rejected, thus suggesting that there is a relationship between transmission type and the number of gears in a car. These tests are examples of correlational statistics, as they are measuring the association between two variables, or the extent to which they co-relate.

4.2.1 How Correlation Works

Correlation implies the desire to find a pattern or relationship between variables (e.g., What is the relationship between X and Y?). Correlation coefficients are returned as number between -1 and 1. This number tells us about the strength of the relationship between the two variables. Correlational analysis is commonly employed in survey studies as an additional analysis step beyond basic descriptive statistics.

Correlational design refers to a type of study that has the appearance of an experiment (e.g., dependent and independent variables, interventions, and even random sampling), but lacks true causality. These designs are used when the

manipulation of variables may not be possible or ethical (e.g., the effects of texting while driving; effect of homelessness on student achievement). Additionally, correlational research designs can be used as a precursor to an experiment (e.g., determining what variables are correlated with student effort before trying to conduct an experiment designed to improve student effort among adolescents).

4.2.2 Correlation, not Causation?

The maxim, "correlation does not imply causation" is frequently stated, but what does it mean? In essence, finding a correlation between two variables does not establish a causal relationship. To do this, we need to know the underlying directionality/ordering of the relationship and rule out confounding variables that might explain the relationship. For example, a strong positive correlation exists between total revenue generated by arcades and computer science doctorates awarded. However, one of these events did not not cause the other.

4.2.3 Statistical Significance

The concept of statistical significance reflects that our findings are likely not due to chance, and that if we reject a null hypothesis (I.e., find a significant difference), it is likely that the same difference will hold true for the entire population. Findings are either statistical or not, (at the alpha level you set .01, .05, .10). Nonsignificant results are just as informative as significant results.

For instance, if you set your apriori alpha level to .05 for a correlation test that measures alignment of math and music performance scores and find a strong positive and significant correlation ($r = .71$, $p = .03$) there is only a 3% probability that the evidenced correlation is due to chance or some extreme, uncontrollable event. Put otherwise, the evidence for the correlation is larger than can reasonably be explained as a chance occurrence.

4.2.4 Choosing the Right Test

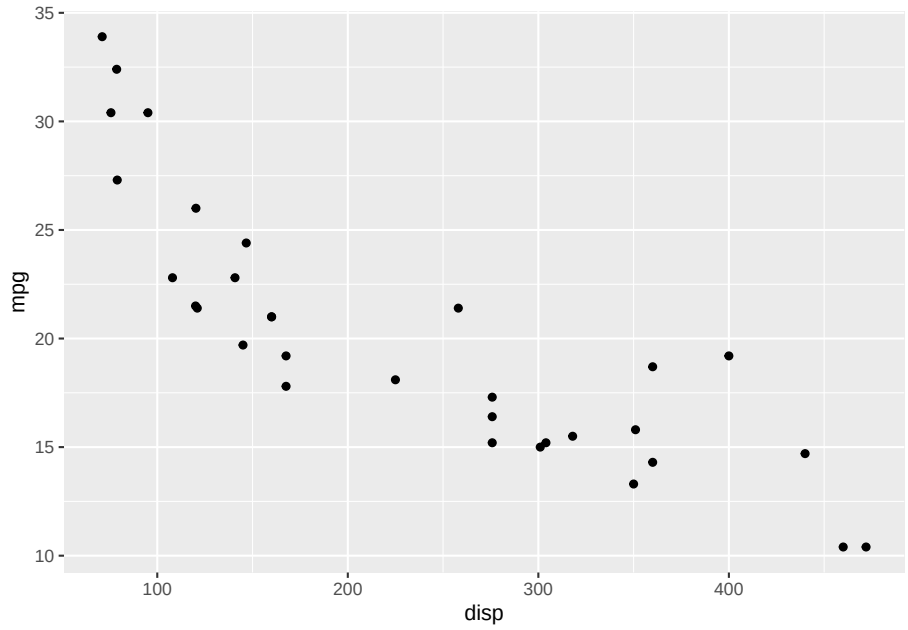
There are several different correlational approaches to use. See the table below for a breakdown of each correlational test and when to use it.

Test	R Function	When to use it
Chi-S quare	`chisq.test()`	(a) when you have two categorical variables with two or more categories in each variable, (b) there is no relationship between subjects in each group, (c) variables are not paired, (d) frequencies exceed 5 in each cell (I.e., category/group.)
Spear man's Rho	`co r.test (method = "spearman")`	(a) when data one or both variables are ordinal, (b) or when data are continuous and fail to meet assumptions of Pearson's r test
Pear son's R	`c or.test (method = "pearson")`	(a) when data are continuous, (b) normally distributed, (c) have minimal outliers (+3 SD from the mean), (d) relationships are linear

4.2.5 Case Study Part Two

To continue the original case study, create a plot of displacement and miles per gallon. There appears to be a moderate to strong negative correlation. Use the `cor.test()` base R function to see correlation coefficients and related statistical output. If your data require nonparametric analysis, use method = `spearman` to call Spearman's Rho.

```
ggplot(mtcars, aes(x = disp, y = mpg)) +
  geom_point()
```

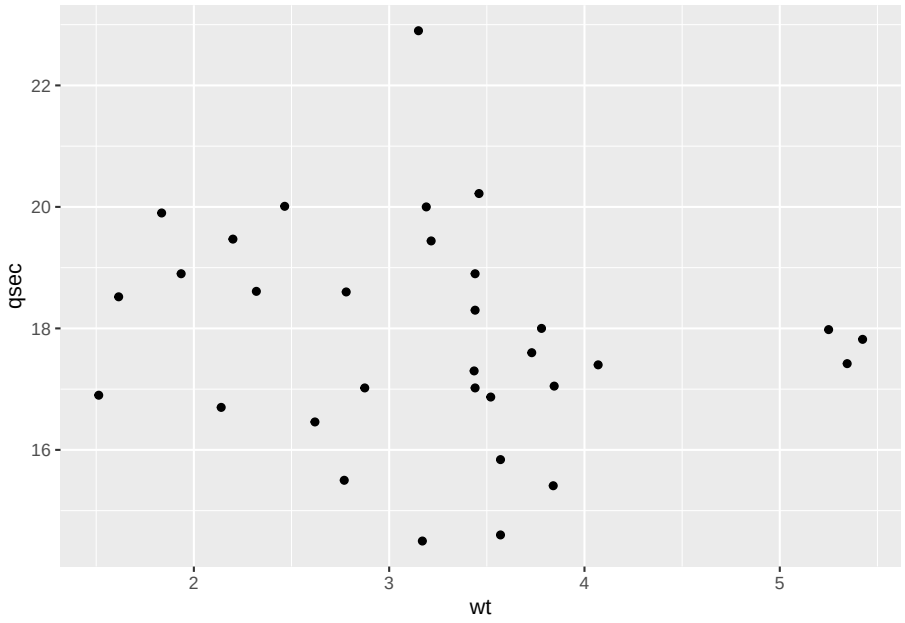



```
cor.test(mtcars$disp, mtcars$mpg, method='pearson')
```

```
##
##  Pearson's product-moment correlation
##
## data:  mtcars$disp and mtcars$mpg
## t = -8.7472, df = 30, p-value = 9.38e-10
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  -0.9233594 -0.7081376
## sample estimates:
##        cor
## -0.8475514
```

There was a negative strong significant correlation (r = -.85) between disp and mpg. Are wt. and qsec correlated?

```
ggplot(mtcars, aes(x=wt, y=qsec)) +
  geom_point()
```




```
cor.test(mtcars$wt, mtcars$qsec, method='pearson')
```

```
##
##  Pearson's product-moment correlation
##
## data:  mtcars$wt and mtcars$qsec
## t = -0.97191, df = 30, p-value = 0.3389
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  -0.4933536  0.1852649
## sample estimates:
##        cor
## -0.1747159
```

The p value is above .05 which indicates that we can retain the null hypothesis that there is no relationship between qsec and wt. Or as R puts it, the true correlation is equal to 0.

Kendall rank correlation coefficient can also be called for nonparametric correlation when the assumptions of a test (e.g. sample size, normality, etc.) are violated.

```
cor.test(x=mtcars$disp, y=mtcars$mpg, method='pearson')
```

```
##
##  Pearson's product-moment correlation
##
## data:  mtcars$disp and mtcars$mpg
## t = -8.7472, df = 30, p-value = 9.38e-10
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  -0.9233594 -0.7081376
## sample estimates:
##        cor
## -0.8475514
```

```
cor.test(x=mtcars$disp, y=mtcars$mpg, method="kendall")
```

```
## Warning in cor.test.default(x = mtcars$disp, y = mtcars$mpg, method =
## "kendall"): Cannot compute exact p-value with ties
```

```
##
##  Kendall's rank correlation tau
##
## data:  mtcars$disp and mtcars$mpg
## z = -6.1083, p-value = 1.007e-09
## alternative hypothesis: true tau is not equal to 0
## sample estimates:
##        tau
## -0.7681311
```

Kendall's rank correlation can be interpreted exactly the same as Pearson's correlation coefficient in terms of magnitude and valence. Kendall's estimates, however, are often more conservative than Pearson's (tau = -.77, r = -.85). Judging by either test, there is a significant strong negative correlation between engine volume (disp) and miles per gallon (mpg).

4.3 Review

In this chapter we introduced descriptive statistics, concepts underlying normality, and some correlational methods. To make sure you understand this material, there is a practice assessment to go along with this and the next chapter at https://jayholster.shinyapps.io/StatsinRAssessment/

4.4 References

Revelle, W. (2017) psych: Procedures for Personality and Psychological Research, Northwestern University, Evanston, Illinois, USA, <https://CRAN.R-project.org/package=psych> Version = 1.7.8.

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L.D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T.L., Miller, E., Bache, S.M., Müller, K., Ooms, J., Robinson, D., Seidel, D.P., Spinu, V., Takahashi, K., Vaughan, D., Wilke, C., Woo, K., & Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software, 4*(43), 1686. https://doi.org/10.21105/joss.01686.

4.4.1 R Short Course Series

Video lectures of each guidebook chapter can be found at https://osf.io/6jb9t/. For this chapter, find the follow the folder path Statistics in R -> AY 2021-2022 Spring and access the video files, r markdown documents, and other materials for each short course.

4.4.2 Acknowledgements

This guidebook was created with support from the Center for Research Data and Digital Scholarship and the Laboratory for Interdisciplinary Statistical Analysis at the University of Colorado Boulder, as well as the U.S. Agency for International Development under cooperative agreement #7200AA18CA00022. Individuals who contributed to materials related to this project include Jacob Holster, Eric Vance, Michael Ramsey, Nicholas Varberg, and Nickoal Eichmann-Kalwara.

Chapter 5

Exploring and Modeling Data

The purpose of this chapter is to expose you to several options for meaning-making with numbers beyond descriptive statistics. More specifically, you will be asked to apply your previous knowledge regarding descriptive statistics, data visualization, and data manipulation in the pursuit of the scientific method through statistical modeling.

Many interesting research questions are addressed with descriptive statistics. For instance, the census might ask the question: How many people live in the United States?relationships between and within variables tend to provide answers to more nuanced research questions—for instance, what is the relationship between median household income and educational attainment, or even what are the most influential predictors of happiness.

5.1 Quick Review

First, let's review some of the things we've learned in this text so far. To get started, install and/or load the following packages into R Studio.

```
library(tidyverse)
library(SDT)
library(DataExplorer)
library(psych)
library(rstatix)
library(olsrr)
library(car)
```

We will be using a built-in dataset from the `SDT` package that contains information on learning motivation data based on Self Determination Theory. Call the dataset with the object name `learning_motivation` and assign it to the object `data` with the operator `<-`. See the code below.

To get a quick look at many facets of a dataset in one line of code, try `DataExplorer`. `DataExplorer` includes an automated data exploration process for analytic tasks and predictive modeling. Simply run the function `create_report()` on a data frame to produce multiple plots and tables related to descriptive and correlation related analyses. Use the `summary()`, `str()`, and `psych::describe()` functions to examine and assess the variables in the dataset for normality

For some more quick review, arrange the dataset by age in descending order. Then examine the first and last six rows of the dataset.

```
data <- data %>% arrange(desc(age))
head(data)
```

```
##   sex age intrinsic identified introjected external
## 1   2  22       1.4       3.00        3.00     3.50
## 2   1  19       4.6       2.75        3.50     2.25
## 3   1  19       1.4       2.25        1.00     2.00
## 4   2  19       3.0       3.50        3.75     4.00
## 5   1  19       2.6       3.50        3.25     2.75
## 6   1  19       4.8       4.25        1.75     1.50
```

```
tail(data)
```

```
##      sex age intrinsic identified introjected external
## 1145   2  10       3.6       4.75        3.50     4.25
## 1146   1  10       3.2       2.75        3.25     2.75
## 1147   1  10       3.6       4.25        3.00     3.50
## 1148   1  10       3.4       3.25        3.50     2.75
## 1149   2  10       3.2       3.00        3.25     2.75
## 1150   2  10       3.6       3.50        3.25     4.50
```

Create a new column based on values from other columns without opening and saving the data file.

```
data <- data %>% mutate(internalizedmotivation = intrinsic + identified)
data <- data %>% mutate(internalizedmotivationlog = log10(intrinsic + identified))
head(data)
```

```
##   sex age intrinsic identified introjected external internalizedmotivation
```

```
## 1   2  22       1.4       3.00        3.00     3.50                  4.40
## 2   1  19       4.6       2.75        3.50     2.25                  7.35
## 3   1  19       1.4       2.25        1.00     2.00                  3.65
## 4   2  19       3.0       3.50        3.75     4.00                  6.50
## 5   1  19       2.6       3.50        3.25     2.75                  6.10
## 6   1  19       4.8       4.25        1.75     1.50                  9.05
##   internalizedmotivationlog
## 1                 0.6434527
## 2                 0.8662873
## 3                 0.5622929
## 4                 0.8129134
## 5                 0.7853298
## 6                 0.9566486
```

```
psych::describe(data)
```

```
##                           vars    n  mean   sd median trimmed  mad  min max
## sex                          1 1150  1.50 0.50   1.00    1.50 0.00  1.0   2
## age                          2 1150 14.09 1.85  14.00   13.98 1.48 10.0  22
## intrinsic                    3 1150  3.13 1.09   3.20    3.15 1.19  1.0   5
## identified                   4 1150  3.29 1.01   3.25    3.34 1.11  1.0   5
## introjected                  5 1150  2.75 1.04   2.75    2.73 1.11  1.0   5
## external                     6 1150  2.94 0.94   3.00    2.95 1.11  1.0   5
## internalizedmotivation       7 1150  6.42 1.80   6.55    6.47 1.82  2.0  10
## internalizedmotivationlog    8 1150  0.79 0.14   0.82    0.80 0.12  0.3   1
##                           range  skew kurtosis   se
## sex                         1.0  0.01    -2.00 0.01
## age                        12.0  0.51    -0.04 0.05
## intrinsic                   4.0 -0.15    -0.75 0.03
## identified                  4.0 -0.39    -0.49 0.03
## introjected                 4.0  0.12    -0.80 0.03
## external                    4.0 -0.10    -0.49 0.03
## internalizedmotivation      8.0 -0.27    -0.39 0.05
## internalizedmotivationlog   0.7 -1.17     1.58 0.00
```

Select rows of a data frame based on a certain condition. Use the `dim()`function to ascertain the dimensions of the resultant dataset. If 1 represents males and 2 represents females, how many males are 18 or older in the dataset?

```
malesover18 <- data %>% filter(age >= 18, sex == 1)
dim(malesover18)
```

```
## [1] 34  8
```

Select specific columns of a data frame using the `select()` function. Or select all columns except one using -column (e.g. `select(-intrinsic)`)

```
selected_data <- data %>% select(sex, age, intrinsic, external)
head(selected_data)
```

```
##   sex age intrinsic external
## 1   2  22       1.4     3.50
## 2   1  19       4.6     2.25
## 3   1  19       1.4     2.00
## 4   2  19       3.0     4.00
## 5   1  19       2.6     2.75
## 6   1  19       4.8     1.50
```

Use transformations as an opportunity to label your data categorically.

```
newdata <- data %>% mutate(preteen = age <= 12)
head(newdata)
```

```
##   sex age intrinsic identified introjected external internalizedmotivation
## 1   2  22       1.4       3.00        3.00     3.50                   4.40
## 2   1  19       4.6       2.75        3.50     2.25                   7.35
## 3   1  19       1.4       2.25        1.00     2.00                   3.65
## 4   2  19       3.0       3.50        3.75     4.00                   6.50
## 5   1  19       2.6       3.50        3.25     2.75                   6.10
## 6   1  19       4.8       4.25        1.75     1.50                   9.05
##   internalizedmotivationlog preteen
## 1                 0.6434527   FALSE
## 2                 0.8662873   FALSE
## 3                 0.5622929   FALSE
## 4                 0.8129134   FALSE
## 5                 0.7853298   FALSE
## 6                 0.9566486   FALSE
```

Save your edited data frame to a .csv file.

```
write_csv(data, "sdtedit.csv")
```

5.2 Correlational Models

Next we will review and explore correlational models, which were conceptually covered in detail in the previous chapter. Most correlation coefficients are

interpreted as a number between -1 and 1. This number tells us about the strength of the relationship between the two variables. Even if your final goal is regression or an inferential statistical test, correlations are useful to look at during exploratory data analysis before trying to fit a predictive model. Here are several correlation tasks using the Rstatix library.

```
# Correlation test between two variables
data %>% cor_test(age, intrinsic, method = "pearson")
```

```
## # A tibble: 1 x 8
##   var1  var2        cor statistic            p conf.low conf.high method
##   <chr> <chr>     <dbl>     <dbl>        <dbl>    <dbl>     <dbl> <chr>
## 1 age   intrinsic -0.16     -5.61 0.0000000261   -0.219    -0.106 Pearson
```

```
# Correlation of one variable against all
data %>% cor_test(intrinsic, method = "pearson")
```

```
## # A tibble: 7 x 8
##   var1      var2              cor statistic         p conf.low conf.high method
##   <chr>     <chr>           <dbl>     <dbl>     <dbl>    <dbl>     <dbl> <chr>
## 1 intrinsic sex              0.22      7.60  6.03e- 14   0.163     0.273  Pears~
## 2 intrinsic age             -0.16     -5.61  2.61e-  8  -0.219    -0.106  Pears~
## 3 intrinsic identified       0.48     18.4   2.61e- 66   0.431     0.520  Pears~
## 4 intrinsic introjected      0.29     10.4   2.41e- 24   0.240     0.346  Pears~
## 5 intrinsic external        -0.026    -0.881 3.78e-  1  -0.0837    0.0319 Pears~
## 6 intrinsic internalizedmo~  0.87     60.3   0           0.857     0.885  Pears~
## 7 intrinsic internalizedmo~  0.84     52.7   6.05e-309   0.824     0.857  Pears~
```

```
# Pairwise correlation test between all variables
data %>% cor_test(method = "pearson")
```

```
## # A tibble: 64 x 8
##    var1  var2                  cor statistic         p conf.low conf.high method
##    <chr> <chr>               <dbl>     <dbl>     <dbl>    <dbl>     <dbl> <chr>
##  1 sex   sex                 1        Inf     0           1         1      Pears~
##  2 sex   age                 0.027      0.899 3.69e- 1  -0.0313    0.0842 Pears~
##  3 sex   intrinsic           0.22       7.60  6.03e-14   0.163     0.273  Pears~
##  4 sex   identified          0.13       4.60  4.62e- 6   0.0774    0.191  Pears~
##  5 sex   introjected         0.039      1.33  1.82e- 1  -0.0185    0.0969 Pears~
##  6 sex   external            0.0094     0.317 7.51e- 1  -0.0485    0.0671 Pears~
##  7 sex   internalizedmotiva~ 0.21       7.19  1.13e-12   0.152     0.262  Pears~
##  8 sex   internalizedmotiva~ 0.2        6.96  5.78e-12   0.145     0.256  Pears~
##  9 age   sex                 0.027      0.899 3.69e- 1  -0.0313    0.0842 Pears~
## 10 age   age                 1        Inf     0           1         1      Pears~
## # ... with 54 more rows
```

```
# Compute correlation matrix
cor.mat <- data %>% cor_mat()
cor.mat
```

```
## # A tibble: 8 x 9
##   rowname                    sex    age intrinsic identified introjected external
## * <chr>                    <dbl>  <dbl>     <dbl>      <dbl>       <dbl>    <dbl>
## 1 sex                      1       0.027     0.22        0.13       0.039   0.0094
## 2 age                      0.027   1        -0.16       -0.16      -0.13   -0.074
## 3 intrinsic                0.22   -0.16      1           0.48       0.29   -0.026
## 4 identified               0.13   -0.16      0.48        1          0.46    0.29
## 5 introjected              0.039  -0.13      0.29        0.46       1       0.5
## 6 external                 0.0094 -0.074    -0.026       0.29       0.5     1
## 7 internalizedmotivation 0.21   -0.19      0.87        0.85       0.44    0.14
## 8 internalizedmotivatio~ 0.2    -0.16      0.84        0.84       0.43    0.15
## # ... with 2 more variables: internalizedmotivation <dbl>,
## #   internalizedmotivationlog <dbl>
```

```
# Show the significance levels
cor.mat %>% cor_get_pval()
```

```
## # A tibble: 8 x 9
##   rowname                  sex      age intrinsic identified introjected external
##   <chr>                  <dbl>    <dbl>     <dbl>      <dbl>       <dbl>    <dbl>
## 1 sex                0        3.69e- 1 6.03e- 14  4.62e-  6    1.82e- 1 7.51e- 1
## 2 age                3.69e- 1 0        2.61e-  8  2.16e-  8    5.05e- 6 1.2 e- 2
## 3 intrinsic          6.03e-14 2.61e- 8 0          2.61e- 66    2.41e-24 3.78e- 1
## 4 identified         4.62e- 6 2.16e- 8 2.61e- 66  0            1.26e-61 4.26e-23
## 5 introjected        1.82e- 1 5.05e- 6 2.41e- 24  1.26e- 61    0        1.03e-73
## 6 external           7.51e- 1 1.2 e- 2 3.78e-  1  4.26e- 23    1.03e-73 0
## 7 internalizedmotiv~ 1.13e-12 7.54e-11 0          2.08e-316    2.47e-54 9.85e- 7
## 8 internalizedmotiv~ 5.78e-12 3.57e- 8 6.05e-309  1.90e-300    6.78e-52 5.24e- 7
## # ... with 2 more variables: internalizedmotivation <dbl>,
## #   internalizedmotivationlog <dbl>
```

```
# Replacing correlation coefficients by symbols
cor.mat %>%
  cor_as_symbols() %>%
  pull_lower_triangle()
```

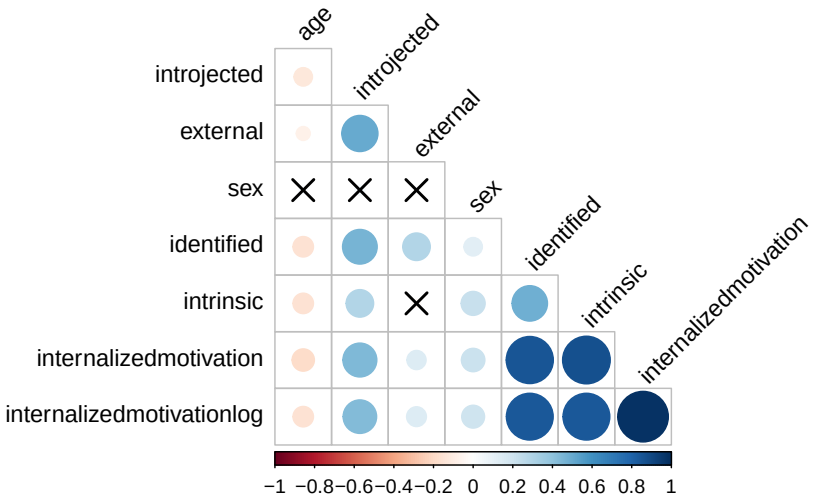
```
##                     rowname sex age intrinsic identified introjected external
## 1                       sex
## 2                       age
```

```
## 3                intrinsic
## 4               identified                .
## 5              introjected                .          .
## 6                 external                .          .          .
## 7    internalizedmotivation                *          *          .
## 8 internalizedmotivationlog                *          *          .
##   internalizedmotivation internalizedmotivationlog
## 1
## 2
## 3
## 4
## 5
## 6
## 7
## 8                      *
```

```
# Mark significant correlations
cor.mat %>%
  cor_mark_significant()
```

```
##                     rowname      sex       age intrinsic identified introjected
## 1                       sex
## 2                       age    0.027
## 3                 intrinsic 0.22**** -0.16****
## 4                identified 0.13**** -0.16****  0.48****
## 5               introjected    0.039 -0.13****  0.29****   0.46****
## 6                  external   0.0094   -0.074*    -0.026   0.29****     0.5****
## 7    internalizedmotivation 0.21**** -0.19****  0.87****   0.85****    0.44****
## 8 internalizedmotivationlog  0.2**** -0.16****  0.84****   0.84****    0.43****
##   external internalizedmotivation internalizedmotivationlog
## 1
## 2
## 3
## 4
## 5
## 6
## 7 0.14****
## 8 0.15****               0.98****
```

```
# Draw correlogram using R base plot
cor.mat %>%
  cor_reorder() %>%
  pull_lower_triangle() %>%
  cor_plot()
```



5.3 Simple and Multiple Regression

Researchers use either bivariate regression (simple—one dependent and one independent) or multiple regression (more than one independent and one dependent) as even more nuanced models of influence between two or more variables in a dataset. Dependent variables in each of these scenarios are often called criterion, outcome, or response variable, while the independent variable is often called the predictor or explanatory variable.

Regression is used to find relationships between variables—similarly to correlation analysis—however, there are three major differences between correlation and regression.

Test Purpose

"While correlation analyses are used to determine relationships and the strength of relationships between variables, researchers use regression analysis to predict, based on information from another variable or set of variables, the outcome of one variable. Otherwise, researchers can use regression analysis as a tool to explain why, based on any given predictor variables, differences exist between participants" (Russell, 2018, p. 193).

Variable Names

There must be a dependent variable in regression that a researcher is trying to predict based on other predictor (independent) variables. This concept is not

used in correlation methods.

Regression Coefficients and Slope Intercepts

Researchers can use regression coefficients to infer beyond the direction and magnitude of relationships and discuss changes in Y due to specific changes in X (e.g., for every increase of 1 unit in Y, X increases .5)

The unstandardized regression formula is `Y = a + bX` where a is the intercept (where the line of best fit crosses the y-axis) and b is the slope of the regression line (½ is up 1 over 2). Put otherwise, a is the value of Y when X = 0 and b is the regression coefficient. The null hypothesis of the regression model is that independent variables are not able to significantly predict the dependent variable (i.e., the slope or standardized beta weight is 0, and that the predictor variable had no impact on the dependent variable).

Here's how you perform simple linear regression in R. This example uses the mtcars dataset which describes several measures of a set of cars. Let's model quarter mile time (qsec) as predicted by horsepower then examine the intercepts and coefficients before visualizing the relationship in a scatter plot.

The following code represents the unstandardized regression formula using the function `lm()` assigned to the object `mod`

```
mod <- lm(qsec ~ hp, data = mtcars)
```

Use `summary(mod)` to display the statistical output. See the asterisks for the level of significance, and the coefficient labelled 'estimate' as a description of the extent to which the target variable should change as the predictor variable increases by 1 unit. The r-squared, F-statistics, degrees of freedom, and p-value provide important model fit and parsimony statistics that provide important context for the interpretation of regression coefficients.

```
summary(mod)
```

```
##
## Call:
## lm(formula = qsec ~ hp, data = mtcars)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.1766 -0.6975  0.0348  0.6520  4.0972
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 20.556354   0.542424  37.897  < 2e-16 ***
## hp          -0.018458   0.003359  -5.495 5.77e-06 ***
```

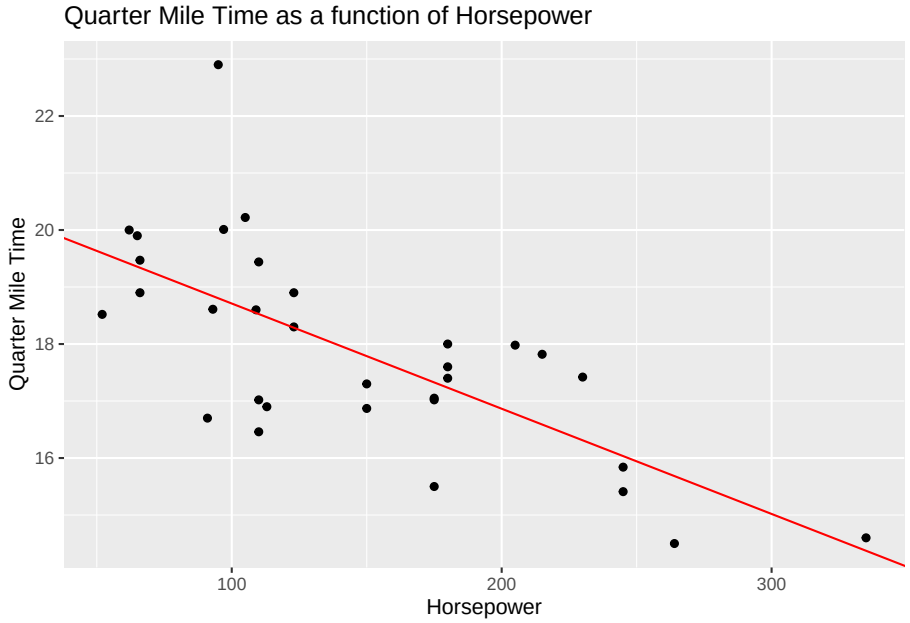
```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.282 on 30 degrees of freedom
## Multiple R-squared:  0.5016, Adjusted R-squared:  0.485
## F-statistic: 30.19 on 1 and 30 DF,  p-value: 5.766e-06
```

It's helpful to store the model coefficients as objects to be referred to in plotting procedures.

```
beta0 <- coef(mod)[1]
beta1 <- coef(mod)[2]
```

Use the `geom_point()` and `geom_abline()` functions to display a scatterplot with a line of best fit.

```
ggplot(mtcars, aes(x = hp, y = qsec)) +
  geom_point() +
  geom_abline(intercept = beta0, slope = beta1, colour='red') +
  ggtitle("Quarter Mile Time as a function of Horsepower") +
  xlab('Horsepower') +
  ylab('Quarter Mile Time')
```



Practically, these findings suggest that horsepower is a significant predictor of quarter mile time. More specifically, the model output tells us that quarter

mile time decreases by about 2 tenths of a second (.018) for each unit increase of horsepower in a given car. While statistically significant, this may seem practically insignificant. However, an increase of 100 horsepower tends to predict a loss of 1.8 seconds in quarter mile time, and a change in 300 horsepower tends to predict a loss of 5.4 seconds (which is good when the goal is speed). Is horsepower acting as a proxy for another variable? Put otherwise, would another variable (or set of variables) serve as better predictors of quarter mile time than hp?

Checking for Regression Assumptions

The assumptions of regression are as follows:

1. Linearity—relationship between dependent and independent variables is linear.

2. Homoscedasticity—the variance remains equal along the entire slope of the data

3. Normal distribution for variables included in the model.

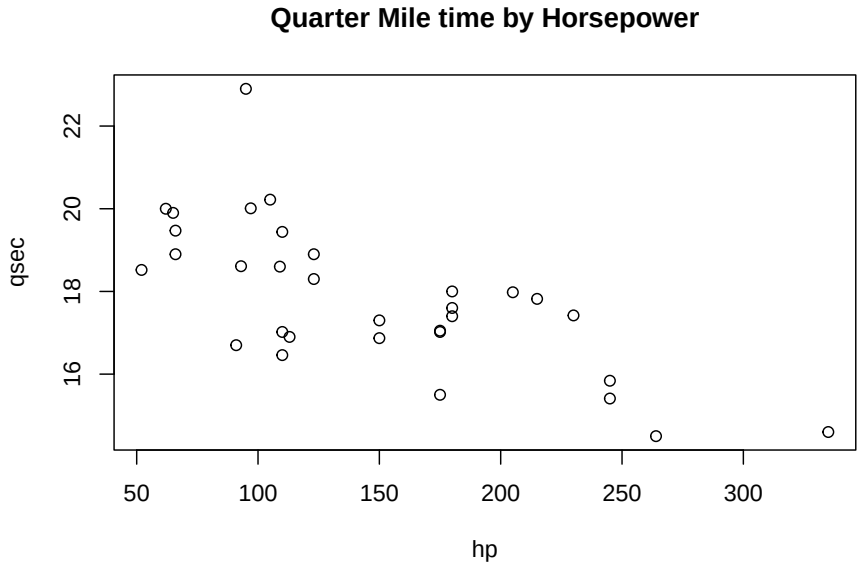
4. Number of variables—rule of thumb is that you have 10-20 participants per variable. Others argue less works (5).

5. No multicollinearity—no two independent variables should be highly correlated as they would then be redundant in the model (no new information to interpret).

Let's assess each of these assumptions for our model of quarter mile time as a function of horsepower.

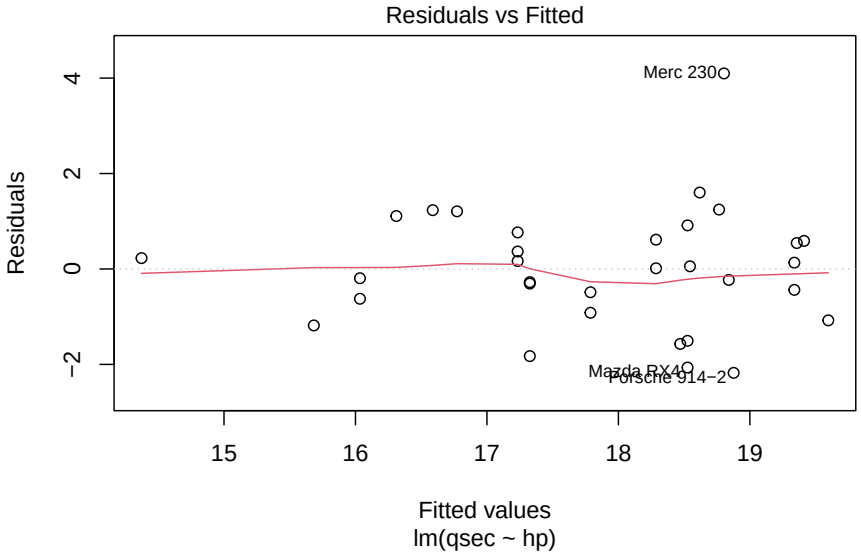
Are the data linear? Use a quickplot to visually assess linearity.

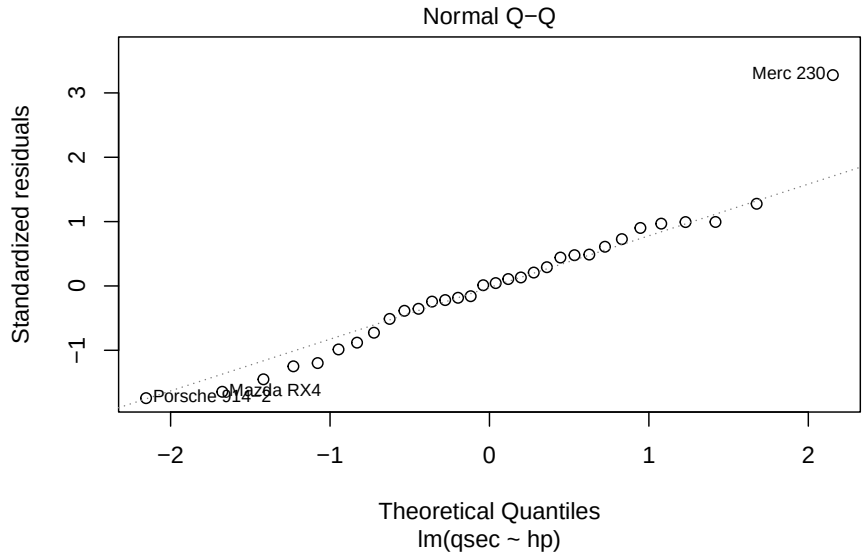
```
plot(qsec ~ hp, data = mtcars, main = 'Quarter Mile time by Horsepower') # plots raw data
```

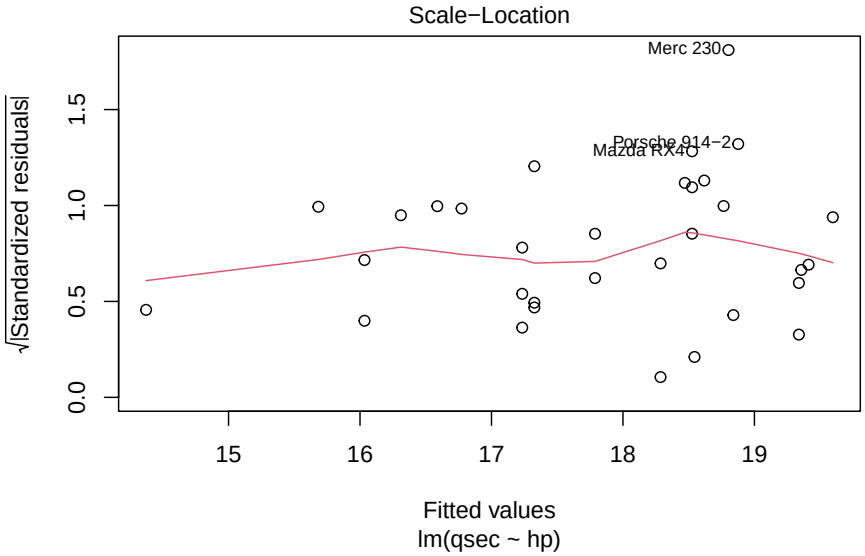


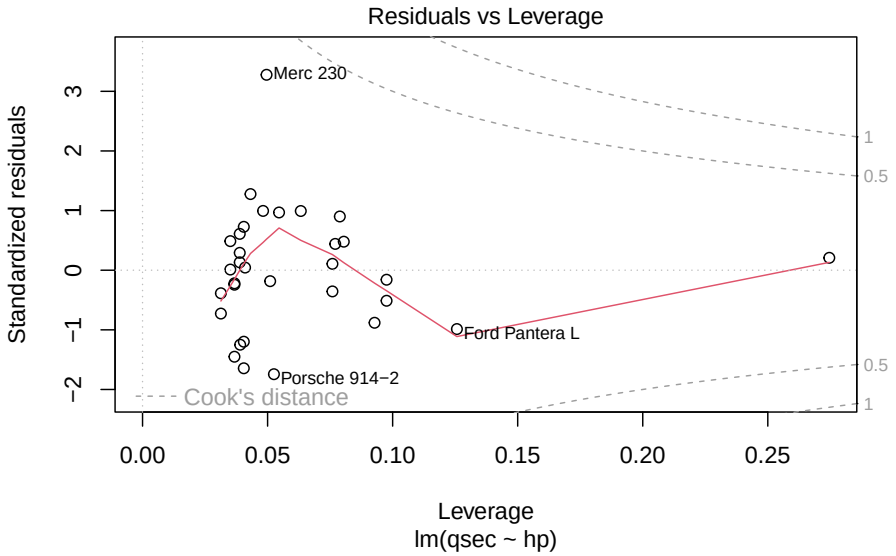
Are the data normally distributed? Use the `plot()` function to display output relevant to several assumptions.

```
plot(mod)
```




Normal Q−Q
Standardized residuals
Merc 230
Mazda RX4
Porsche 914−2
−2
−1
0
1
2
3
Theoretical Quantiles
lm(qsec ~ hp)


Scale−Location
√|Standardized residuals|
Merc 230
Porsche 914−2
Mazda RX4
0.0
0.5
1.0
1.5
15
16
17
18
19
Fitted values
lm(qsec ~ hp)



The **residuals vs. fitted** plot used to check linear relationship. A horizontal line without distinct patterns is an indication that the linear relationship is present. The **normal Q-Q** is used to examine whether residuals are normally distributed. If they are they will appear to be linear on this plot. The **scale-location** plot is used to check homogeneity of variance of residuals. A horizontal line with equally spread points is agood indication of homogeneity of variance. The **residuals vs. leverage** plot is used to identify potential outliers that are influencing regression results. The red lines are curves of constant **Cook's distance,** another metric to determine the influence of a point which combines leverage and residual size.

You do not want to see a pattern to residuals when ordered by independent variable as "the assumption of homoscedasticity (meaning"same variance") is central to linear regression models. Homoscedasticity describes a situation in which the error term (that is, the "noise" or random disturbance in the relationship between the independent variables and the dependent variable) is the same across all values of the independent variables. **Heteroscedasticity** (the violation of homoscedasticity) is present when the size of the error term differs across values of an independent variable. The impact of violating the assumption of homoscedasticity is a matter of degree, increasing as heteroscedasticity increases" (https://www.statisticssolutions.com/homoscedasticity/).

You can also check the multicollinearity assumption using variance inflation factors with the command `vif()`. Variation inflation factors greater than 5 or 10 (depending on the area of research) are often taken as indicative that there may be a multicollinearity problem. This is useful when constructing

models with multiple predictor variables—ensuring that two predictors are not too highly correlated and predicting the same variance in the outcome variable.

```
mod <- lm(qsec ~ hp + cyl, data = mtcars)
vif(mod)
```

```
##         GVIF Df GVIF^(1/(2*Df))
## hp  3.494957  1        1.869480
## cyl 3.494957  2        1.367289
```

5.3.1 Multiple Regression

To estimate multiple regression equations, simply add additional predictors. This allows you to eliminate confounds and provide evidence for causality with your model (given sufficient internal/external validity and appropriate sampling procedures). In the example below, Quarter Mile Time is modeled as a function of horsepower and engine volume (disp). The adjusted r-squared value tells us that 51% of the variance in quarter mile time was explained by variance in horsepower.

```
mod2 <- lm(qsec ~ hp + disp, data = mtcars)
summary(mod2)
```

```
##
## Call:
## lm(formula = qsec ~ hp + disp, data = mtcars)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.0266 -0.9156  0.1675  0.6109  4.1752
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 20.454114   0.531156  38.509  < 2e-16 ***
## hp          -0.025422   0.005339  -4.761 4.93e-05 ***
## disp         0.004870   0.002954   1.649     0.11
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.247 on 29 degrees of freedom
## Multiple R-squared:  0.5443, Adjusted R-squared:  0.5129
## F-statistic: 17.32 on 2 and 29 DF,  p-value: 1.124e-05
```

Let's model quarter mile time as a function of variance in mpg, number of cylinders, horsepower, and automatic/manual transmission.

```
mod2 <- lm(qsec ~ mpg + cyl + hp + am, data = mtcars)
summary(mod2)
```

```
##
## Call:
## lm(formula = qsec ~ mpg + cyl + hp + am, data = mtcars)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.7437 -0.6751  0.1015  0.5319  2.1426
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 20.426987   1.772405  11.525 1.02e-11 ***
## mpg         -0.012876   0.066295  -0.194 0.847512
## cyl.L       -1.951811   0.631743  -3.090 0.004730 **
## cyl.Q        0.173954   0.358954   0.485 0.632009
## hp          -0.008707   0.005815  -1.497 0.146337
## ammanual    -2.307268   0.513173  -4.496 0.000127 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.931 on 26 degrees of freedom
## Multiple R-squared:  0.7724, Adjusted R-squared:  0.7286
## F-statistic: 17.64 on 5 and 26 DF,  p-value: 1.249e-07
```

This model seems to indicate that the number of cylinders and type of transmission (am) are the strongest predictors of quarter mile time (-2.37 seconds per change in transmission type, and -0.70 seconds per change in cylinders). These variables collectively account for a larger portion of variance given the adjusted r-squared (used with multiple variables) of .74.

We can also use the `olsrr` library to run stepwise regression and identify the best fitting model. Insert a period instead of variables to call all variables in the dataset.

```
model <- lm(qsec ~ ., data = mtcars)
modelchoice <- ols_step_all_possible(model) %>% arrange(desc(adjr)) %>% filter(n <= 4)
head(modelchoice)
```

```
##   Index N               Predictors  R-Square Adj. R-Square Mallow's Cp
## 1   177 4  hp wt gear wtqsec_ratio 0.9735351     0.9684457    12.20563
## 2   176 4 cyl wt carb wtqsec_ratio 0.9750342     0.9648209    10.26807
## 3   180 4   mpg hp wt wtqsec_ratio 0.9661794     0.9611690    21.71273
```

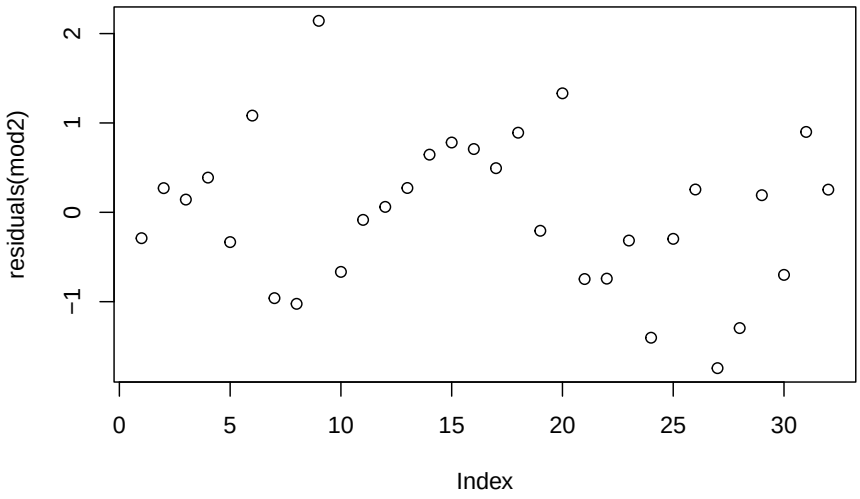
```
## 4   179 4   cyl hp wt wtqsec_ratio 0.9669874     0.9606388    20.66849
## 5   182 4    hp wt vs wtqsec_ratio 0.9648571     0.9596508    23.42181
## 6    56 3       hp wt wtqsec_ratio 0.9635221     0.9596137    23.14730
```

```
model2 <- lm(qsec ~ disp + wt + vs + carb, data=mtcars)
summary(model2)
```

```
##
## Call:
## lm(formula = qsec ~ disp + wt + vs + carb, data = mtcars)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.25620 -0.51160 -0.05549  0.37667  2.53603
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 13.000767   0.734959  17.689 6.90e-15 ***
## disp        -0.013590   0.003696  -3.677 0.001249 **
## wt           2.099573   0.399921   5.250 2.51e-05 ***
## vsstraight   1.319133   0.588394   2.242 0.034906 *
## carb.L      -3.180956   0.697400  -4.561 0.000139 ***
## carb.Q      -0.022039   0.586603  -0.038 0.970355
## carb.C       0.243124   0.571370   0.426 0.674420
## carb^4       0.006197   0.563858   0.011 0.991327
## carb^5       0.246119   0.495808   0.496 0.624323
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7901 on 23 degrees of freedom
## Multiple R-squared:  0.8549, Adjusted R-squared:  0.8045
## F-statistic: 16.95 on 8 and 23 DF,  p-value: 6.049e-08
```

After searching for the best model with the fewest number of predictors, it can be determined that displacement, weight, engine style (v shaped vs. straight) and carb explain the 82%, the same amount of variance in quarter mile time as the entire dataset explains.

```
plot(residuals(mod2))
```



You can store the elements of the summary such as r-squared.

```
summ_mod <- summary(mod)
summ_mod$residuals
summ_mod$r.squared
summ_mod$adj.r.squared
summ_mod$coefficients
```

Generalized linear models are fairly common. Logistic regression is used when the response variable is binary, e.g., did you vote (yes or no), do you have a diabetes (yes or no). Poisson regression is used when the response variable is a count (i.e., numeric but not continuous, e.g., how many heart attacks have you had, how many days from outbreak to infection). Fitting such models in R is very similar to how one would fit a linear model. E.g., for poisson regression, `glm(counts ~ outcome + treatment, data, family = poisson())`.

5.4 Inferential Statistics

5.4.1 Hypothesis Testing

A hypothesis is a proposed solution to a problem. Hypothesis testing is a means to take a rational (logical) decision based on observed data In music education, hypotheses tend to have an underlying goal of increased student success* (What

can success mean here?) After formulating some hypothesis (e.g., there is no difference in performance ability between 6th and 7th grade band students), we collect and use data to accept or reject the hypothesis (null in this case).

A null hypothesis H_0: a statement that represents the phenomenon you may want to provide evidence against (e.g., two means are the same where group difference is expected). An alternative hypothesis H_1: represents the opposite of the null hypothesis (e.g., two group means are statistically different).

For example, if you want to test whether automatic/manual transmission affect miles per gallon (mpg), then H_0 would be that manual and automatic transmission cars have the same mpg and H_1 would be that manual and automatic transmission cars have different mpg.

5.4.2 The Role of Statistics in Hypothesis Testing

Statistical tests are designed to reject or not reject a null hypothesis based on data. They do not allow you to find evidence for the alternative; just evidence for/against the null hypothesis. Assuming that H_0 is true, the p-value for a test is the probability of observing data as extreme or more extreme than what you actually observed (i.e., how surprised should I be about this?).

The significance level α is what you decide this probability should be (i.e., the surprise threshold that you will allow), that is, the probability of a Type I error (rejecting H_0 when it is true). Oftentimes, $\alpha = 5\% = 0.05$ is accepted although much lower alpha values are common in some research communities, and higher values are used for exploratory research in less defined fields. If we set $\alpha = 0.05$, we reject H_0 if p-value < 0.05 and say that we have found evidence against H_0.

Note that statistical tests and methods usually have assumptions about the data and the model. The accuracy of the analysis and the conclusions derived from an analysis depend crucially on the validity of these assumptions. So, it is important to know what those assumptions are and to determine whether they hold.

5.4.3 Checking Biases

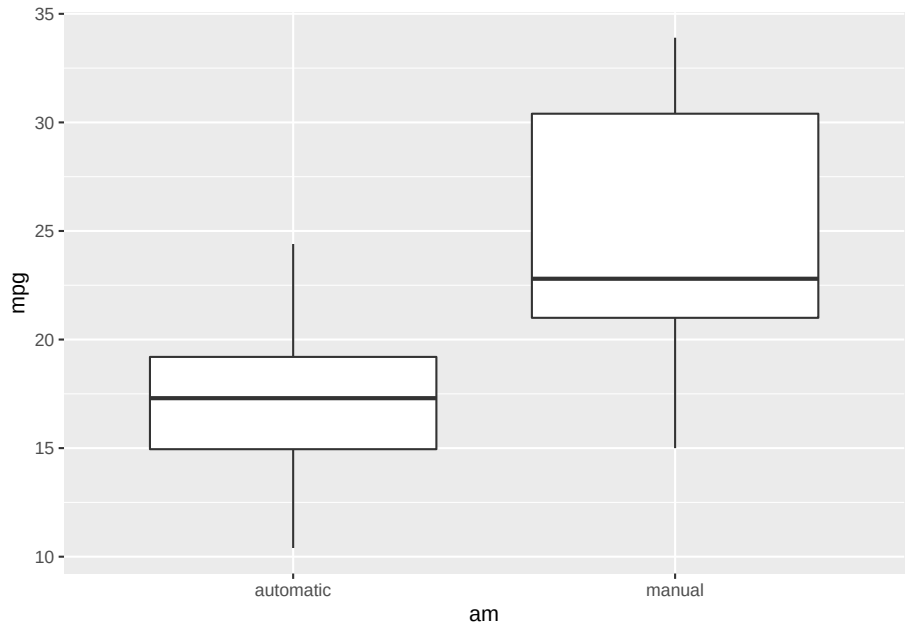
Biases are in reference to preconceived notions about how something does or does not work, an opinion you hold about how something should work, and your understanding of how the world operates. **Positivism** is an episitomological belief that the knowledge serves only to describe what we observe. However, we, the observer, are entrenched in known and unknown biases. In the view of the positivist, the world is deterministic, and can be modeled and understood in complete objectivity. As positivism does not account for our biases, social scientists shifted to a post-positivist view. **Post-positivism** holds space for our biases through the acknowledgement that scientific reasoning and our acquired

sense of the world are not the same. Most importantly, this epistemological view allows for the revision of theory, and holds that the goal of science is to describe reality in the most accurate way possible, understanding that we can never achieve that goal.

5.4.4 Tests of Group Differences

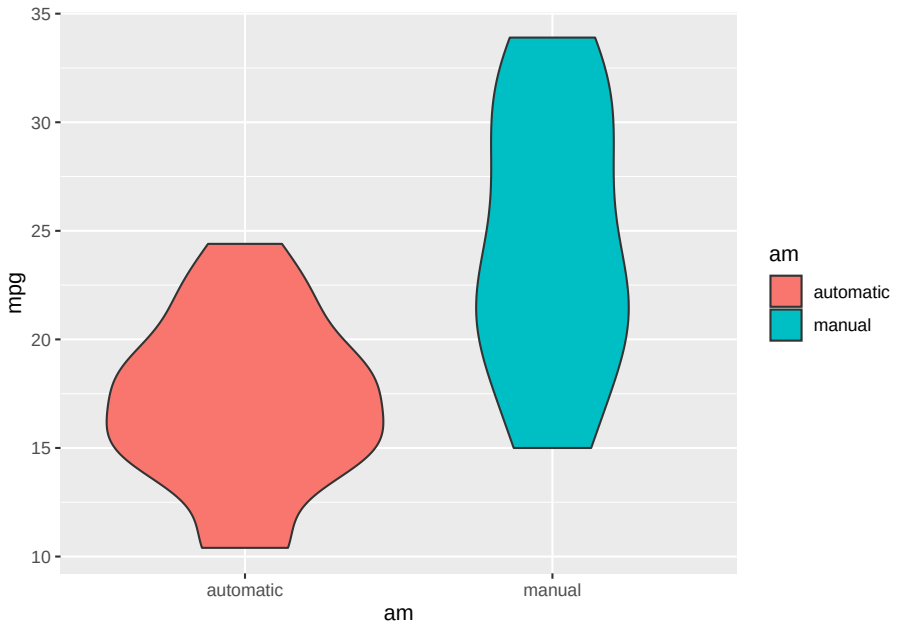
Before testing whether means are different among groups, it can be helpful to visualize the data distribution.

```
ggplot(mtcars) +
  geom_boxplot(aes(x=am, y=mpg))
```



Remember, violin plots are similar to box plots except that violin plots show the probability density of the data at different values.

```
ggplot(mtcars) +
  geom_violin(aes(x = am, y = mpg, fill = am))
```



T-tests can be used to test whether the means of two groups are significantly different. The null hypothesis H_0 assumes that the true difference in means μ_d is equal to zero. The alternative hypothesis can be that $\mu_d \neq 0$, $\mu_d > 0$, or $\mu_d < 0$. By default, the alternative hypothesis is that $\mu_d \neq 0$.

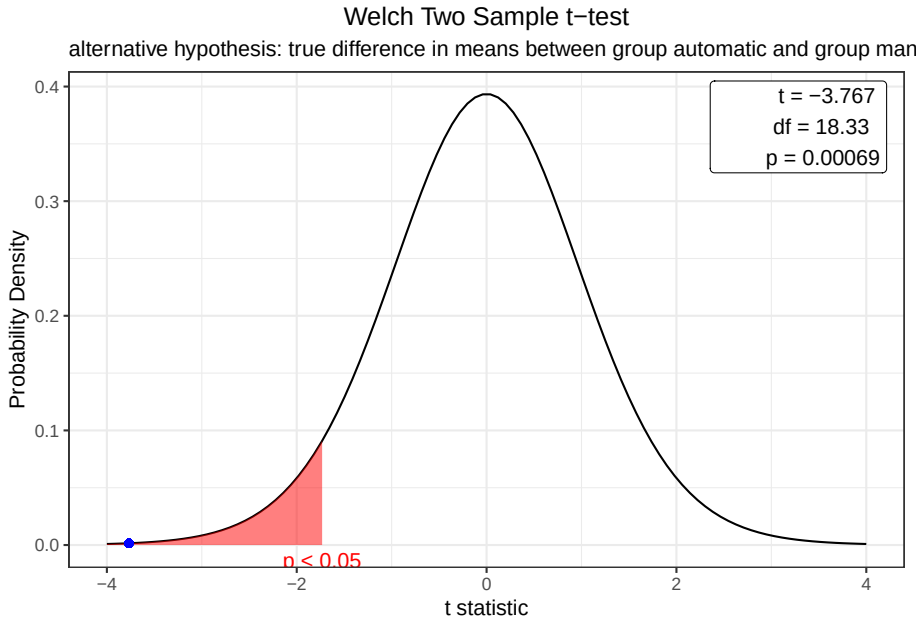
```
t.test(mtcars$mpg ~ mtcars$am, alternative="two.sided")
```

```
##
##  Welch Two Sample t-test
##
## data:  mtcars$mpg by mtcars$am
## t = -3.7671, df = 18.332, p-value = 0.001374
## alternative hypothesis: true difference in means between group automatic and group manual is n
## 95 percent confidence interval:
##  -11.280194  -3.209684
## sample estimates:
## mean in group automatic    mean in group manual
##                17.14737                24.39231
```

```
library(webr)
```

```
## Registered S3 method overwritten by 'ggforce':
##   method           from
##   scale_type.units units
```

```
model <- t.test(mtcars$mpg ~ mtcars$am, alternative = "less")
plot(model)
```



The t test results indicate that there is a significant difference in mpg between cars with a manual (0) and automatic (0) transmission (t = -3.77, p < .001). The webr library also allows you to easily produce distribution plots. Using this plot, we can determine that the null hypothesis will only be true (no difference between mpg by transmission type) < .01% of the time. Most differences are significant (p < .05) when t values exceed +- 1.96.

T-tests require a normality assumption which may or may not be valid depending on the situation. The function `wilcox.test` allows us compare group medians (not means) in a non-parametric setting (i.e., without the assumption of the data coming from some probability distribution). You may have heard of the Mann-Whitney U test. This test is just a two-sample Wilcoxon test. Is the median mpg different for automatic and manual cars?

```
wilcox.test(mtcars$mpg ~ mtcars$am)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  mtcars$mpg by mtcars$am
## W = 42, p-value = 0.001871
## alternative hypothesis: true location shift is not equal to 0
```

Yes, there is a significant difference in the median of mpg across transmission types (W = 42, p = .002)

5.4.5 Reporting Support

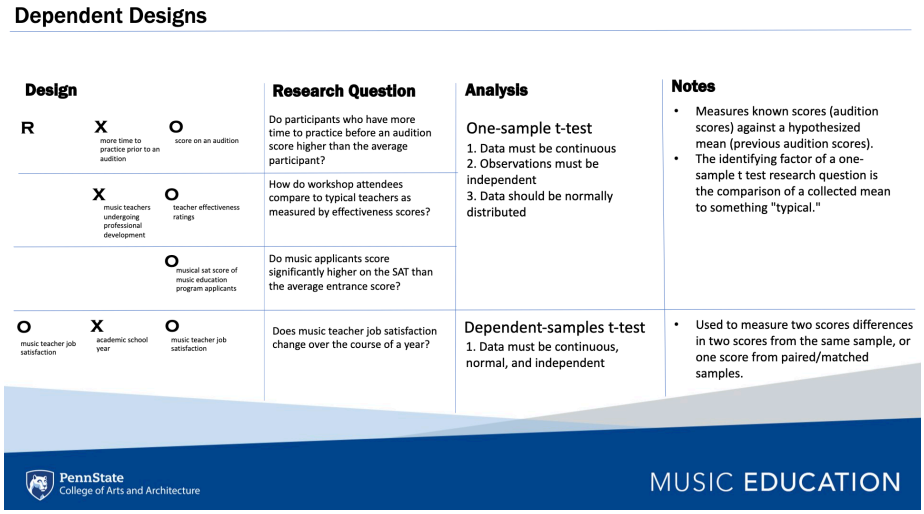
Writing up statistical reports are a vital skill for researchers. The report library supports automated APA compliant reports for statistical analyses. For instance, the previous t-test is assigned to an object, and that object is passed through the "report()" function. The output includes all the important facets for reporting a t-test—including confidence intervals, a t-value, degrees of freedom, a p-value, and Cohen's d (i.e., a reflection of the mean difference between groups in terms of standard deviations).

```
library(report)
ttestoutcome <- t.test(mtcars$mpg ~ mtcars$am, alternative="two.sided")
report(ttestoutcome)
```

```
## Effect sizes were labelled following Cohen's (1988) recommendations.
##
## The Welch Two Sample t-test testing the difference of mtcars$mpg by mtcars$am (mean in group a
```

5.4.6 Inferential Modeling

In this section, each chunk will include the code for running an univariate method of statistical analysis in `baseR` and other libraries. The figure below describes four separate dependent designs that are associated with two basic inferential models which can be analyzed using a one-sample t-test, where a mean with an unknown distribution (e.g., `M = 48`) is assessed against a known distribution of data. The dependent samples t-test is used to measure score differences from paired or matched samples.



5.4.6.1 One Sample T-Test

We're going to use the band participation dataset for these examples. For the one sample t-test, call in the vector where your data are loaded from, and use the argument `mu` to define the previously known mean (with an unknown distribution). Examine both the output of the statistical test and the automated written report.

```
data <- read_csv('participationdata.csv')
data <- data %>% drop_na(GradeLevel)

t.test(data$intentionscomp, mu = 5)
```

```
##
##  One Sample t-test
##
## data:  data$intentionscomp
## t = 9.5182, df = 39, p-value = 1.017e-11
## alternative hypothesis: true mean is not equal to 5
## 95 percent confidence interval:
##  11.18181 14.51819
## sample estimates:
## mean of x
##     12.85
```

```
onesamplewriteup <- t.test(data$intentionscomp, mu = 5)
report(onesamplewriteup)
```

```
## Effect sizes were labelled following Cohen's (1988) recommendations.
##
## The One Sample t-test testing the difference between data$intentionscomp (mean = 12.85) and mu
```

5.4.6.2 Dependent Sample T-Test

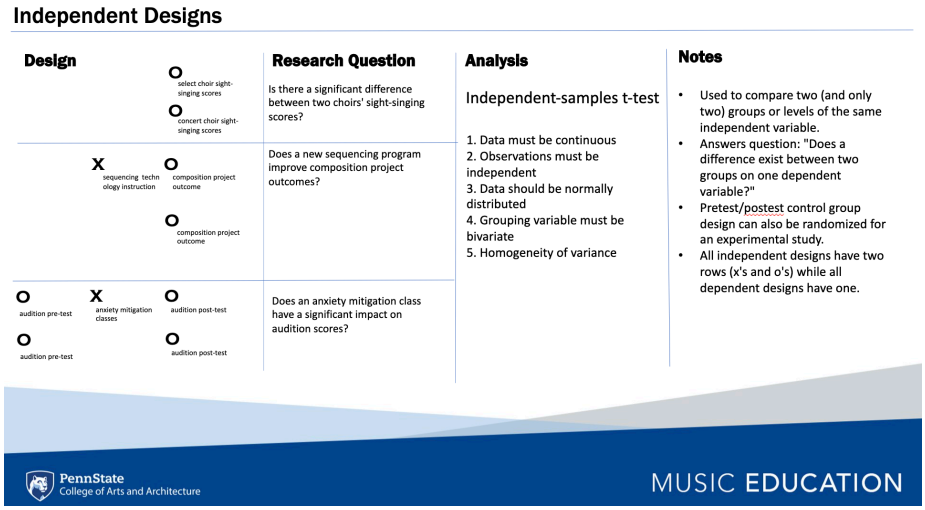
To run a dependent sample t-test, use the argument `paired = TRUE`

```
t.test(data$Intentions1, data$Intentions2, paired = TRUE)
```

```
##
##  Paired t-test
##
## data:  data$Intentions1 and data$Intentions2
## t = 0.1383, df = 39, p-value = 0.8907
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -0.3406334  0.3906334
## sample estimates:
## mean difference
##           0.025
```

```
dependentsamplewriteup <- t.test(data$Intentions1, data$Intentions2, paired = TRUE)
report(dependentsamplewriteup)
```

```
## Effect sizes were labelled following Cohen's (1988) recommendations.
##
## The Paired t-test testing the difference between data$Intentions1 and data$Intentions2 (mean d
```



Independent designs are used to compare a measures across differing samples, whereas dependent designs assess differences within a sample.

5.4.6.3 Independent Samples T-Test

Here is another chance to run a basic t-test

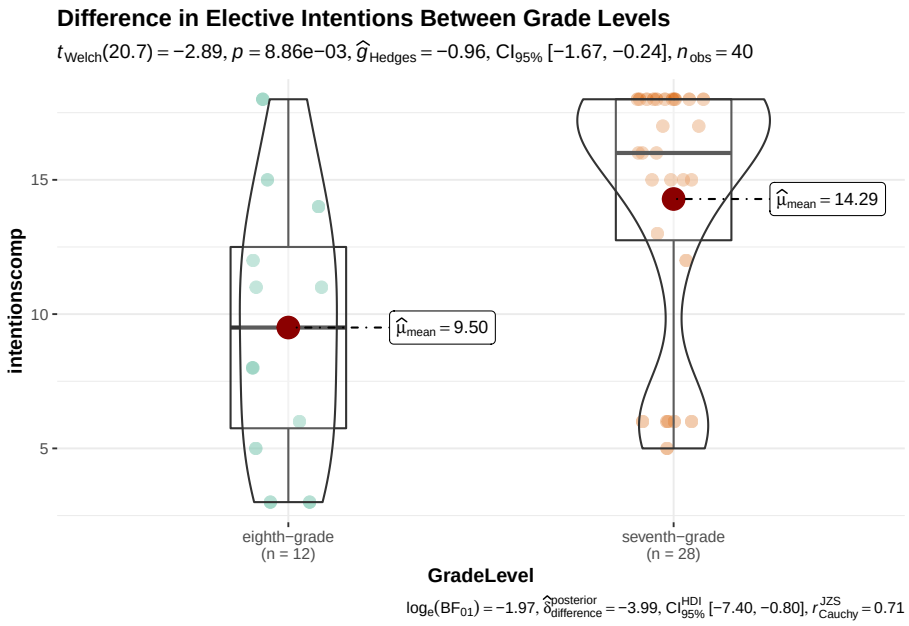
```
t.test(data$intentionscomp ~ data$GradeLevel, var.equal = T)
```

```
##
##  Two Sample t-test
##
## data:  data$intentionscomp by data$GradeLevel
## t = -2.9009, df = 38, p-value = 0.006156
## alternative hypothesis: true difference in means between group eighth-grade and grou
## 95 percent confidence interval:
##  -8.125373 -1.446055
## sample estimates:
##  mean in group eighth-grade mean in group seventh-grade
##                     9.50000                    14.28571
```

Let's try plotting the difference using `ggstatsplot`.

```
library(ggstatsplot)

ggbetweenstats(
  data = data,
  x = GradeLevel,
  y = intentionscomp,
  title = "Difference in Elective Intentions Between Grade Levels"
)
```



And for the write up?

```
ttestwriteup <- t.test(data$intentionscomp ~ data$GradeLevel, var.equal = T)
report(ttestwriteup)
```

```
## Effect sizes were labelled following Cohen's (1988) recommendations.
##
## The Two Sample t-test testing the difference of data$intentionscomp by data$GradeLevel (mean i
```

5.4.6.4 One-Way ANOVA

There are multiple new terms that emerge when discussing ANOVA approaches. These include:

- **Within-subjects factors**: the task(s) all participants did (i.e., repeated measures)
- **Between-subjects factors**: separate groups did certain conditions (e.g., levels of an experimental variable)
- **Mixed design**: differences were assessed for variables by participants (within-subjects), and something separates them into groups (e.g., between-subjects)
- A **main effect** is the overall effect of a variable on particular groups.

- Sometimes you have more than one IV, so you might have two main effects: one for Variable 1 and one for Variable 2.
- Also, you can have an **interaction** between the variables such that performance in "cells" of data is affected variably by the other variable. **You can report a significant p-value for the interaction line IF there is a significant main effect p-value.**
- You can only report a significant interaction if there is a significant main effect.
- Interpreting the main effect doesn't make sense: interpreting the interaction DOES (e.g., the main effect changes based on membership in one of the groups)

One-way ANOVAs are like t-tests, but they compare differences across more than two groups (i.e., categorical variables. For instance, in the participation dataset, you might compare socioeconomic status across the four schools contributing students to the sample. To run the ANOVA, first assign the model using the function `aov()` with the continuous comparison variable (e.g., SES and the categorical vector of comparison (e.g., School). To see the statistical output, run the `summary()` function with the model name inside the parenthesis.

```
model <- aov(SESComp ~ data$School, data = data)
summary(model)
```

```
##              Df Sum Sq Mean Sq F value Pr(>F)
## data$School   3  2.804  0.9345   7.881  4e-04 ***
## Residuals    34  4.032  0.1186
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## 2 observations deleted due to missingness
```

The output reflected a significant difference is present among the four schools. However, it is not possible to determine which schools are significantly different from others without Tukey's HSD, which allows you to examine mean differences between each combination of school. Note the school relationships with a p value below .05 to see what schools differ statistically.

```
TukeyHSD(model)
```

```
##   Tukey multiple comparisons of means
##     95% family-wise confidence level
##
## Fit: aov(formula = SESComp ~ data$School, data = data)
```

```
##
## $`data$School`
##                         diff        lwr         upr     p adj
## School2-School1  0.6673611  0.2721651  1.06255711 0.0003541
## School3-School1  0.2377315 -0.2141924  0.68965540 0.4955172
## School4-School1  0.3895833 -0.2400647  1.01923134 0.3542291
## School3-School2 -0.4296296 -0.8093217 -0.04993759 0.0215282
## School4-School2 -0.2777778 -0.8577669  0.30221139 0.5732634
## School4-School3  0.1518519 -0.4681826  0.77188635 0.9107596
```

The reporting function produces information regarding the main effect in the model. For more on how to report Tukey's HSD, visit https://statistics.laerd.com/spss-tutorials/one-way-anova-using-spss-statistics-2.php

```
report(model)
```

```
## For one-way between subjects designs, partial eta squared is equivalent to eta squared.
## Returning eta squared.

## Warning: 'data_findcols()' is deprecated and will be removed in a future update.
##   Its usage is discouraged. Please use 'data_find()' instead.

## Warning: Using `$` in model formulas can produce unexpected results. Specify your
##   model using the `data` argument instead.
##   Try: SESComp ~ School, data =
##   data

## The ANOVA (formula: SESComp ~ data$School) suggests that:
##
##   - The main effect of data$School is statistically significant and large (F(3, 34) = 7.88, p
##
## Effect sizes were labelled following Field's (2013) recommendations.
```

Factorial/Two-Way ANOVA

For factorial ANOVAs, there should be more than one categorical independent variable with one dependent variable (more dependent variables should be MANOVA) You are assessing first for main effects (**between groups**) and then for interaction effects **(within groups)**

Factorial ANOVAs are named more specifically for their number groupings Some examples are listed below:

- 2 x 2 x 2 ANOVA = Three independent variables with two levels each (e.g., gender, certified to teach or not, instrumentalist or not—main effects and interaction effects)

- 2 x 3 x 4 ANOVA = three independent variables, one has two levels (certified or not), one has three levels (teach in a rural, suburban, or urban school) and one has four levels (no music ed degree, music ed bachelors, music ed masters, music ed doctorate)
- 2 x 3 = two independent variables each one with two levels (gender) another with three levels (degree type)

To conduct a factorial ANOVA in R, use an asterisk to connect the two categorical variables of interest, as seen below. Then, use the summary function, Tukey HSD, and report function to interpret the output.

```
factorial <- aov(intentionscomp ~ Class*GradeLevel, data = data)
summary(factorial)
```

```
##                   Df Sum Sq Mean Sq F value  Pr(>F)
## Class              1   16.6   16.55   0.701 0.40783
## GradeLevel         1  178.1  178.14   7.548 0.00932 **
## Class:GradeLevel   1   16.8   16.82   0.713 0.40415
## Residuals         36  849.6   23.60
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
TukeyHSD(factorial)
```

```
##   Tukey multiple comparisons of means
##     95% family-wise confidence level
##
## Fit: aov(formula = intentionscomp ~ Class * GradeLevel, data = data)
##
## $Class
##                     diff       lwr     upr     p adj
## orchestra-band 1.288221 -1.831278 4.40772 0.4078281
##
## $GradeLevel
##                                 diff      lwr      upr     p adj
## seventh-grade-eighth-grade 4.525003 1.125608 7.924398 0.0105076
##
## $`Class:GradeLevel`
##                                                        diff         lwr       upr
## orchestra:eighth-grade-band:eighth-grade          2.6250000 -5.38701652 10.637017
## band:seventh-grade-band:eighth-grade              5.8365385 -0.04267434 11.715751
## orchestra:seventh-grade-band:eighth-grade         5.5083333 -0.21962035 11.236287
## band:seventh-grade-orchestra:eighth-grade         3.2115385 -4.26927785 10.692355
```

```
## orchestra:seventh-grade-orchestra:eighth-grade  2.8833333 -4.47920176 10.245868
## orchestra:seventh-grade-band:seventh-grade     -0.3282051 -5.28599268  4.629582
##                                                        p adj
## orchestra:eighth-grade-band:eighth-grade       0.8139391
## band:seventh-grade-band:eighth-grade           0.0522852
## orchestra:seventh-grade-band:eighth-grade      0.0631394
## band:seventh-grade-orchestra:eighth-grade      0.6577297
## orchestra:seventh-grade-orchestra:eighth-grade 0.7187103
## orchestra:seventh-grade-band:seventh-grade     0.9979489
```

```
report(factorial)
```

```
## Warning: 'data_findcols()' is deprecated and will be removed in a future update.
##   Its usage is discouraged. Please use 'data_find()' instead.
```

```
## The ANOVA (formula: intentionscomp ~ Class * GradeLevel) suggests that:
##
##   - The main effect of Class is statistically not significant and small (F(1, 36) = 0.70, p =
##   - The main effect of GradeLevel is statistically significant and large (F(1, 36) = 7.55, p =
##   - The interaction between Class and GradeLevel is statistically not significant and small (F
##
## Effect sizes were labelled following Field's (2013) recommendations.
```

The more complex a model is the larger the sample size needs to be. As such, the lack of a significant effect may be due to low study power rather than the lack of the phenomenon of interest.

The MANOVA is an extended technique which determines the effects of independent categorical variables on multiple continuous dependent variables. It is usually used to compare several groups with respect to multiple continuous variables.

5.4.6.5 ANCOVA

The Analysis of Covariance (ANCOVA) is used in examining the differences in the mean values of the dependent variables that are related to the effect of the controlled independent variables while taking into account the influence of the uncontrolled independent variables as covariates. Use an addition operator to add covariates to the model.

```
ancova_model <- aov(intentionscomp ~ GradeLevel + valuescomp, data = data)
summary(ancova_model, type="III")
```

```
##             Df Sum Sq Mean Sq F value  Pr(>F)
## GradeLevel   1  178.9  178.92    9.73 0.00362 **
## valuescomp   1  208.4  208.37   11.33 0.00186 **
## Residuals   35  643.6   18.39
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## 2 observations deleted due to missingness
```

```
report(ancova_model)
```

```
## Warning: 'data_findcols()' is deprecated and will be removed in a future update.
##   Its usage is discouraged. Please use 'data_find()' instead.
```

```
## The ANOVA (formula: intentionscomp ~ GradeLevel + valuescomp) suggests that:
##
##   - The main effect of GradeLevel is statistically significant and large (F(1, 35) =
##   - The main effect of valuescomp is statistically significant and large (F(1, 35) =
##
## Effect sizes were labelled following Field's (2013) recommendations.
```

5.5 Review

In this chapter we reviewed necessary skills for successful R coding, in addition to introducing correlational and inferential statistical modeling procedures. To make sure you understand this material, there is a practice assessment to go along with this and the previous chapter at https://jayholster.shinyapps.io/StatsinRAssessment/

5.6 References

Cui B., (2020). DataExplorer. https://cran.r-project.org/web/packages/DataExplorer/index.html

Fox, J., & Weisberg, S. (2019). *An R Companion to Applied Regression*, Third edition. Sage, Thousand Oaks CA. https://socialsciences.mcmaster.ca/jfox/Books/Companion/.

Kassambara, A. (2021). Rstatix. https://cran.r-project.org/web/packages/rstatix/index.html

Hebbali, A. (2020). olsrr. https://cran.r-project.org/web/packages/olsrr/index.html

Makowski, D., Lüdecke, D., Mattan S., Patil, I., Wiernik, B. M., & Siegel, R. (2022). report: Automated Reporting of Results and Statistical Models. https://cran.r-project.org/web/packages/report/index.html

Revelle, W. (2017) psych: Procedures for Personality and Psychological Research, Northwestern University, Evanston, Illinois, USA, <https://CRAN.R-project.org/package=psych> Version = 1.7.8.

Russell, J. A. (2018). *Statistics in music education research*. Oxford University Press.

Uenlue, A. (2018). Self-Determination Theory Measures. https://cran.r-project.org/web/packages/SDT/index.html

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L.D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T.L., Miller, E., Bache, S.M., Müller, K., Ooms, J., Robinson, D., Seidel, D.P., Spinu, V., Takahashi, K., Vaughan, D., Wilke, C., Woo, K., & Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software, 4*(43), 1686. https://doi.org/10.21105/joss.01686.

5.6.1 R Short Course Series

Video lectures of each guidebook chapter can be found at https://osf.io/6jb9t/. For this chapter, find the follow the folder path Statistics in R -> AY 2021-2022 Spring and access the video files, r markdown documents, and other materials for each short course.

5.6.2 Acknowledgements

This guidebook was created with support from the Center for Research Data and Digital Scholarship and the Laboratory for Interdisciplinary Statistical Analysis at the University of Colorado Boulder, as well as the U.S. Agency for International Development under cooperative agreement #7200AA18CA00022. Individuals who contributed to materials related to this project include Jacob Holster, Eric Vance, Michael Ramsey, Nicholas Varberg, and Nickoal Eichmann-Kalwara.

Chapter 6

Structural Equation Modeling

In this chapter, we will extend our statistical understandings regarding correlation and regression to the concept of Structural Equation Modeling (SEM).

SEM comprises a diverse set of analysis techniques, including confirmatory factor analysis and path analysis, each of which lend to the testing of theoretical models. Researchers who utilize SEM might review literature related to the constructs of interest (i.e., unobservable phenomena), and then hypothesize specific pathways within a plausible model through a set of regression equations and factor analyses, thus allowing for efficient exploration of complex relationships within and between psychological constructs. The variables within any model might include both measured variables and latent variables, or hypothetical constructs that cannot be directly observed (e.g., intelligence).

SEM utilizes maximum likelihood estimation (MLE), a statistical method which determines values for model parameters. Specifically, MLE maximizes the likelihood that the observed data will fit a theoretical model over other potential data distributions. SEM is straightforward in R Studio, but it is helpful to first build confidence in how you interact with R studio as a user.

6.1 SEM Syntax

SEM differs from multiple regression in that the individual items are allowed to contribute to the overall construct in an unequal manner, whereby in regression items are summed or averaged, thus limiting potential effects to aggregate scores of observable measures. Because of this level of specificity, latent variables can be removed in the event that items do not load appropriately on a construct.

Simple regressions, as well as multiple regression are simultaneously estimated within this model. Additionally, SEM affords the calculation of both direct and indirect effects.

Like in `baseR`, `lavaan` uses tildes the regression operator `~`, read as 'is regressed on'. The dependent variable is called prior to the tilde with independent variables called after the tilde. A typical path model is defined by a set of regression formulas.

```
dependent1 ~ independent1 + independent2 + independent3
```

```
## dependent1 ~ independent1 + independent2 + independent3
```

```
dependent2 ~ independent1 + independent2
```

```
## dependent2 ~ independent1 + independent2
```

Latent variables are called with `=~` which can be read as "is measured by."

```
latent1 =~ item1 + item2 + item3
latent2 =~ item4 + item5 + item6
```

Use double tildes `~~` to specify variances and covariances. This can be read as "is correlated with."

```
observed1 ~~ observed1 (variance)
observed1 ~~ observed2 (covariance)
```

Directly measure variable intercepts with the tilde `~` and the numeric value 1.

```
y1 ~ 1
f1 ~ 1
```

A specified Full SEM might look like this. This code will produce a model syntax object, called model that can be called later to attain model estimates and produce visualizations.

```
model <- ' # regressions
              targetvariable ~ construct1 + construct2 + observed1 + observed2
              construct1 ~ construct2 + construct3
              construct2 ~ construct3 + observed1 + observed2

            # latent variable definitions
```

```
    construct1 =~ item1 + item2 + item3
    construct2 =~ item4 + item5 + item6
    construct3 =~ item7 + item8 + item9 + item10

  # variances and covariances
    item1 ~~ item1
    item1 ~~ item2
    construct1 ~~ construct2

  # intercepts
    item1 ~ 1
    construct1 ~ 1
'
```

6.1.1 Running, Interpreting, and Visualizing Models

6.1.1.1 CFA

Confirmatory Factor Analysis estimates models comprised of latent variables that constitute the measurement model, which assesses how several paths might represent a theoretical assumption about the data. The measurement model only requires the =~sign to be used unless you are setting latent constructs as correlates in your CFA.

```
#### THIS CHUNK IS FOR TROUBLESHOOTING SEMPLOT INSTALLATION
#options(pkgType="binary")
#devtools::install_version("latticeExtra", version="0.6-28")
#install.packages('backports')
#install.packages("Hmisc")
#library(Hmisc)
```

Model fit is assessed model using 2, the **Comparative Fit Index** (CFI), and the **Root Mean Square Error of Approximation** (RMSEA). Each of these are examples of fit statistics, which are used to assess the extent to which a statistical model fits a set of observations. Fit statistics can also be used to compare sets of models against thresholds of good fit. For instance, a model with good fit should have a nonsignificant 2 value, though this fit statistic is sensitive to larger sample sizes. Thus, 2 is rarely used as a standalone measurement of model fit. The CFI compares the theoretical model to a null model and is considered to be demonstrative of good fit at or above 0.95, though models with a CFI above 0.90 are deemed to have acceptable fit. RMSEA is sensitive to parsimony of the model. In contrast with the CFI, a RMSEA value of 0.05 or below demonstrates good fit, however values below 0.1 are still considered acceptable.

We will be drawing from the `lavaan` package documentation for these explanatory examples. `lavaan` comes with three preinstalled datasets, the first of which is the "classic Holzinger and Swineford (1939) dataset [which] consists of mental ability test scores of seventh- and eighth-grade children from two different schools" (Pasteur and Grant-White). Demographic variables include an index, gender, age in years, age in months, school attended and grade level. Measured variables included assessments of visual perception, using cubes, lozenges, paragraph comprehension, sentence completion, word meaning, speeded addition, speeded counting of dots, and speeded text discrimination.

The authors hypothesized that these measured items might represent the latent constructs of visual, textual, and speed-related ability in educational contexts. The code below includes a specified model, a summary of the fit model, and a visualization of the model using the `semPlot` package.

```
#install.packages('lavaan')
#install.packages('semPlot')
library(lavaan)
library(semPlot)

# specify the model
HS.model <- ' visual ~ textual + speed

              visual  =~ x1 + x3
              textual =~ x4 + x5 + x6
              speed   =~ x7 + x8 + x9

              visual~~textual
              textual~~speed
              visual~~speed'

# fit the model
fit <- cfa(HS.model, data = HolzingerSwineford1939)
```

```
## Warning in lav_model_vcov(lavmodel = lavmodel, lavsamplestats = lavsamplestats, : la
##     Could not compute standard errors! The information matrix could
##     not be inverted. This may be a symptom that the model is not
##     identified.
```

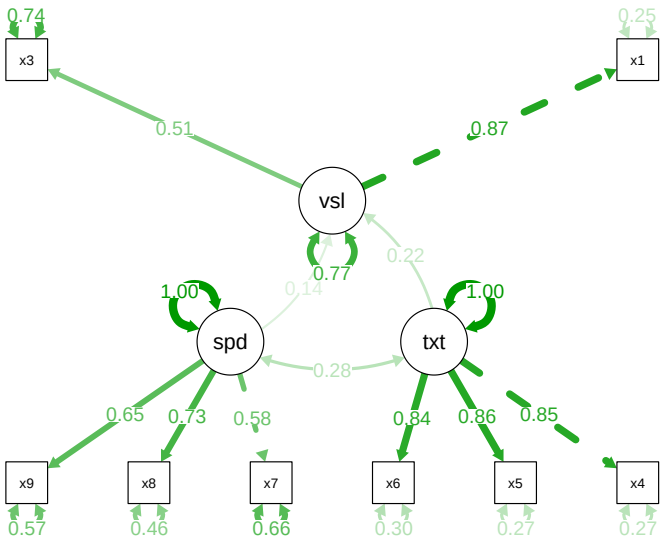
```
# display summary output
summary(fit, fit.measures=TRUE, standardized=TRUE)
```

```
## lavaan 0.6-11 ended normally after 31 iterations
##
##   Estimator                                         ML
```

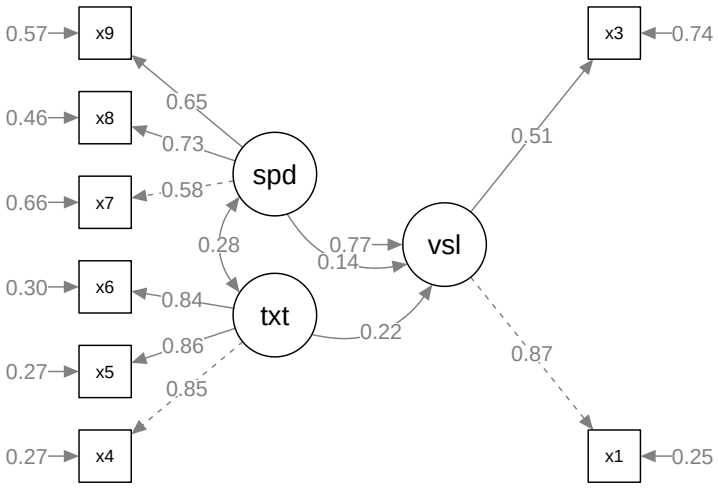
```
##   Optimization method                           NLMINB
##   Number of model parameters                        21
##
##   Number of observations                           301
##
## Model Test User Model:
##
##   Test statistic                                66.892
##   Degrees of freedom                                15
##   P-value (Chi-square)                           0.000
##
## Model Test Baseline Model:
##
##   Test statistic                               861.843
##   Degrees of freedom                                28
##   P-value                                        0.000
##
## User Model versus Baseline Model:
##
##   Comparative Fit Index (CFI)                    0.938
##   Tucker-Lewis Index (TLI)                       0.884
##
## Loglikelihood and Information Criteria:
##
##   Loglikelihood user model (H0)              -3281.274
##   Loglikelihood unrestricted model (H1)      -3247.828
##
##   Akaike (AIC)                                6604.548
##   Bayesian (BIC)                              6682.397
##   Sample-size adjusted Bayesian (BIC)         6615.797
##
## Root Mean Square Error of Approximation:
##
##   RMSEA                                          0.107
##   90 Percent confidence interval - lower         0.082
##   90 Percent confidence interval - upper         0.134
##   P-value RMSEA <= 0.05                          0.000
##
## Standardized Root Mean Square Residual:
##
##   SRMR                                           0.060
##
## Parameter Estimates:
##
##   Standard errors                             Standard
##   Information                                 Expected
```

```
##   Information saturated (h1) model          Structured
##
## Latent Variables:
##                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   visual =~
##     x1               1.000                               1.012    0.868
##     x3               0.566       NA                      0.573    0.508
##   textual =~
##     x4               1.000                               0.991    0.853
##     x5               1.112       NA                      1.102    0.855
##     x6               0.923       NA                      0.915    0.837
##   speed =~
##     x7               1.000                               0.630    0.579
##     x8               1.174       NA                      0.739    0.731
##     x9               1.042       NA                      0.657    0.652
##
## Regressions:
##                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   visual ~
##     textual          0.230       NA                      0.225    0.225
##     speed            0.217       NA                      0.135    0.135
##
## Covariances:
##                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  .visual ~~
##     textual          0.186       NA                      0.212    0.212
##   textual ~~
##     speed            0.174       NA                      0.279    0.279
##  .visual ~~
##     speed            0.148       NA                      0.264    0.264
##
## Variances:
##                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .x1               0.335       NA                      0.335    0.246
##    .x3               0.946       NA                      0.946    0.742
##    .x4               0.369       NA                      0.369    0.273
##    .x5               0.446       NA                      0.446    0.269
##    .x6               0.359       NA                      0.359    0.300
##    .x7               0.786       NA                      0.786    0.664
##    .x8               0.475       NA                      0.475    0.465
##    .x9               0.584       NA                      0.584    0.575
##    .visual           0.786       NA                      0.768    0.768
##     textual          0.982       NA                      1.000    1.000
##     speed            0.397       NA                      1.000    1.000
```

```
#visualize output with standard estimates
semPaths(fit, "std", edge.label.cex = 1.0, curvePivot = FALSE, rotation = 3)
```




```
semPaths(fit, what = "path", whatLabels = "std", style = "lisrel", edge.label.cex = .9, rotation
```



The output indicates a satisfactory fitting model despite the significant chi-square result (CFI = .93, RMSEA = .092, p = .001). Visual factors x1 and x3 loaded as expected, while x2 is approaching the lower bound for acceptable factor loading. As no items were non-significant, we might consider removing this item in a subsequent analysis to improve overall model fit. The textual and speed factor loadings were each satisfactory with the textual model holding strong consistency across each measured item (all loaded onto factor with a standardized estimate of .85 or .84). Covariances were satisfactory for this model (visual to textual is .46, visual to speed is .47, textual to speed is .28).

When we remove x2 from the model the CFI is improved at the expense of the RMSEA value. As there are no other benefits to removing this item, it should be retained. This model can be visualized using the `semPlot()` package. The `semPath()` function in particular is what we use to plot our SEM diagram. The `semPaths()` function takes the following arguments. For a description of each, see https://www.rdocumentation.org/packages/semPlot/versions/1.1.2/topics/semPaths

```
semPaths(object, what = "paths", whatLabels, style, layout = "tree",
  intercepts = TRUE, residuals = TRUE, thresholds = TRUE, intStyle = "multi",
  rotation = 1, curve, curvature = 1, nCharNodes = 3, nCharEdges = 3, sizeMan = 5,
   sizeLat = 8,  sizeInt = 2, sizeMan2, sizeLat2, sizeInt2, shapeMan, shapeLat,
  shapeInt = "triangle", ask, mar, title, title.color = "black", title.adj = 0.1,
  title.line = -1, title.cex = 0.8, include, combineGroups = FALSE, manifests,
  latents, groups, color, residScale, gui = FALSE, allVars = FALSE, edge.color,
  reorder = TRUE, structural = FALSE, ThreshAtSide = FALSE, thresholdColor,
```

```
  thresholdSize = 0.5, fixedStyle = 2, freeStyle = 1,
  as.expression = character(0), optimizeLatRes = FALSE, inheritColor = TRUE,
  levels, nodeLabels, edgeLabels, pastel = FALSE, rainbowStart = 0, intAtSide,
  springLevels = FALSE, nDigits = 2, exoVar, exoCov = TRUE, centerLevels = TRUE,
  panelGroups = FALSE, layoutSplit = FALSE, measurementLayout = "tree", subScale,
  subScale2, subRes = 4, subLinks, modelOpts = list(mplusStd = "std"),
  curveAdjacent = '<->', edge.label.cex = 0.6,  cardinal = "none",
  equalizeManifests = FALSE, covAtResiduals = TRUE, bifactor, optimPoints = 1:8 * (pi/4),
  ...)
```

```
## Error in eval(expr, envir, enclos): '...' used in an incorrect context
```

6.1.1.2 Full SEM

We will be using another `lavaan` dataset for a Full SEM example. The PoliticalDemocracy datasets includes 75 observations of 11 variables that measure political democracy and industrialization in developing countries. Variables are labeled with the following descriptions:

Political Democracy in 1960

- y1: Expert ratings of the freedom of the press in 1960
- y2: The freedom of political opposition in 1960
- y3: The fairness of elections in 1960
- y4: The effectiveness of the elected legislature in 1960

Political Democracy in 1965

- y5: Expert ratings of the freedom of the press in 1965
- y6: The freedom of political opposition in 1965
- y7: The fairness of elections in 1965
- y8: The effectiveness of the elected legislature in 1965

Level of Industrialization in 1960

- x1: The gross national product (GNP) per capita in 1960
- x2: The inanimate energy consumption per capita in 1960
- x3: The percentage of the labor force in industry in 1960

The bold headers represent latent, unobservable variables that are quantified by the observable bullet points. Take a quick look at the dataset using any of the methods covered thus far.

```
str(PoliticalDemocracy)
```

```
## 'data.frame':    75 obs. of  11 variables:
##  $ y1: num  2.5 1.25 7.5 8.9 10 7.5 7.5 7.5 2.5 10 ...
##  $ y2: num  0 0 8.8 8.8 3.33 ...
##  $ y3: num  3.33 3.33 10 10 10 ...
##  $ y4: num  0 0 9.2 9.2 6.67 ...
##  $ y5: num  1.25 6.25 8.75 8.91 7.5 ...
##  $ y6: num  0 1.1 8.09 8.13 3.33 ...
##  $ y7: num  3.73 6.67 10 10 10 ...
##  $ y8: num  3.333 0.737 8.212 4.615 6.667 ...
##  $ x1: num  4.44 5.38 5.96 6.29 5.86 ...
##  $ x2: num  3.64 5.06 6.26 7.57 6.82 ...
##  $ x3: num  2.56 3.57 5.22 6.27 4.57 ...
```

```
psych::describe(PoliticalDemocracy)
```

```
##    vars  n mean   sd median trimmed  mad  min   max range  skew kurtosis   se
## y1    1 75 5.46 2.62   5.40    5.46 3.11 1.25 10.00  8.75 -0.09    -1.15 0.30
## y2    2 75 4.26 3.95   3.33    4.09 4.94 0.00 10.00 10.00  0.32    -1.47 0.46
## y3    3 75 6.56 3.28   6.67    6.92 4.94 0.00 10.00 10.00 -0.59    -0.72 0.38
## y4    4 75 4.45 3.35   3.33    4.33 4.94 0.00 10.00 10.00  0.12    -1.21 0.39
## y5    5 75 5.14 2.61   5.00    5.23 3.71 0.00 10.00 10.00 -0.23    -0.78 0.30
## y6    6 75 2.98 3.37   2.23    2.51 3.31 0.00 10.00 10.00  0.89    -0.47 0.39
## y7    7 75 6.20 3.29   6.67    6.47 4.94 0.00 10.00 10.00 -0.55    -0.73 0.38
## y8    8 75 4.04 3.25   3.33    3.82 4.40 0.00 10.00 10.00  0.45    -0.96 0.37
## x1    9 75 5.05 0.73   5.08    5.03 0.82 3.78  6.74  2.95  0.25    -0.75 0.08
## x2   10 75 4.79 1.51   4.96    4.85 1.53 1.39  7.87  6.49 -0.35    -0.57 0.17
## x3   11 75 3.56 1.41   3.57    3.53 1.51 1.00  6.42  5.42  0.08    -0.94 0.16
```

A model is specified below. Run, interpret, and visualize the model.

```
model <- '
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8
  # regressions
    dem60 ~ ind60
    dem65 ~ ind60 + dem60
```

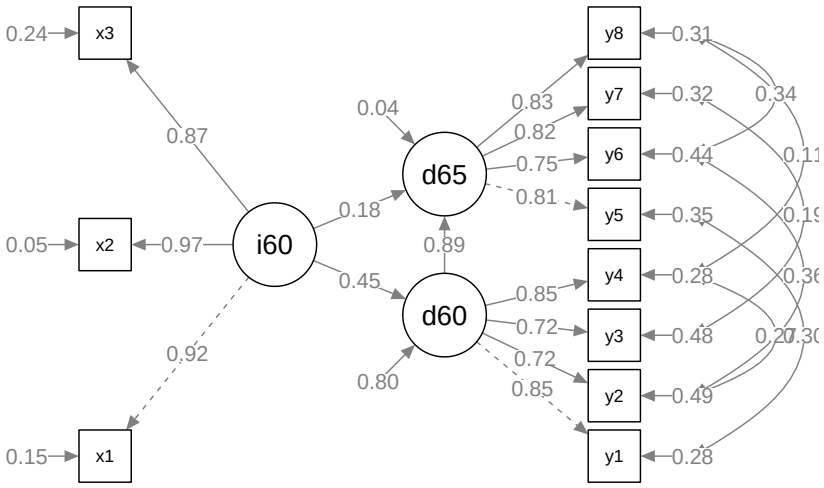
```
  # residual correlations
    y1 ~~ y5
    y2 ~~ y4 + y6
    y3 ~~ y7
    y4 ~~ y8
    y6 ~~ y8
'
fit <- sem(model, data=PoliticalDemocracy)
summary(fit, fit.measures = TRUE, standardized=TRUE)
```

```
## lavaan 0.6-11 ended normally after 68 iterations
##
##   Estimator                                         ML
##   Optimization method                           NLMINB
##   Number of model parameters                        31
##
##   Number of observations                            75
##
## Model Test User Model:
##
##   Test statistic                                38.125
##   Degrees of freedom                                35
##   P-value (Chi-square)                           0.329
##
## Model Test Baseline Model:
##
##   Test statistic                               730.654
##   Degrees of freedom                                55
##   P-value                                        0.000
##
## User Model versus Baseline Model:
##
##   Comparative Fit Index (CFI)                    0.995
##   Tucker-Lewis Index (TLI)                       0.993
##
## Loglikelihood and Information Criteria:
##
##   Loglikelihood user model (H0)              -1547.791
##   Loglikelihood unrestricted model (H1)      -1528.728
##
##   Akaike (AIC)                                3157.582
##   Bayesian (BIC)                              3229.424
##   Sample-size adjusted Bayesian (BIC)         3131.720
##
## Root Mean Square Error of Approximation:
```

```
##
##   RMSEA                                          0.035
##   90 Percent confidence interval - lower         0.000
##   90 Percent confidence interval - upper         0.092
##   P-value RMSEA <= 0.05                          0.611
##
## Standardized Root Mean Square Residual:
##
##   SRMR                                           0.044
##
## Parameter Estimates:
##
##   Standard errors                             Standard
##   Information                                 Expected
##   Information saturated (h1) model          Structured
##
## Latent Variables:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   ind60 =~
##     x1                1.000                               0.670    0.920
##     x2                2.180    0.139   15.742    0.000    1.460    0.973
##     x3                1.819    0.152   11.967    0.000    1.218    0.872
##   dem60 =~
##     y1                1.000                               2.223    0.850
##     y2                1.257    0.182    6.889    0.000    2.794    0.717
##     y3                1.058    0.151    6.987    0.000    2.351    0.722
##     y4                1.265    0.145    8.722    0.000    2.812    0.846
##   dem65 =~
##     y5                1.000                               2.103    0.808
##     y6                1.186    0.169    7.024    0.000    2.493    0.746
##     y7                1.280    0.160    8.002    0.000    2.691    0.824
##     y8                1.266    0.158    8.007    0.000    2.662    0.828
##
## Regressions:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   dem60 ~
##     ind60             1.483    0.399    3.715    0.000    0.447    0.447
##   dem65 ~
##     ind60             0.572    0.221    2.586    0.010    0.182    0.182
##     dem60             0.837    0.098    8.514    0.000    0.885    0.885
##
## Covariances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  .y1 ~~
##    .y5                0.624    0.358    1.741    0.082    0.624    0.296
##  .y2 ~~
```

```
##    .y4                1.313    0.702    1.871    0.061    1.313    0.273
##    .y6                2.153    0.734    2.934    0.003    2.153    0.356
##  .y3 ~~
##    .y7                0.795    0.608    1.308    0.191    0.795    0.191
##  .y4 ~~
##    .y8                0.348    0.442    0.787    0.431    0.348    0.109
##  .y6 ~~
##    .y8                1.356    0.568    2.386    0.017    1.356    0.338
##
## Variances:
##                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .x1                0.082    0.019    4.184    0.000    0.082    0.154
##    .x2                0.120    0.070    1.718    0.086    0.120    0.053
##    .x3                0.467    0.090    5.177    0.000    0.467    0.239
##    .y1                1.891    0.444    4.256    0.000    1.891    0.277
##    .y2                7.373    1.374    5.366    0.000    7.373    0.486
##    .y3                5.067    0.952    5.324    0.000    5.067    0.478
##    .y4                3.148    0.739    4.261    0.000    3.148    0.285
##    .y5                2.351    0.480    4.895    0.000    2.351    0.347
##    .y6                4.954    0.914    5.419    0.000    4.954    0.443
##    .y7                3.431    0.713    4.814    0.000    3.431    0.322
##    .y8                3.254    0.695    4.685    0.000    3.254    0.315
##     ind60             0.448    0.087    5.173    0.000    1.000    1.000
##    .dem60             3.956    0.921    4.295    0.000    0.800    0.800
##    .dem65             0.172    0.215    0.803    0.422    0.039    0.039
```

```
semPaths(fit, what = "path", whatLabels = "std", style = "lisrel", edge.label.cex = .9, rotation
```



The results suggest that the model has excellent fit with a non significant chi square, a CFI of .995, and a RMSEA value .035. All factors (Ind60 x1 = .92, x2 = .97, x3 = .87; Dem60 y1 = .85, y2 = .71, y3 = .72, y4 = .85; Dem65 y5 = .81, y6 = .75, y7 = .82, y = .83) and paths (Dem60 ~ ind60 = .45; Dem65 ~ ind60(.18) + dem60(.89)) were significant and there was no evidence of colinearity in the model. Pruning would not be advisable as the model demonstrates outstanding parsimony. How do you interpret these findings on a practical level?

6.1.1.3 Mediation

We can create mediation models where intermediate variables are considered and called the mediator. We use these variables to attempt to explain the impact of an independent variable on an outcome. For instance, it is logical to deduce that past democratization of a country (d60) would predict future democratization (d65). However, it is questionable whether the industrialization of a country predicts a move toward democratization over time.

Let's set up a mediation model to answer the research question: "To what extent does a country's level of industrialization impact the development of political democracy over time?" Retain the same latent variables from the previous model. To create a mediation model, use the following formula:

direct effect Y ~ c*X # mediator M ~ a*X Y ~ b*M # indirect effect (a*b) ab := a*b # total effect total := c + (a*b)

The := operator "defines" new parameters which take on values that are an arbitrary function of the original model parameters" (https://lavaan.ugent.be/tutorial/mediation.html).

```
model <-  '
  # measurement model
    ind60 =~ x1 + x2 + x3
    dem60 =~ y1 + y2 + y3 + y4
    dem65 =~ y5 + y6 + y7 + y8

  # mediation
    dem65 ~ c*dem60
    ind60 ~ a*dem60
    dem65 ~ b*ind60

  # indirect effect (a*b)
    ab := a*b

  # total effect
    total := c + (a*b)

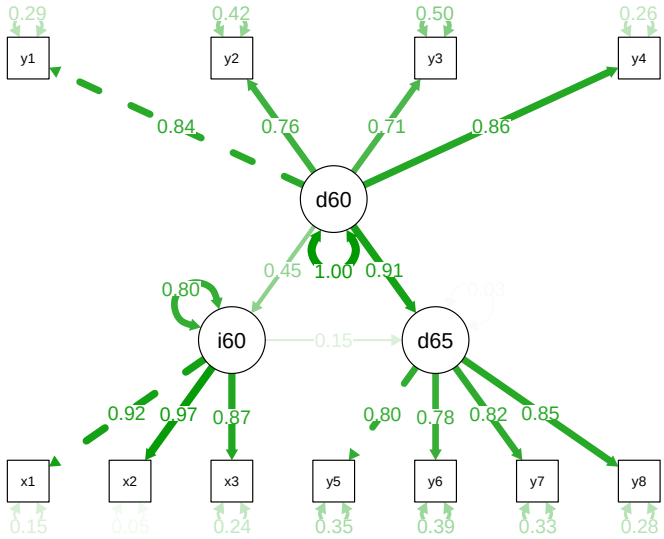
'
fit <- sem(model, data=PoliticalDemocracy)
summary(fit, fit.measures = TRUE, standardized=TRUE)
```

```
## lavaan 0.6-11 ended normally after 37 iterations
##
##   Estimator                                         ML
##   Optimization method                           NLMINB
##   Number of model parameters                        25
##
##   Number of observations                            75
##
## Model Test User Model:
##
##   Test statistic                                72.462
##   Degrees of freedom                                41
##   P-value (Chi-square)                           0.002
##
## Model Test Baseline Model:
##
##   Test statistic                               730.654
##   Degrees of freedom                                55
##   P-value                                        0.000
##
```

```
## User Model versus Baseline Model:
##
##   Comparative Fit Index (CFI)                    0.953
##   Tucker-Lewis Index (TLI)                       0.938
##
## Loglikelihood and Information Criteria:
##
##   Loglikelihood user model (H0)              -1564.959
##   Loglikelihood unrestricted model (H1)      -1528.728
##
##   Akaike (AIC)                                3179.918
##   Bayesian (BIC)                              3237.855
##   Sample-size adjusted Bayesian (BIC)         3159.062
##
## Root Mean Square Error of Approximation:
##
##   RMSEA                                          0.101
##   90 Percent confidence interval - lower         0.061
##   90 Percent confidence interval - upper         0.139
##   P-value RMSEA <= 0.05                          0.021
##
## Standardized Root Mean Square Residual:
##
##   SRMR                                           0.055
##
## Parameter Estimates:
##
##   Standard errors                             Standard
##   Information                                 Expected
##   Information saturated (h1) model          Structured
##
## Latent Variables:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   ind60 =~
##     x1                1.000                               0.669    0.920
##     x2                2.182    0.139   15.714    0.000    1.461    0.973
##     x3                1.819    0.152   11.956    0.000    1.218    0.872
##   dem60 =~
##     y1                1.000                               2.201    0.845
##     y2                1.354    0.175    7.755    0.000    2.980    0.760
##     y3                1.044    0.150    6.961    0.000    2.298    0.705
##     y4                1.300    0.138    9.412    0.000    2.860    0.860
##   dem65 =~
##     y5                1.000                               2.084    0.803
##     y6                1.258    0.164    7.651    0.000    2.623    0.783
##     y7                1.282    0.158    8.137    0.000    2.673    0.819
```

```
##     y8                1.310    0.154    8.529    0.000    2.730    0.847
##
## Regressions:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   dem65 ~
##     dem60      (c)    0.864    0.113    7.671    0.000    0.913    0.913
##   ind60 ~
##     dem60      (a)    0.136    0.036    3.738    0.000    0.448    0.448
##   dem65 ~
##     ind60      (b)    0.453    0.220    2.064    0.039    0.146    0.146
##
## Variances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .x1                0.082    0.020    4.180    0.000    0.082    0.154
##    .x2                0.118    0.070    1.689    0.091    0.118    0.053
##    .x3                0.467    0.090    5.174    0.000    0.467    0.240
##    .y1                1.942    0.395    4.910    0.000    1.942    0.286
##    .y2                6.490    1.185    5.479    0.000    6.490    0.422
##    .y3                5.340    0.943    5.662    0.000    5.340    0.503
##    .y4                2.887    0.610    4.731    0.000    2.887    0.261
##    .y5                2.390    0.447    5.351    0.000    2.390    0.355
##    .y6                4.343    0.796    5.456    0.000    4.343    0.387
##    .y7                3.510    0.668    5.252    0.000    3.510    0.329
##    .y8                2.940    0.586    5.019    0.000    2.940    0.283
##    .ind60             0.358    0.071    5.020    0.000    0.799    0.799
##     dem60             4.845    1.088    4.453    0.000    1.000    1.000
##    .dem65             0.115    0.200    0.575    0.565    0.026    0.026
##
## Defined Parameters:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##     ab                0.062    0.031    2.012    0.044    0.065    0.065
##     total             0.926    0.114    8.097    0.000    0.978    0.978
```

```
semPaths(fit, "std", edge.label.cex = 1.0, curvePivot = TRUE)
```



The chi-square is non significant now. The CFI is .95, and RMSEA is .101 and statistically significant. Factor structures remain intact and all paths are significant. The total effect of ind60 and dem60 explains 84% (beta weights are equivalent to r (then squared for variance explained) of the variance in dem65, however less than 1% of that variance is attributable to industrialization (p = .044). Depending on the measurement validity of this dataset, industrialization should be ruled out as a strong contributor to any change in political democracy. An additional model might be specified with other explanatory variables such as ?

6.1.1.4 Multiple Groups

If your sample size is large enough you can use `lavaan` to conduct multiple group analysis. To do this add the name of the grouping variable to group argument. The same model will be fitted to each group.

```
HS.model <- ' visual =~ x1 + x3
              textual =~ x4 + x5 + x6
              speed   =~ x7 + x8 + x9
              visual ~ textual'

fit <- cfa(HS.model, data = HolzingerSwineford1939, group = "school")
```

```
## Warning in lav_object_post_check(object): lavaan WARNING: some estimated ov
```

```
## variances are negative
```

```
summary(fit, fit.measures = TRUE, standardized=TRUE)
```

```
## lavaan 0.6-11 ended normally after 53 iterations
##
##   Estimator                                         ML
##   Optimization method                           NLMINB
##   Number of model parameters                        52
##
##   Number of observations per group:
##     Pasteur                                        156
##     Grant-White                                    145
##
## Model Test User Model:
##
##   Test statistic                               107.318
##   Degrees of freedom                                36
##   P-value (Chi-square)                           0.000
##   Test statistic for each group:
##     Pasteur                                     48.124
##     Grant-White                                 59.194
##
## Model Test Baseline Model:
##
##   Test statistic                               893.935
##   Degrees of freedom                                56
##   P-value                                        0.000
##
## User Model versus Baseline Model:
##
##   Comparative Fit Index (CFI)                    0.915
##   Tucker-Lewis Index (TLI)                       0.868
##
## Loglikelihood and Information Criteria:
##
##   Loglikelihood user model (H0)              -3236.131
##   Loglikelihood unrestricted model (H1)      -3182.472
##
##   Akaike (AIC)                                6576.262
##   Bayesian (BIC)                              6769.032
##   Sample-size adjusted Bayesian (BIC)         6604.118
##
## Root Mean Square Error of Approximation:
##
```

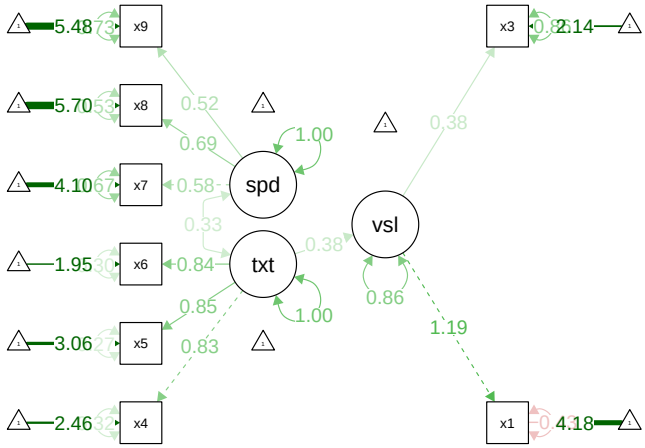
```
##   RMSEA                                          0.115
##   90 Percent confidence interval - lower         0.090
##   90 Percent confidence interval - upper         0.140
##   P-value RMSEA <= 0.05                          0.000
##
## Standardized Root Mean Square Residual:
##
##   SRMR                                           0.079
##
## Parameter Estimates:
##
##   Standard errors                             Standard
##   Information                                 Expected
##   Information saturated (h1) model          Structured
##
##
## Group 1 [Pasteur]:
##
## Latent Variables:
##                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   visual =~
##     x1               1.000                               1.411    1.195
##     x3               0.311    0.166    1.870    0.061    0.439    0.379
##   textual =~
##     x4               1.000                               0.949    0.826
##     x5               1.177    0.101   11.668    0.000    1.117    0.855
##     x6               0.870    0.076   11.455    0.000    0.825    0.836
##   speed =~
##     x7               1.000                               0.622    0.575
##     x8               1.080    0.269    4.014    0.000    0.672    0.689
##     x9               0.821    0.202    4.057    0.000    0.511    0.517
##
## Regressions:
##                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   visual ~
##     textual          0.566    0.101    5.591    0.000    0.380    0.380
##
## Covariances:
##                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   textual ~~
##     speed            0.195    0.073    2.684    0.007    0.330    0.330
##
## Intercepts:
##                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .x1               4.941    0.095   52.249    0.000    4.941    4.183
##    .x3               2.487    0.093   26.778    0.000    2.487    2.144
```

```
##    .x4                2.823    0.092   30.689    0.000    2.823    2.457
##    .x5                3.995    0.105   38.183    0.000    3.995    3.057
##    .x6                1.922    0.079   24.320    0.000    1.922    1.947
##    .x7                4.432    0.087   51.181    0.000    4.432    4.098
##    .x8                5.563    0.078   71.214    0.000    5.563    5.702
##    .x9                5.418    0.079   68.440    0.000    5.418    5.480
##    .visual            0.000                              0.000    0.000
##     textual           0.000                              0.000    0.000
##     speed             0.000                              0.000    0.000
##
## Variances:
##                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .x1              -0.597    0.974   -0.613    0.540   -0.597   -0.428
##    .x3               1.153    0.161    7.162    0.000    1.153    0.857
##    .x4               0.419    0.069    6.096    0.000    0.419    0.317
##    .x5               0.460    0.086    5.367    0.000    0.460    0.269
##    .x6               0.293    0.050    5.862    0.000    0.293    0.301
##    .x7               0.783    0.128    6.113    0.000    0.783    0.669
##    .x8               0.501    0.119    4.199    0.000    0.501    0.526
##    .x9               0.716    0.103    6.922    0.000    0.716    0.733
##    .visual           1.704    0.994    1.715    0.086    0.855    0.855
##     textual          0.901    0.150    5.998    0.000    1.000    1.000
##     speed            0.387    0.135    2.867    0.004    1.000    1.000
##
##
## Group 2 [Grant-White]:
##
## Latent Variables:
##                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   visual =~
##     x1                1.000                              0.777    0.676
##     x3                0.885    0.210    4.217    0.000    0.687    0.663
##   textual =~
##     x4                1.000                              0.965    0.861
##     x5                0.998    0.087   11.436    0.000    0.964    0.832
##     x6                0.967    0.085   11.338    0.000    0.934    0.825
##   speed =~
##     x7                1.000                              0.727    0.705
##     x8                1.155    0.175    6.594    0.000    0.839    0.802
##     x9                0.927    0.145    6.399    0.000    0.673    0.657
##
## Regressions:
##                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   visual ~
##     textual           0.471    0.101    4.677    0.000    0.585    0.585
##
```

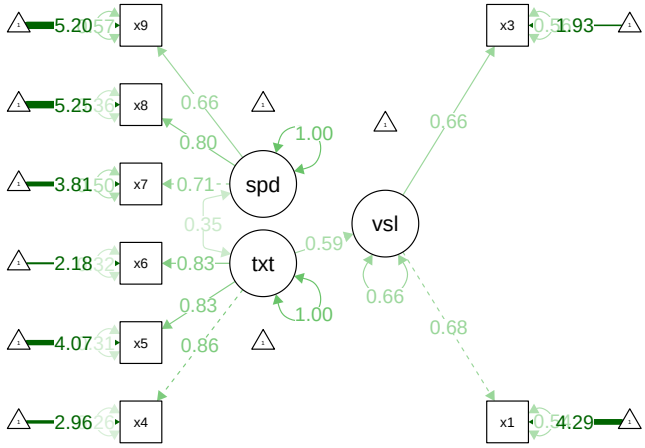
```
## Covariances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   textual ~~
##     speed             0.243    0.078    3.132    0.002    0.347    0.347
##
## Intercepts:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .x1                4.930    0.095   51.696    0.000    4.930    4.293
##    .x3                1.996    0.086   23.195    0.000    1.996    1.926
##    .x4                3.317    0.093   35.625    0.000    3.317    2.958
##    .x5                4.712    0.096   48.986    0.000    4.712    4.068
##    .x6                2.469    0.094   26.277    0.000    2.469    2.182
##    .x7                3.921    0.086   45.819    0.000    3.921    3.805
##    .x8                5.488    0.087   63.174    0.000    5.488    5.246
##    .x9                5.327    0.085   62.571    0.000    5.327    5.196
##    .visual            0.000                              0.000    0.000
##     textual           0.000                              0.000    0.000
##     speed             0.000                              0.000    0.000
##
## Variances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .x1                0.716    0.159    4.514    0.000    0.716    0.543
##    .x3                0.601    0.127    4.741    0.000    0.601    0.560
##    .x4                0.325    0.065    5.036    0.000    0.325    0.259
##    .x5                0.413    0.072    5.770    0.000    0.413    0.308
##    .x6                0.408    0.069    5.924    0.000    0.408    0.319
##    .x7                0.534    0.092    5.811    0.000    0.534    0.503
##    .x8                0.390    0.099    3.932    0.000    0.390    0.357
##    .x9                0.598    0.091    6.537    0.000    0.598    0.569
##    .visual            0.396    0.140    2.839    0.005    0.657    0.657
##     textual           0.932    0.152    6.138    0.000    1.000    1.000
##     speed             0.528    0.127    4.156    0.000    1.000    1.000
```

```
semPaths(fit, "std", edge.label.cex = 1.0, rotation = 2, curvePivot = TRUE)
```

1



2



The x2 variable at the Pasteur school did not load onto the visual achievement factor. We can retool our questionnaire or consider the study validity to fix this, but for now let's just remove it from the model again. Once we do that we can see that the x1 and x3 items are loading very differently onto the visual factor, though both items meet the threshold of .40. What would you do if this was

your study?

6.1.1.5 Measurement Invariance

You can also test for measurement invariance across the grouping variable using the `measurmentInvariance()` function which "performs a number of multiple group analyses in a particular sequence, with increasingly more restrictions on the parameters"(https://lavaan.ugent.be/tutorial/groups.html). Load the `semTools` library to use this function.

There are four types of measurment invariance.

The first is equal form **configural invariance**, which assumes the factor structures (number of items) and patterns (relationships between factors) are identical across groups. The second is **equal loadings** (metric invariance), which assumes that factor loadings (extent which each item loads onto factors) are equal across groups. The third is **equal intercepts** (scalar invariance), which assumes that the intercepts of observed scores are equal across groups. Finally, **equal residual variances** (strict invariance) assumes that the residual variances of observed paths are equal across groups.

```
#install.packages('semTools')
library(semTools)
measurementInvariance(model = HS.model,
                      data = HolzingerSwineford1939,
                      group = "school")
```

```
##
## Measurement invariance models:
##
## Model 1 : fit.configural
## Model 2 : fit.loadings
## Model 3 : fit.intercepts
## Model 4 : fit.means
##
## Chi-Squared Difference Test
##
##                Df    AIC    BIC  Chisq Chisq diff Df diff Pr(>Chisq)
## fit.configural 36 6576.3 6769.0 107.32
## fit.loadings   41 6576.3 6750.5 117.35     10.030       5    0.07439 .
## fit.intercepts 46 6600.7 6756.4 151.78     34.431       5  1.954e-06 ***
## fit.means      49 6638.2 6782.7 195.21     43.431       3  1.993e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
##
## Fit measures:
##
##                   cfi rmsea cfi.delta rmsea.delta
## fit.configural 0.915 0.115        NA          NA
## fit.loadings   0.909 0.111     0.006       0.003
## fit.intercepts 0.874 0.124     0.035       0.012
## fit.means      0.826 0.141     0.048       0.017
```

This model assesses the four measurement invariance types across the grouping variable 'school' in the HS dataset. No model met the acceptable threshold for RMSEA. Judging by CFI alone, which is not always advisable, form, loadings, and intercept models demonstrate good fit. Thus, each school represented in the HS dataset can be assumed to be equal in these respects, but fail to demonstrate strict invariance. These statistics might be used to validate this particular measure as congruent across different groups of students.

6.2 Case Study

In our case study we will be modeling data from Holzer et al.'s (2021) published data on self regulated learning and well being. The abstract and source are pasted below.

"In the wake of COVID-19, university students have experienced fundamental changes of their learning and their lives as a whole. The present research identifies psychological characteristics associated with students' well-being in this situation. We investigated relations of basic psychological need satisfaction (experienced competence, autonomy, and relatedness) with positive emotion and intrinsic learning motivation, considering self-regulated learning as a moderator. Self-reports were collected from 6,071 students in Austria (Study 1) and 1,653 students in Finland (Study 2). Structural equation modeling revealed competence as the strongest predictor for positive emotion. Intrinsic learning motivation was predicted by competence and autonomy in both countries and by relatedness in Finland. Moderation effects of self-regulated learning were inconsistent, but main effects on intrinsic learning motivation were identified. Surprisingly, relatedness only exerted a minor effect on positive emotion. The results inform strategies to promote students' well-being through distance-learning, mitigating negative effects of the situation."

The authors saved the data in an SPSS file (.sav). Let's use the haven library to read and convert the data file into a dataset.

```
#install.packages('haven')
library(haven)
```

```
path = file.path("semcasestudy.sav")
dataset = read_sav(path)
```

Examine the dataset using str(), summary() or DataExplorer::create_report()

```
dataset
```

```
## # A tibble: 7,724 x 39
##    country  gender   age     sr1     sr2     sr3   comp1   comp2   comp3   auto1
##    <dbl+l> <dbl+l> <dbl> <dbl+l> <dbl+l> <dbl+l> <dbl+l> <dbl+l> <dbl+l> <dbl+l>
##  1 0 [Aus~ 1 [fem~    36 5 [str~ 4 [dis~ 2 [agr~ 2 [agr~ 3 [som~ 3 [som~ 3 [som~
##  2 0 [Aus~ 2 [mal~    36 4 [dis~ 4 [dis~ 1 [str~ 4 [dis~ 4 [dis~ 3 [som~ 4 [dis~
##  3 0 [Aus~ 2 [mal~    36 1 [str~ 1 [str~ 1 [str~ 3 [som~ 4 [dis~ 2 [agr~ 5 [str~
##  4 0 [Aus~ 2 [mal~    36 1 [str~ 2 [agr~ 1 [str~ 2 [agr~ 2 [agr~ 2 [agr~ 4 [dis~
##  5 0 [Aus~ 1 [fem~    36 1 [str~ 1 [str~ 3 [som~ 3 [som~ 2 [agr~ 3 [som~ 4 [dis~
##  6 0 [Aus~ 1 [fem~    36 1 [str~ 1 [str~ 1 [str~ 2 [agr~ 2 [agr~ 2 [agr~ 3 [som~
##  7 0 [Aus~ 1 [fem~    36 2 [agr~ 2 [agr~ 2 [agr~ 4 [dis~ 3 [som~ 3 [som~ 4 [dis~
##  8 0 [Aus~ 1 [fem~    36 2 [agr~ 1 [str~ 1 [str~ 2 [agr~ 1 [str~ 2 [agr~ 3 [som~
##  9 0 [Aus~ 3 [oth~    36 5 [str~ 4 [dis~ 2 [agr~ 3 [som~ 1 [str~ 3 [som~ 4 [dis~
## 10 0 [Aus~ 1 [fem~    36 4 [dis~ 3 [som~ 2 [agr~ 3 [som~ 2 [agr~ 3 [som~ 3 [som~
## # ... with 7,714 more rows, and 29 more variables: auto2 <dbl+lbl>,
## #   auto3 <dbl+lbl>, pa1 <dbl+lbl>, pa2 <dbl+lbl>, pa3 <dbl+lbl>,
## #   lm1 <dbl+lbl>, lm2 <dbl+lbl>, lm3 <dbl+lbl>, gp1 <dbl+lbl>, gp2 <dbl+lbl>,
## #   gp3 <dbl+lbl>, sr1.rec <dbl+lbl>, sr2.rec <dbl+lbl>, sr3.rec <dbl+lbl>,
## #   comp1.rec <dbl+lbl>, comp2.rec <dbl+lbl>, comp3.rec <dbl+lbl>,
## #   auto1.rec <dbl+lbl>, auto2.rec <dbl+lbl>, auto3.rec <dbl+lbl>,
## #   pa1.rec <dbl+lbl>, pa2.rec <dbl+lbl>, pa3.rec <dbl+lbl>, ...
```

Subjective Well-Being (SWB) describes "how people experience the quality of their lives, which includes both emotional reactions and cognitive judgements", encompassing moods, emotions, and satisfaction in general and specific areas of life (Martela & Ryan, 2015). Situations outside of our control, such as natural disasters or pandemics, have a significant impact on SWB. For instance, the COVID-19 outbreak has been associated with a 74% dip in SWB in China, a phenomenon also felt across Europe and the United States.

Self regulation style is represented by a motivational spectrum, developed by Deci & Ryan (2000) where students exhibit behavior regulation styles as they perceive motivational influences from either external or internal sources. This spectrum ranges from extrinsic to intrinsic. For instance, financial gain may serve as an extrinsic regulator, as in the salary a person earns motivates them to come to work every day, while the pressure of a duty or a job is an example of introjected regulation, where a person feels they must complete an action due to external pressure, often from other people or the environment. Identified regulation occurs when a person's identity intersects with an expected set of

tasks for success, for instance, I am a musician, so I have to practice. On the far end of the spectrum, intrinsic regulation is self-sustaining, and catalyzed by an identified interest and curiosity for a task. Hundreds of studies across multiple continents suggest that the satisfaction of basic psychological needs can encourage students to move along the spectrum towards intrinsic regulation, where growth, learning, and skill acquisition become self-regulated processes. To practice these skills, follow the directions below.

- Considering the theory behind these data, start drafting some research questions. I tend to find that when constructing initial models, it is imperative to draw my thinking on paper and construct model syntax.
- Use the code box below to specify your own model based on Holzer et al.'s (2021) dataset. You may need to create composite indexes using multiple subscales of observed variables before specifying your model. If you aren't sure where to start, go back to the abstract at the start of the case study for evidence of Holzer et al.'s model specifications. Also see the codebook titled 'HolzerReadme' at https://osf.io/6jb9t/ within this short course's folder. Hint: Most items were collected with strongly agree coded as 1, with strongly disagree coded as 5. All items have been recoded to match magnitude with numerical order. These items are labeled as item.rec.
- Run the model and interpret the results.
- Visualize the model.
- Do you need to specify a new model? Do any paths need to be pruned? Write new formulas below. Then run, visualize and interpret the model.
- Does your model hold across Austrian and Finnish populations? Group your model by country to see the output of each model, then run a test of measurement invariance on your model.
- Make a few notes here regarding potential next steps for research. Consider how the COVID-19 pandemic might impact the measured relationships.
- Specify and run a mediation model including variables in the Holzer et al. (2021) dataset.

6.3 Review

In this chapter we introduced SEM concepts and methods. To make sure you understand this material, there is a practice assessment to go along with this chapter at https://jayholster.shinyapps.io/SEMinRAssessment/

6.4 References

Deci, E. L., & Ryan, R. M. (2000) The "What and"Why" of goal pursits: Human needs and the self-determination of behavior. *Psychological Inquiry, 11*(4), 227–268.

Epskamp, S. (2015). semPlot: Unified visualizations of structural equation models. *Structural Equation Modeling, 22*(3), 474–483. <https://doi.org/10.1080/10705511.2014.937847>

Holzer, J., Lüftenegger, M., Korlat, S., Pelikan, E., Salmela-Aro, K., Spiel, C., & Schober, B. (2021). Higher dducation in times of COVID-19: University students' basic need satisfaction, self-regulated learning and well-being. Ann Arbor, MI: Inter-university Consortium for Political and Social Research [distributor]. https://doi.org/10.3886/E133601V1

Martela, F, & Ryan, R. (2015). The benefits of benevolence: Basic psychological needs, beneficence, and the enhancement of well-being. *Journal of Personality, 84*(6), 750–764. https://doi.org/10.1111/jopy.12215

Rosseel, Y. (2012). lavaan: An R Package for Structural Equation Modeling. *Journal of Statistical Software*, *48*(2), 1–36. [https://doi.org/[10.18637/jss.v048.i02](https://doi.org/[10.1

6.4.1 R Short Course Series

Video lectures of each guidebook chapter can be found at https://osf.io/6jb9t/. For this chapter, find the follow the folder path Structural Equation Modeling in R -> AY 2021-2022 Spring and access the video files, r markdown documents, and other materials for each short course.

6.4.2 Acknowledgements

This guidebook was created with support from the Center for Research Data and Digital Scholarship and the Laboratory for Interdisciplinary Statistical Analaysis at the University of Colorado Boulder, as well as the U.S. Agency for International Development under cooperative agreement #7200AA18CA00022. Individuals who contributed to materials related to this project include Jacob Holster, Eric Vance, Michael Ramsey, Nicholas Varberg, and Nickoal Eichmann-Kalwara.

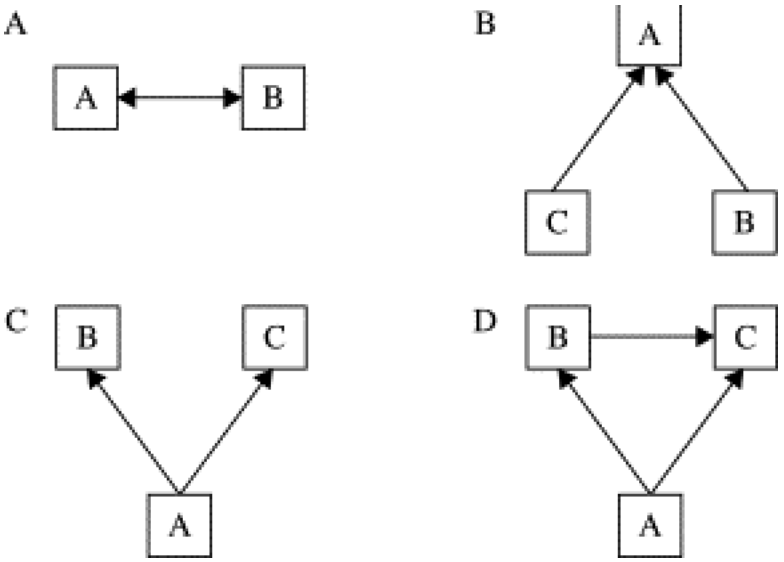
Chapter 7

Network Analysis

In this chapter, we will cover concepts and procedures related to network analysis in R. "Networks enable the visualization of complex, multidimensional data as well as provide diverse statistical indices for interpreting the resultant graphs" (Jones et al., 2018). Put otherwise, network analysis is a collection of techniques that visualize and estimate relationships among agents in a social context. Furthermore, network analysis is used "to analyze the social structures that emerge from the recurrence of these relations" where "[the] basic assumption is that better explanations of social phenomena are yielded by analysis of the relations among entities" (Science Direct; Linked Below).

Networks are made up of **nodes** (i.e., individual actors, people, or things within the network) and the **ties**, **edges**, or **links** (i.e., relationships or interactions) that connect them. The extent to which nodes are connected lends to interpretations of the measured social context.

"By comparison with most other branches of quantitative social science, network analysts have given limited attention to statistical issues. Most techniques and measures examine the structure of specific data sets without addressing sampling variation, measurement error, or other uncertainties. Such issues are complex because of the dependencies inherent in network data, but they are now receiving increased study. The most widely investigated approach to the statistical analysis of networks stresses the detection of **formal regularities** in local relational structure.



The figure above illustrates some of the relational structures commonly found in analyses of social networks.

A: Demonstrates a relationship of reciprocity/mutuality.

B: Demonstrates a directed relationship with a common target.

C: Relationships emerge from a common source.

D: Transitive direct relationships with indirect influences.

Another type is **homophily,** which is present, for example, when same-sex friendships are more common than between-sex friendships. This involves an interaction between a property of units and the presence of relationships" (Peter V. Marsden, in Encyclopedia of Social Measurement, 2005). This sort of model might reflect the tendency of people to seek out those that are similar to themselves.

7.0.0.1 Measures of Centrality

Measures of centrality provide quantitative context regarding the importance of a node within a network. There are four measures of centrality that we will cover.

Degree Centrality: The degree of a node is the number of other nodes that single node is connected to. Important nodes tend to have more connections to other nodes. Highly connected nodes are interpreted to have high degree centrality.

Eigenvector Centrality: The extent to which adjacent nodes are connected

themselves also indicate importance (e.g., Important nodes increase the importance of other nodes).

Closeness centrality: Closeness centrality measures how many steps are required to access every other node from a given node. In other words, important nodes have easy access to other nodes given multiple connections.

Betweenness Centrality: This ranks the nodes based on the flow of connections through the network. Importance is demonstrated through high frequency of connection with multiple other nodes. Nodes with high levels of betweenness tend to serve as a bridge for multiple sets of other important nodes.

See this link for a set of journals and books that cover the topic.

Also, examine this (paid) online tool for text-based network analysis: https://www.infranodus.com

7.1 Zacharies Karate Club Case Study

We will be working with a dataset called Zacharies Karate Club, a seminal dataset in network analysis literature. First we need to install the relevant packages. Today we will need a package called `igraph`, a package useful for creating, analyzing, and visualizing networks. If you do not have the packages already, install the `tidyverse`, `igraph`, `ggnetwork`, and `intergraph`. `igraph` helps us perform network analysis. `ggnetwork` and `intergraph` are both packages used for plotting networks in the ggplot framework.

```
# Load the libraries
library(tidyverse)
library(igraph)
library(ggnetwork)
library(intergraph)
```

Zachary's Karate Club Background

Taken from wikipedia: "A social network of a karate club was studied by Wayne W. Zachary for a period of three years from 1970 to 1972. The network captures 34 members of a karate club, documenting pairwise links between members who interacted outside the club. During the study a conflict arose between the administrator"John A" and instructor "Mr. Hi" (pseudonyms), which led to the split of the club into two. Half of the members formed a new club around Mr. Hi; members from the other group found a new instructor or gave up karate. Based on network analysis Zachary correctly predicted each member's decision except member #9, who went with Mr. Hi instead of John A." In this case study, we will try to infer/predict the group splits with network analysis techniques.

7.1.0.1 Load Data and Extract Model Features

Now it's time to extract the relevant information that we need from the dataset. We need the associations between members (edges), the groupings after the split of the network, and the labels of the nodes.

```
# Load and view the data
members <- read_csv("Zacharies_Karate_Club.csv")
edges <- read_csv("Zacharies_Karate_Club_edges.csv")

# Extract information for nodes
nodes <- members$node

# Extract information on edges
edges <- as.vector(rbind(edges$From, edges$To))
```

Extract the groups and labels of the vertices and store them in vectors. Make sure that the labels are called as characters and not factors using the "str()" function, as igraph requires character data to cast labels.

7.1.0.2 Creating Networks From Data

Now that we have extracted the relevant data that we need, let's construct a network of Zachary's Karate club.

```
# Create our network
# Note that this will automatically enumerate the nodes (eg. 1, 2, 3, ...)
G <- make_empty_graph(n = length(nodes), directed = F) %>%
  add_edges(edges)
```

We can also create vertex attributes. Let's make a vertex attribute for each group (Mr. Hi and John A).

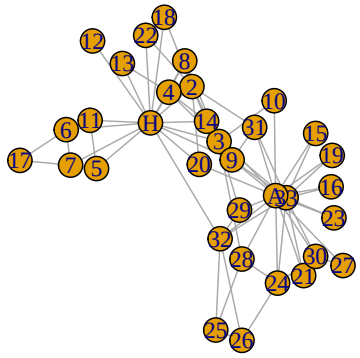
```
# Create a vertex attribute
  G <- G %>%
  igraph::set.vertex.attribute('group', index = V(G), value = groups)
```

Create a vertex attribute for node label. Call the attribute 'label'.

7.1.0.3 Visualizing Networks with baseR

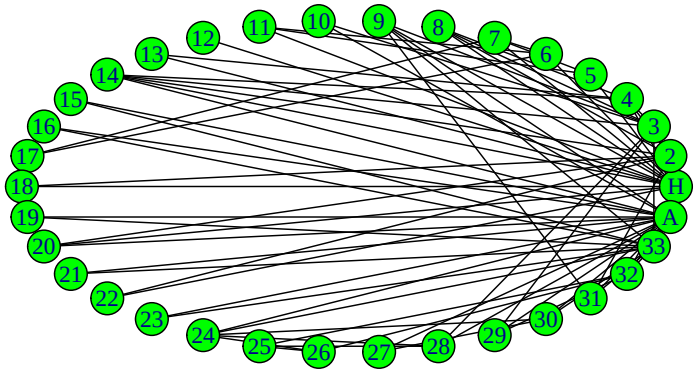
Now visualize the network by running the plot function on our network 'G'.

```
# Plot igraph object
plot(G)
```



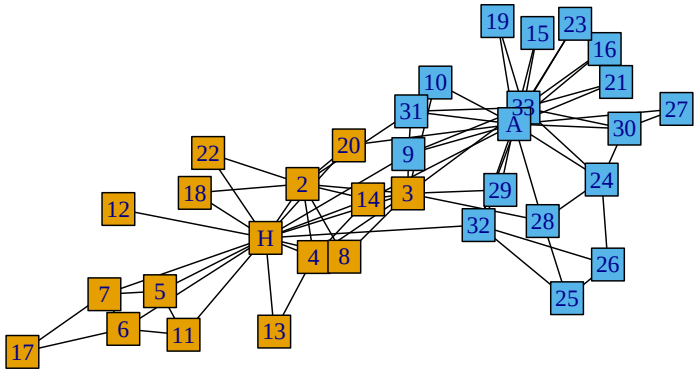
Let's change some of the plot aesthetics. We can change the vertex colors, edge colors, vertex sizes, etc. Play around with the arguments for plotting a network.

```
# Edit baseR plot aesthetics
plot(G, vertex.color="green",  # Changes node color
     edge.color = 'black',     # Changes edge color
     vertex.size = 10,         # Changes vertex size
     vertex.shape = 'circle',  # Changes vertex shape
     asp = 0,                  # Spread out nodes
     layout = layout_in_circle)# Format nodes in a circle
```



We can also change the color of our vertices according to group.

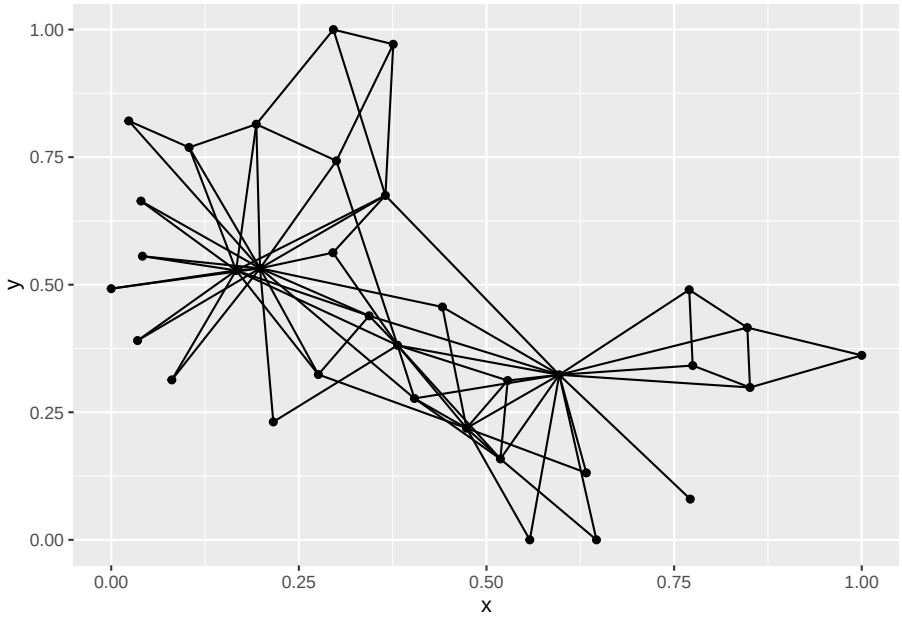
```
plot(G, vertex.color = groups, # Changes node color
     edge.color = 'black',     # Changes edge color
     vertex.size = 10,         # Changes vertex size
     vertex.shape = 'square',  # Changes vertex shape
     asp = 0)                  # Spread out node)
```



7.1.0.4 Visualizing Networks with ggnetwork

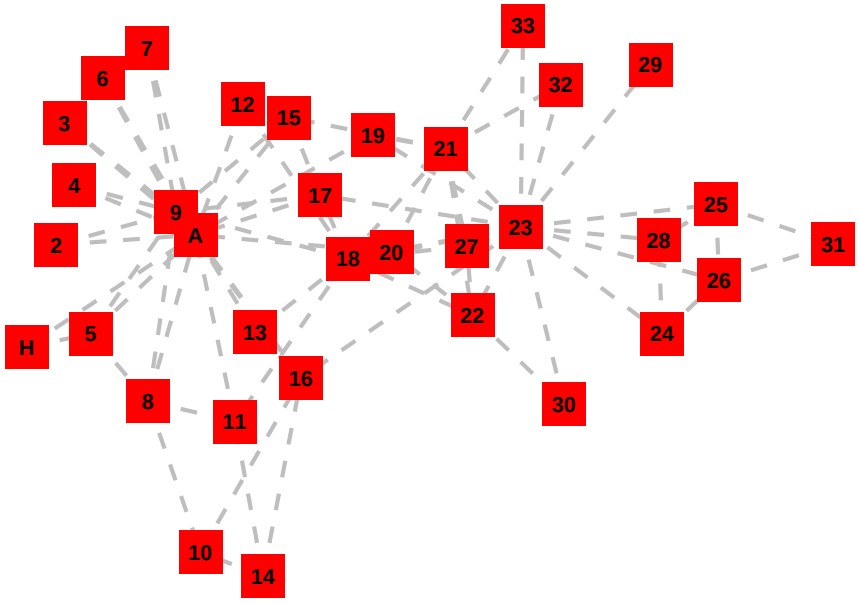
You can also use ggplot to visualize igraph objects.

```
# Plot igraph object with ggplot
ggplot(G, aes(x = x, y = y, xend = xend, yend = yend)) +
  geom_edges() +
  geom_nodes()
```



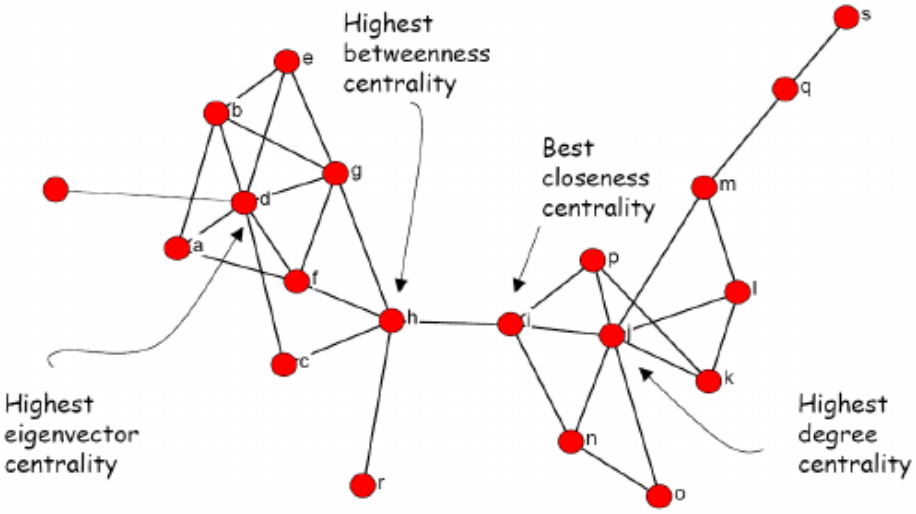
Let's see if we can make our the ggplot version look better.

```
# Plot igraph object with ggplot
ggplot(G, aes(x = x, y = y, xend = xend, yend = yend)) +
  geom_edges(color = 'grey', size = 1, linetype = 'dashed') + # Alter edge attributes
  geom_nodes(size = 10, color = 'red', shape = 'square') +    # Alter node attributes
  geom_nodetext(label = people, fontface = "bold") +          # Add text to nodes
  theme_blank()                                               # Remove grid
```



Using ggnetwork and ggplot, color or shape the nodes by karate group. Also make some other plot aesthetic changes to your liking.

7.1.0.5 Measuring Centrality



```
# Compute the degree centrality for our graph G.
degr_cent <- centr_degree(G, mode = 'all')
degr_cent <- degr_cent$res

# Compute the eigenvector centrality of our network
eign_cent <- eigen_centrality(G)
eign_cent <- eign_cent$vector

# Compute the closeness centraility
clos_cent <- igraph::closeness(G)

# Compute betweeness centrality
betw_cent <- igraph::betweenness(G)
```

Finally, Let's put all of the centrality measures in one table so that we can compare the outputs.

```
# Create data frame storing all of the measures of centrality
data <- data.frame(vertex = nodes,
                   label = people,
                   degree = degr_cent,
                   eigen = eign_cent,
                   closeness = clos_cent,
                   betweeness = betw_cent)

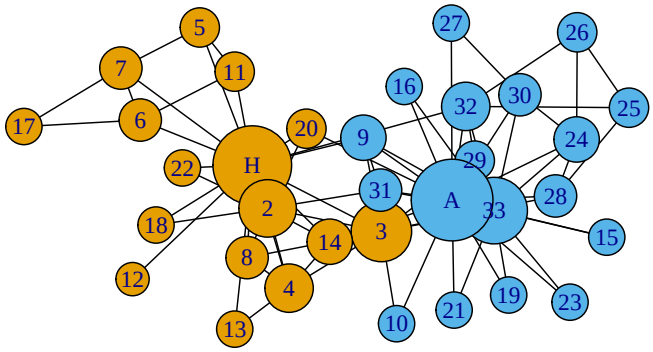
# Order the data by degree centrality
data <- data %>% arrange(desc(degree))

# View the head of the data frame
head(data)
```

```
##   vertex label degree     eigen  closeness betweeness
## 1     34     A     17 1.0000000 0.01666667 160.551587
## 2      1     H     16 0.9521324 0.01724138 231.071429
## 3     33    33     12 0.8266589 0.01562500  76.690476
## 4      3     3     10 0.8495542 0.01694915  75.850794
## 5      2     2      9 0.7123351 0.01470588  28.478571
## 6      4     4      6 0.5656143 0.01408451   6.288095
```

It makes sense that the most connected members of the network are indeed John A. and Mr. Hi. We can view the centrality measures from the perspective of the graph. Here, we add the object `degr_cent` to the vertex size to display the nodes via their degree centrality using `baseR`.

```
# Plot ZKC with igraph
plot(G,                                 # Plot igraph object
     vertex.color = groups,             # Change vertex colors
     edge.color = 'black',              # Change edge color
     vertex.size = 10+degr_cent,        # Change node size
     vertex.shape = 'circle',           # Specify node shape
     asp = 0,                           # Spreads out nodes
     layout=layout_with_lgl)            # Specify layout
```



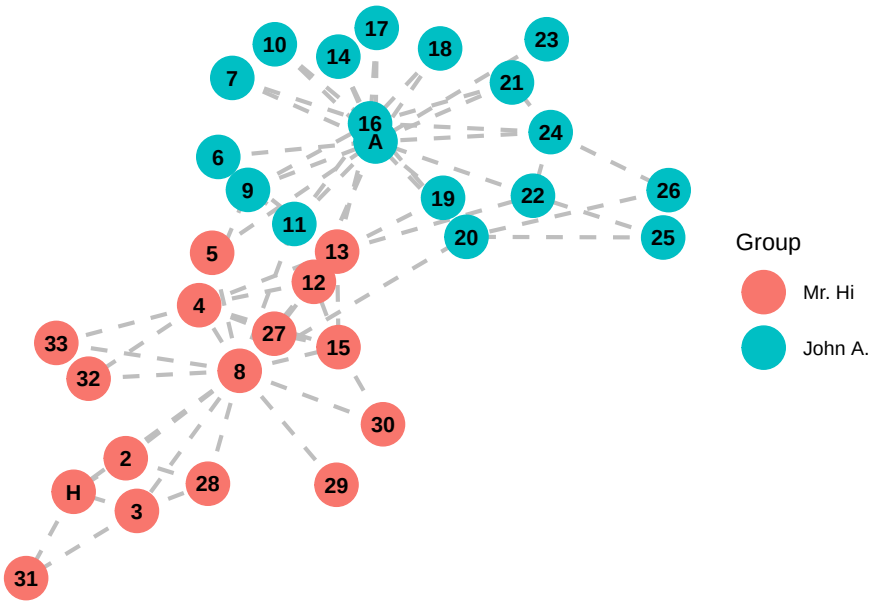

```
tkplot(G,
       vertex.color = groups,
       edge.color = 'black',
       vertex.label.color = 'white',
       vertex.size = 10+degr_cent)
```

```
## [1] 1
```

Now, using the `tidyverse`! Change the code below to make a graph of our network where node sizes are scaled by the degree centrality.

```
ggplot(G, aes(x = x, y = y, xend = xend, yend = yend)) +      # Do not change
  geom_edges(color = 'grey', size = 1, linetype = 'dashed') + # Alter edge attributes
  geom_nodes(aes(color = as.factor(group)),
```

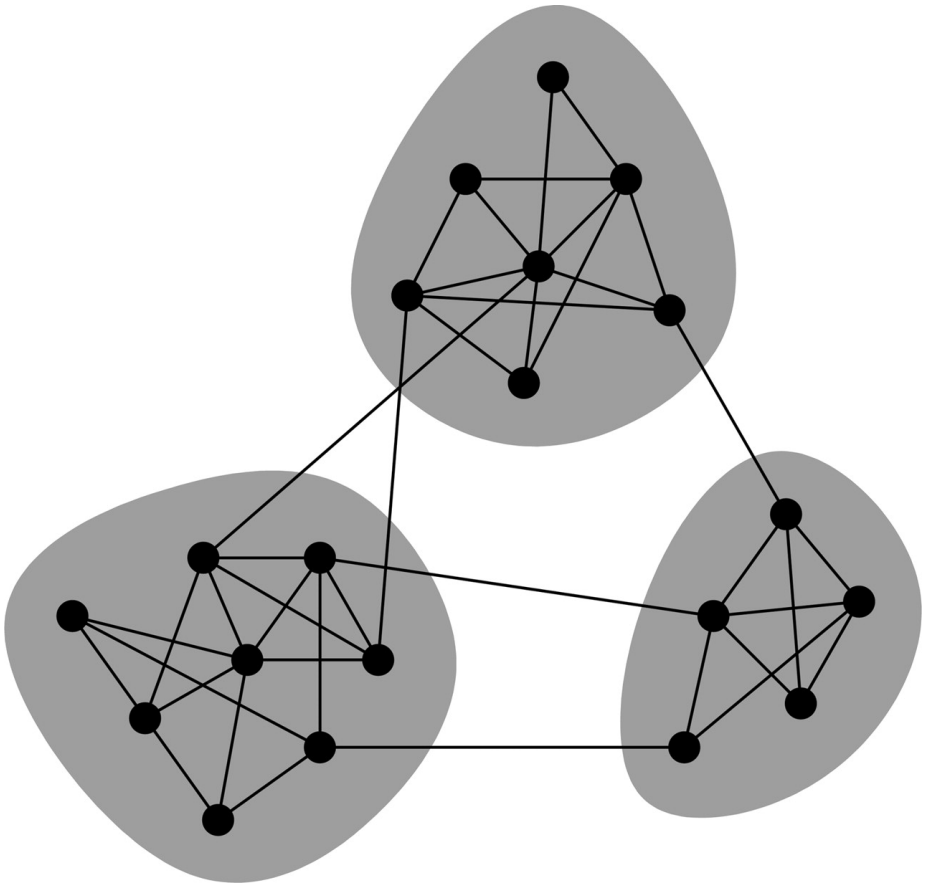
```
            size = 10,
            shape = 'circle') +                                  # Alter node attributes
  geom_nodetext(label = people, fontface = "bold") +             # Add text to nodes
  theme_blank() +                                                # Remove grid
  scale_color_discrete(name = 'Group',                           # Edit legend
                       label= c('Mr. Hi','John A.'))
```




```
#Answer: add degr_cent to size under the "geom_nodes" layer.
```

7.1.0.6 Modularity

Modularity is a measure that describes the extent to which community structure is present within a network when the groups are labeled. A modularity score close to 1 indicates the presence of strong community structure in the network. In other words, nodes in the same group are more likely to be connected than nodes in different groups. A modularity score close to -1 indicates the opposite of community structure. In other words, nodes in different groups are more likely to be connected than nodes in the same group. A modularity score close to 0 indicates that no community structure (or anti-community structure) is present in the network.



Compute the modularity of the Zacharies Karate Club network using the modularity() function.

```
## [1] 0.3714661
```

Higher modularity scores are better, however, modularity should not be used alone to assess the presence of communities in network. Rather, multiple measures should be used to provide an argument for community in a network.

7.2 Community Detection

Suppose we no longer have the group labels, but we want to infer the existence of groups in our network. This process is known as community detection. There are many different ways to infer the existence of groups in a network.

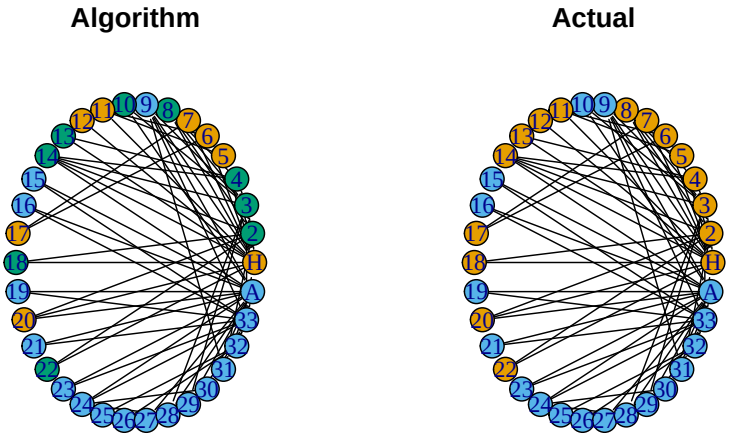
7.2.0.1 Via Modularity Maximization

The goal here is to find the groupings of nodes that lead to the highest possible modularity score.

```
# Find communites using the modularity maximization algorithm
mod_groups <- cluster_fast_greedy(G)
mod_groups <- mod_groups$membership

# Plot the computed modularity groupings
par(mfrow=c(1,2))
plot(G, vertex.color = mod_groups, # Changes node color
     edge.color = 'black',     # Changes edge color
     vertex.size = 20,         # Changes vertex size
     vertex.shape = 'circle',  # Changes vertex shape
     asp = 0,
     layout = layout_in_circle,
     main = 'Algorithm')

# Plot the actual modularity groupings
plot(G, vertex.color = groups, # Changes node color
     edge.color = 'black',     # Changes edge color
     vertex.size = 20,         # Changes vertex size
     vertex.shape = 'circle',  # Changes vertex shape
     asp = 0,
     layout = layout_in_circle,
     main = 'Actual')
```



It turns out that the modularity maximization algorithm finds 3 communities within the Zacharies Karate Club network. But, if we merge those two groups into two, only one node is incorrectly grouped. Let's try another community detection algorithm.

7.2.0.2 Via Edge Betweenness

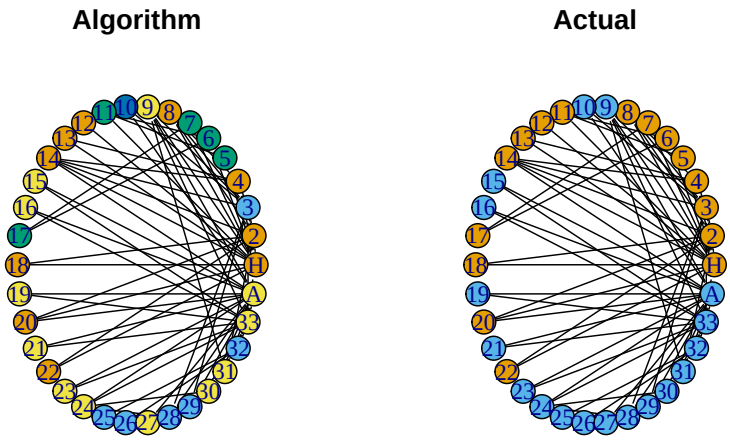
Edge betweenness community structure detection is based on the following assumption; that edges connecting separate groupings have high edge betweenness as all the shortest paths from one module to another must traverse through them. Practically this means that if we gradually remove the edge with the highest edge betweenness score, our network will separate into communities.

```
# Find communites using the edge betweeness algorithm
btw_groups <- cluster_edge_betweenness(G)
btw_groups <- btw_groups$membership

# Plot the computed betweeness groupings
par(mfrow=c(1,2))
plot(G, vertex.color = btw_groups, # Changes node color
     edge.color = 'black',     # Changes edge color
     vertex.size = 20,         # Changes vertex size
     vertex.shape = 'circle',  # Changes vertex shape
     asp = 0,
```

```
     layout = layout_in_circle,
     main = 'Algorithm')

# Plot the actual modularity groupings
plot(G, vertex.color = groups, # Changes node color
     edge.color = 'black',     # Changes edge color
     vertex.size = 20,         # Changes vertex size
     vertex.shape = 'circle',  # Changes vertex shape
     asp = 0,
     layout = layout_in_circle,
     main = 'Actual')
```



7.3 Network Simulation

Say you want to model a new network with no data. it's possible to simulate a network to find out if it is actually interesting, or random. If you are familiar with hypothesis testing, we can view these random networks as our "null models". We assume that our null model is true until there is enough evidence to suggest that our null model does not describe the real-life network. If our null-model is a good fit, then we have achieved a good representation of our network. If we don't have a good fit, then there is likely additional structure in the network that is unaccounted for.

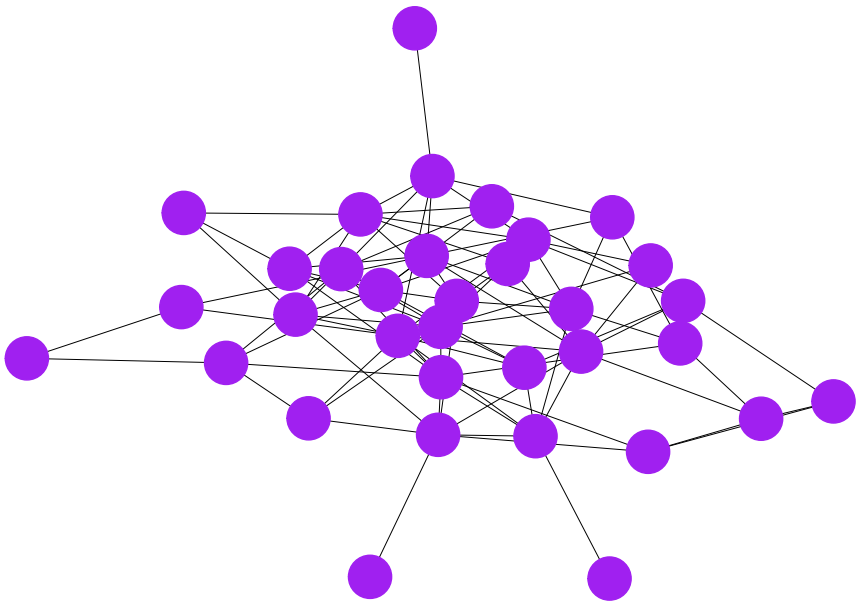
Our Question: How can we explain the group structure of our network? Is it random or can we explain it via the degree sequence?

7.3.0.1 Random Network Generation

Erdos-Renyi random networks in R require that we specify a number of nodes n, and an edge construction probability p. Essentially, for every pair of nodes, we flip a biased coin with the probability of "heads" being p. If we get a "heads", then we draw an edge between that pair of nodes. This process simulates the social connections rather than plotting them from a dataset.

```
# Simulate an erdos-renyi random network and display the result
ER <- sample_gnp(n = length(nodes), p = .15, directed = FALSE, loops = FALSE)

# Plot the erdos-renyi random network
ggplot(ER, aes(x = x, y = y, xend = xend, yend = yend)) +
  geom_edges(color = 'black', size = 0) +
  geom_nodes(color = 'purple',
             size = 10,
             shape = 'circle') +
  theme_blank()
```



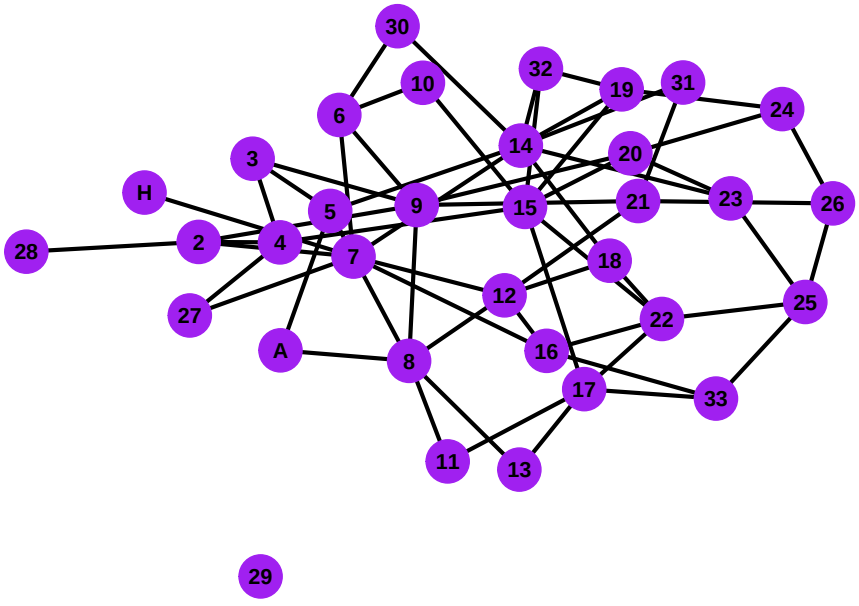
Is this Erdos-Renyi random network a good representative model of the Zacharies Karate Club Network? Let's construct the Erdos-Renyi random network that is most similar to our network.

We can map in parameters in the Erdo-Renyi random graph by specifying the number of nodes and the edge connection probability p. Considering the Zacharies Karate Club Network, we want to use 34 nodes in our graph. If we change the number of nodes, then we lose the ability to compare our network with the theoretical model. We can estimate a probability value for the simulated network using the mean of degr_cent over the length of the nodes - 1 from the ZKC network.

```
# Estimate parameter p for ZCC
pval <- mean(degr_cent)/(length(nodes)-1)

# Simulate an erdos-renyi random network and display the result
ER <- sample_gnp(n = length(nodes),
                 p = pval,
                 directed = FALSE,
                 loops = FALSE)

# Plot the erdos-renyi random network
ggplot(ER, aes(x = x, y = y, xend = xend, yend = yend)) +
  geom_edges(color = 'black', size = 1) +
  geom_nodes(color = 'purple',
             size = 10,
             shape = 'circle') +
  geom_nodetext(label = people, fontface = "bold") +
  theme_blank()
```



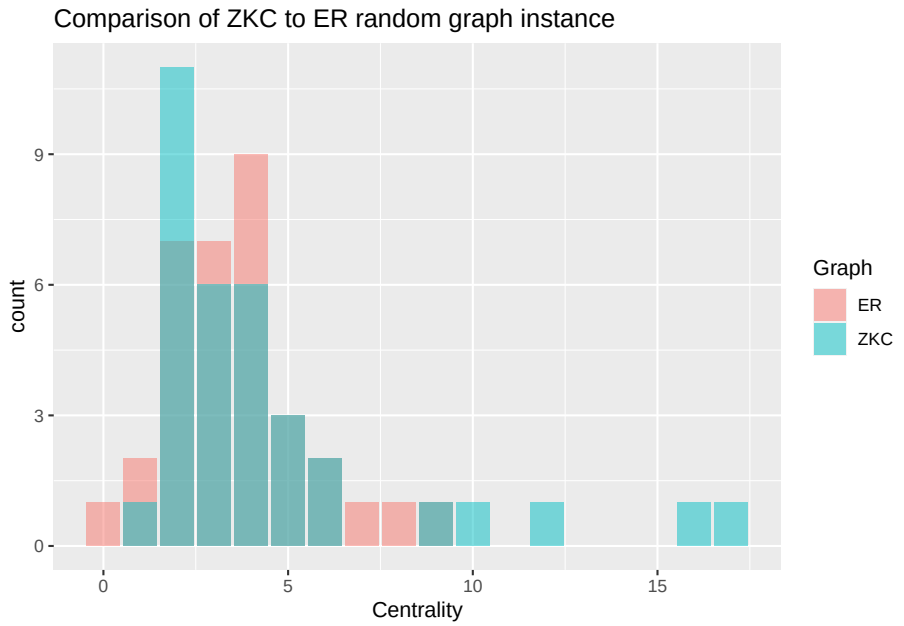
Let's check out the degree distribution for our random graph and the actual ZCC graph.

```
# Compute degree centrality of ER Model
degr_ER <- centr_degree(ER, mode = 'all')$res

# Construct data frame for centrality
centr_compar <- data.frame(Nodes = nodes,
                           ZKC = degr_cent,
                           ER = degr_ER)

# Reformat data frame to tidy
centr_compar <- centr_compar %>%
  gather(key = 'Graph', value = 'Centrality', ZKC, ER)

# Create a bar plot of degree distributions
ggplot(data = centr_compar, aes(x = Centrality, fill = Graph)) +
  geom_bar(alpha = .5, position = 'identity') +
  ggtitle('Comparison of ZKC to ER random graph instance')
```



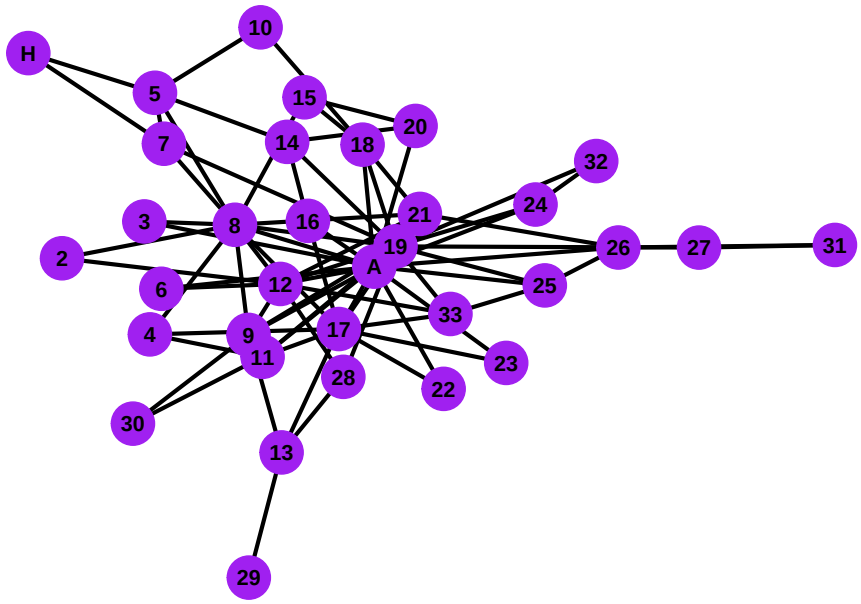
7.3.0.2 Configuration Model

For this kind of random-graph model, we specify the exact degree sequence of all the nodes. We then construct a random graph that has the exact degree

sequence as the one given.

```
# Simulate a configuration model
# Note: The method simple.no.multiple prevents self loops and multiple edges
config <- sample_degseq(degr_cent, method = "simple.no.multiple")

# Plot the configuration model
ggplot(config, aes(x = x, y = y, xend = xend, yend = yend)) +
  geom_edges(color = 'black', size = 1) +
  geom_nodes(color = 'purple',
             size = 10,
             shape = 'circle') +
  geom_nodetext(label = people, fontface = "bold") +
  theme_blank()
```



Is the configuration model random network a good representative model of the Zachary's Karate Club Network?

Let's see if the configuration model captures the group structure of the model. We are going to perform a permutation test in which we generate 1000 different configuration models (with the same degree sequence as ZKC), and then estimate how the actual value of the ZKC modularity lines up with the distribution of configuration model modularities.

```
# Initialize vector to store values
sims <- 1000
mod_vals <- rep(0,sims)

# Loop through simulations
for (i in c(1:sims))
{
  # Simulate a configuration model
  config <- sample_degseq(degr_cent, method = "simple.no.multiple")

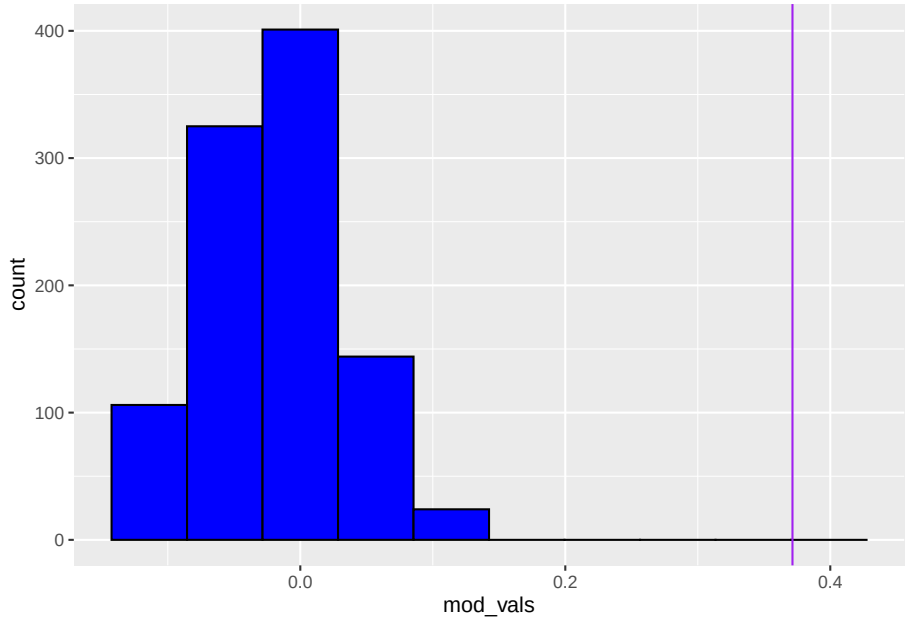
  # Compute the modularity of the network w/ respect to ZKC groups
  mod_score <- modularity(config, groups)

  # Store the modularity value in our vector
  mod_vals[i] <- mod_score

}
```

Now let's plot a histogram of these values, with a vertical line representing the modularity of ZKC network that we computed earlier. This value is stored in the object `ZCCmod`.

```
# Plot a histogram modularity values
ggplot(data = as.data.frame(mod_vals), aes(x = mod_vals)) +
  geom_histogram(bins = 10, color = 'black', fill = 'blue') +
  geom_vline(xintercept = ZCCmod, color = 'purple')
```



We can see from the above that our computed modularity is extremely improbable. No simulations had a modularity that was as high as the one in ZKC. This tells us that the particular degree sequence of ZKC does not capture the community structure. Put otherwise, the configuration model does a bad job reflecting the community structure captured in the ZKC dataset.

7.3.0.3 Stochastic Block Model

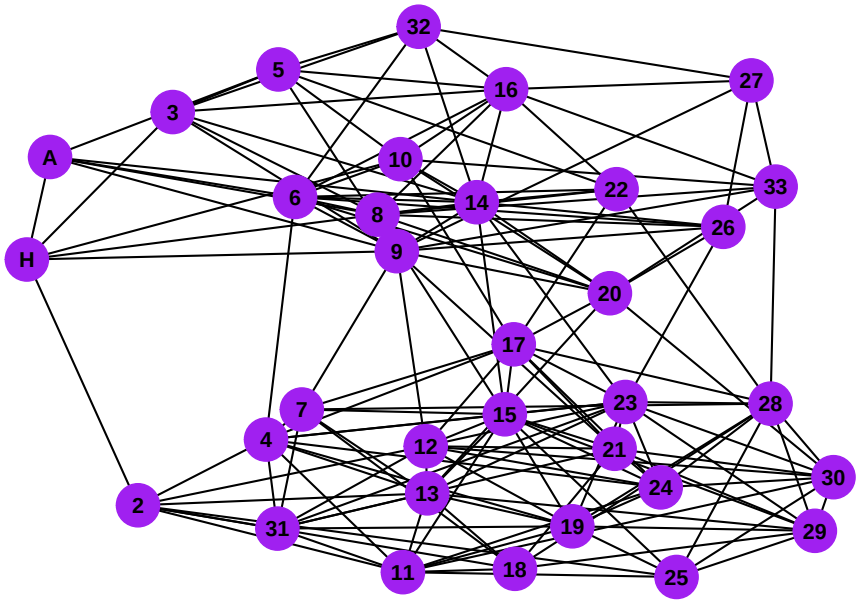
Stochastic Block models are similar to the Erdos-Renyi random network but provide the additional ability to specify additional parameters. The stochastic block model adds a group structure into the random graph model. We can specify the group sizes and the edge construction probability for within group and between group modeling

```
# Construct the edge probability matrix and block sizes
pref.matrix = matrix(c(.5, .05, .05, .5), nrow = 2, ncol = 2)
block.sizes = c(18,34-18)

# Simulate a stochastic block model
mySBM <- sample_sbm(n = length(nodes), pref.matrix, block.sizes, directed = FALSE, loo

# Plot the SBM
ggplot(mySBM, aes(x = x, y = y, xend = xend, yend = yend)) +
  geom_edges(color = 'black', size = .5) +
```

```
  geom_nodes(color = 'purple',
             size = 10,
             shape = 'circle') +
  geom_nodetext(label = people, fontface = "bold") +
  theme_blank()
```



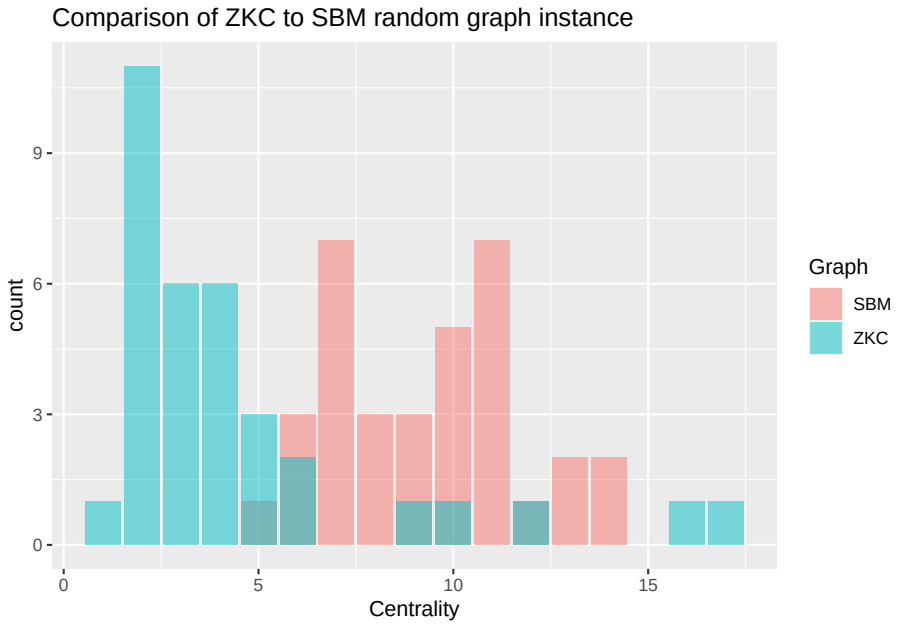
Is the stochastic block model a good representative model of the Zacharies Karate Club Network?

```
# Compute degree centrality of Stochastic Block Model
degr_SBM <- centr_degree(mySBM, mode = 'all')$res

# Construct data frame for centrality
centr_compar <- data.frame(Nodes = nodes,
                           ZKC = degr_cent,
                           SBM = degr_SBM)

# Reformat data frame to tidy
centr_compar <- centr_compar %>%
  gather(key = 'Graph', value = 'Centrality', ZKC, SBM)

# Create a bar plot of degree distributions
ggplot(data = centr_compar, aes(x = Centrality, fill = Graph)) +
  geom_bar(alpha = .5, position = 'identity') +
  ggtitle('Comparison of ZKC to SBM random graph instance')
```



7.4 Advanced Case Study

See this link (https://www.frontiersin.org/articles/10.3389/fpsyg.2018.01742/) to access a paper by Jones, Mair, & McNally (2018), all professors at Harvard University in the Department of Psychology who discuss visualizing psychological networks in R.

See this link (https://www.frontiersin.org/articles/10.3389/fpsyg.2018.01742/full#supplementary-material) to access all supplementary material, including the relevant datasets needed for the code below.

Read the paper and run the code alongside the narrative to get the most out of this case study. For a brief overview of the paper see this abstract:

"Networks have emerged as a popular method for studying mental disorders. Psychopathology networks consist of aspects (e.g., symptoms) of mental disorders (nodes) and the connections between those aspects (edges). Unfortunately, the visual presentation of networks can occasionally be misleading. For instance, researchers may be tempted to conclude that nodes that appear close together are highly related, and that nodes that are far apart are less related. Yet this is not always the case. In networks plotted with force-directed algorithms, the most popular approach, the spatial arrangement of nodes is not

easily interpretable. However, other plotting approaches can render node positioning interpretable. We provide a brief tutorial on several methods including multidimensional scaling, principal components plotting, and eigenmodel networks. We compare the strengths and weaknesses of each method, noting how to properly interpret each type of plotting approach."

```
## Package installations are included here for convenience
#install.packages("MPsychoR")
#install.packages("qgraph")
#install.packages("smacof")
#install.packages("wordcloud")
#install.packages("psych")
#install.packages("eigenmodel")
#install.packages("networktools")

## Note: The following R code is identical to code found in the manuscript

library("MPsychoR")
data(Rogers)
dim(Rogers)

data(Rogers_Adolescent)
dim(Rogers_Adolescent)

colnames(Rogers) <- colnames(Rogers_Adolescent) <- 1:26

library("qgraph")
adult_zeroorder <- cor(Rogers)
qgraph(adult_zeroorder, layout="spring",
       groups = list(Depression = 1:16, "OCD" = 17:26),
       color = c("lightblue", "lightsalmon"))

adult_zeroorder <- cor(Rogers)

library("smacof")
dissimilarity_adult <- sim2diss(adult_zeroorder)

adult_MDS <- mds(dissimilarity_adult)
head(round(adult_MDS$conf, 2)) # top of configuration matrix

adult_MDS_ordinal <- mds(dissimilarity_adult, type="ordinal")
plot(adult_MDS_ordinal, plot.type = "Shepard", main="Ordinal")
text(1.1,0.3, paste("Stress =", round(adult_MDS_ordinal$stress,2)))

adult_MDS_ratio <- mds(dissimilarity_adult, type="ratio")
plot(adult_MDS_ratio, plot.type = "Shepard", main="Ratio")
```

```
text(1.1,0.3, paste("Stress =", round(adult_MDS_ratio$stress,2)))

adult_MDS_interval <- mds(dissimilarity_adult, type="interval")
plot(adult_MDS_interval, plot.type = "Shepard", main="Interval")
text(1.1,0.3, paste("Stress =", round(adult_MDS_interval$stress,2)))

adult_MDS_mspline <- mds(dissimilarity_adult, type="mspline")
plot(adult_MDS_mspline, plot.type = "Shepard", main="Spline")
text(1.1,0.3, paste("Stress =", round(adult_MDS_mspline$stress,2)))

adult_MDS_mspline$stress

qgraph(adult_zeroorder, layout=adult_MDS_mspline$conf,
       groups = list(Depression = 1:16, "OCD" = 17:26),
       color = c("lightblue", "lightsalmon"), vsize=4)
text(-1,-1, paste("Stress=", round(adult_MDS_mspline$stress,2)))

library("wordcloud")
qgraph(adult_zeroorder, layout=adult_MDS_mspline$conf,
       groups = list(Depression = 1:16, "OCD" = 17:26),
       color = c("lightblue", "lightsalmon"),
       vsize=0, rescale=FALSE, labels=FALSE)
points(adult_MDS_mspline$conf, pch=16)
textplot(adult_MDS_mspline$conf[,1]+.03,
         adult_MDS_mspline$conf[,2]+.03,
         colnames(adult_zeroorder),
         new=F)

adult_glasso <- EBICglasso(cor(Rogers), n=408)
qgraph(adult_glasso, layout=adult_MDS_mspline$conf,
       groups = list(Depression = 1:16, "OCD" = 17:26),
       color = c("lightblue", "lightsalmon"), vsize=4)
text(-1,-1, paste("Stress=", round(adult_MDS_mspline$stress,2)))

adolescent_zeroorder <- cor(Rogers_Adolescent)
dissimilarity_adolescent <- sim2diss(adolescent_zeroorder)
adolescent_MDS <- mds(dissimilarity_adolescent, type="mspline")

fit_procrustes <- Procrustes(adult_MDS_mspline$conf, adolescent_MDS$conf)

adolescent_glasso <- EBICglasso(cor(Rogers_Adolescent), n=87, gamma=0)

qgraph(adult_glasso, layout=fit_procrustes$X, groups = list(Depression = 1:16, "OCD" =
       color = c("lightblue", "lightsalmon"), title= "Adults, n=408", vsize=4)
text(-1,-1, paste("Stress=", round(adult_MDS_mspline$stress,2)))
```

```
qgraph(adolescent_glasso, layout=fit_procrustes$Yhat,
       groups = list(Depression = 1:16, "OCD" = 17:26),
       color = c("lightblue", "lightsalmon"), title="Adolescents, n=87", vsize=4)
text(-1,-1, paste("Stress=", round(adolescent_MDS$stress,2)))

round(fit_procrustes$congcoef, 3)

library("psych")
PCA_adult <- principal(cor(Rogers), nfactors = 2)
qgraph(adult_glasso, layout=PCA_adult$loadings, groups = list(Depression = 1:16, "OCD" = 17:26),
       color = c("lightblue", "lightsalmon"), title= "Adults, n=408", layoutOffset=c(.3,.1), vsiz

text(1.5,-.8, paste("% var=", round(sum(PCA_adult$values[1:2]/length(PCA_adult$values)),2)))
title(xlab="Component 1", ylab= "Component 2")

library("eigenmodel")

diag(adult_glasso) <- NA   ## the function needs NA diagonals
p <- 2                      ## 2-dimensional solution
fitEM <- eigenmodel_mcmc(Y = adult_glasso, R = p, S = 1000, burn = 200, seed = 123)
EVD <- eigen(fitEM$ULU_postmean)
evecs <- EVD$vec[, 1:p]      ## eigenvectors (coordinates)

qgraph(adult_glasso, layout=evecs, groups = list(Depression = 1:16, "OCD" = 17:26),
       color = c("lightblue", "lightsalmon"), title= "Adults, n=408", vsize=4)
title(xlab="Dimension 1", ylab= "Dimension 2")

library("networktools")
adult_glasso <- EBICglasso(cor(Rogers), n=408)
adult_qgraph <- qgraph(adult_glasso)
MDSnet(adult_qgraph, MDSadj=cor(Rogers))
PCAnet(adult_qgraph, cormat = cor(Rogers))
EIGENnet(adult_qgraph)
```

7.5 Datasets for Network Analysis

There is a package called “igraphdata” that contains many network datasets. Additionally, there are several more datasets at “The Colorado Index of Complex Networks (ICON)”. Here is the link: https://icon.colorado.edu/#!/

7.6 Review

In this chapter we introduced network analysis concepts and methods. To make sure you understand this material, there is a practice assessment to go along with this chapter at https://jayholster1.shinyapps.io/NetworksinRAssessment/

7.7 References

Bojanowski, M. (2015). intergraph: Coercion routines for network data objects. R package version 2.0-2. http://mbojan.github.io/intergraph

Csardi, G., Nepusz, T. (2006). "The igraph software package for complex network research." *InterJournal*, Complex Systems, 1695. <https://igraph.org>.

Paranyushkin, D. (2019). InfraNodus: Generating insight using text network analysis. In *The World Wide Web Conference* (*WWW '19*). Association for Computing Machinery, New York, NY, USA, 3584–3589. https://doi.org/10.1145/3308558.3314123

Payton, J. J., Mair, P., & McNally, R. J. (2018). Visualizing psychological networks: A tutorial in R. *Frontiers in Psychology, 9*(1), https://doi.org/10.3389/fpsyg.2018.01742

Tyner, S., Briatte, F., & Hofmann, H. (2017). Network Visualization with `ggplot2`, *The R Journal* 9(1): 27–59. https://briatte.github.io/ggnetwork/

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L.D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T.L., Miller, E., Bache, S.M., Müller, K., Ooms, J., Robinson, D., Seidel, D.P., Spinu, V., Takahashi, K., Vaughan, D., Wilke, C., Woo, K., & Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software, 4*(43), 1686. https://doi.org/10.21105/joss.01686.

7.7.1 R Short Course Series

Video lectures of each guidebook chapter can be found at https://osf.io/6jb9t/. For this chapter, find the follow the folder path Network Analysis in R -> AY 2021-2022 Spring and access the video files, r markdown documents, and other materials for each short course.

7.7.2 Acknowledgements

This guidebook was created with support from the Center for Research Data and Digital Scholarship and the Laboratory for Interdisciplinary Statistical Analysis at the University of Colorado Boulder, as well as the U.S. Agency for

International Development under cooperative agreement #7200AA18CA00022. Individuals who contributed to materials related to this project include Jacob Holster, Eric Vance, Michael Ramsey, Nicholas Varberg, and Nickoal Eichmann-Kalwara.

Chapter 8

Text Analysis

Although the bulk of this guidebook is focused on quantitative analysis, the majority of the data in the real world is qualitative. The phone call you just had with a family member, the meetings you have with your peers and colleagues, as well as the thoughts you carry within you can be represented qualitatively. In fact, this tends to happen frequently whether you are aware of the process or not. When you call your health insurance company to and the automated operator asks you to state the reason for your call, you are logging qualitative data. When you end a zoom meeting a transcript is automatically created that lists all the words shared.

Words are an important part of the scientific method, so much so there has been a notable shift in many social sciences towards qualitative research methods. This chapter does not aim to present a breakdown of qualitative research methods. However, this chapter will provide you with an introduction to R packages and functions that assist in the process of text identification, manipulation, and analysis.

This chapter will utilize far more libraries than previous chapters, so it may be helpful to install each of the following packages at the onset.

```
library(tm) # a text mining package
library(stopwords)
library(tidytext) # a tidyverse friendly text mining package
library(stringr) # a package for manipulating strings
library(gutenbergr)
library(SnowballC) # a package for plotting text based data
library(wordcloud) # another package for plotting text data
library(lubridate)
library(tidyverse)
library(harrypotter) # contains a labelled  corpus of all harry potter books
```

```
library(sentimentr) # simple sentiment analysis function
library(textdata) # another package to support parsing
library(topicmodels) # specify, save and load topic models
library(LDAvis) # visualize the output of Latent Dirichlet Allocation
library(servr) # we use this library to access a data set
library(stringi) # natural language processing tools
library(ldatuning) # automatically specify LDA models
library(reshape2)
library(igraph)
library(ggraph)
devtools::install_github("bradleyboehmke/harrypotter")
```

8.1 R and Text Data

We have encountered text data previously as the "character" data type. The fundamental marker of "character" data is that these data are non-numeric. Character data types are referred to as "strings" individually. Any value inside double or single quotes is stored as a string. In the example below, the string "So long and thanks for all the fish" is input and received as output, while "So sad that it should come to this" is assigned to the object sv1 which can be recalled for future use.

```
"So long and thanks for all the fish"
```

```
## [1] "So long and thanks for all the fish"
```

```
sv1 <- "So sad that it should come to this"
sv1
```

```
## [1] "So sad that it should come to this"
```

Strings delineated by commas can also be assigned to objects as a vector. Run the following chunk to see how each word is separated into distinct strings.

```
sv2 <- c("We", "tried", "to", "warn", "you", "all", "but", "oh", "dear!")
sv2
```

```
## [1] "We"    "tried" "to"    "warn"  "you"   "all"   "but"   "oh"    "dear!"
```

When a string is included in a vector, R will assign other instances in that vector the string datatype. See the vector created below including multiple

datatypes—including Boolean responses, strings, and numerical data. When we run the class function, everything was stored in quotes. As such, it is important to remember that if there is text data in a column, the rest of the column will be labelled as string data.

```
sv3 <- c("The world's about to be destroyed", TRUE, (1:7),
         "There's no point getting all annoyed", FALSE)
sv3
```

```
##  [1] "The world's about to be destroyed"
##  [2] "TRUE"
##  [3] "1"
##  [4] "2"
##  [5] "3"
##  [6] "4"
##  [7] "5"
##  [8] "6"
##  [9] "7"
## [10] "There's no point getting all annoyed"
## [11] "FALSE"
```

```
class(sv3)
```

```
## [1] "character"
```

```
length(sv3)
```

```
## [1] 11
```

To create a list of empty strings, use the `character()` function.

```
sv4 <- character(5)
sv4
```

```
## [1] "" "" "" "" ""
```

```
class(sv4)
```

```
## [1] "character"
```

```
length(sv4)
```

```
## [1] 5
```

You can assign text within empty strings by calling the index and assigning it the desired text.

```
sv4[1] <- "first" # Add strings to a vector using its index
sv4
```

```
## [1] "first" ""      ""      ""      ""
```

Matrices also coerce numerical values to the characters. Use the `class()` function to see the data type.

```
string_matrix <- rbind(1:5, letters[1:5])
string_matrix
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,] "1"  "2"  "3"  "4"  "5"
## [2,] "a"  "b"  "c"  "d"  "e"
```

```
class(string_matrix[1,])
```

```
## [1] "character"
```

Dataframes, however, allow columns to retain data type independent of the data type of other columns. Here, we created a new dataframe called `df`, including the variables Sex, Name, and Age. See the structure of the data frame below.

```
df <- data.frame("Sex" = 1:2, "Age" = c(21,15,18,22,19,23,21,22),
                 "Name" = c("John",
                            "Emily",
                            "Sam",
                            "Eleanor",
                            "Jonathan",
                            "Sarah",
                            "Ren",
                            "Jessie"))

str(df)
```

```
## 'data.frame':    8 obs. of  3 variables:
##  $ Sex : int  1 2 1 2 1 2 1 2
##  $ Age : num  21 15 18 22 19 23 21 22
##  $ Name: chr  "John" "Emily" "Sam" "Eleanor" ...
```

The name column was assigned to be a factor as to retain levels for subsequent analysis. Factors are the data objects which are used to categorize the data and store it as levels. R may detects some strings as a factors automatically. Use `stringsAsFactors = False` to avoid this. There are cases in which it makes sense for strings to serve as factors, for instance when categorical data are reported in a string format.

```
df$Name <- as.factor(df$Name)
str(df)
```

```
## 'data.frame':    8 obs. of  3 variables:
##  $ Sex : int  1 2 1 2 1 2 1 2
##  $ Age : num  21 15 18 22 19 23 21 22
##  $ Name: Factor w/ 8 levels "Eleanor","Emily",..: 4 2 7 1 5 8 6 3
```

You can also use `is.character()` to assess whether a vector includes character data or not.

```
df1 <- data.frame("Sex" = 1:2, "Age" = c(21,15,18,22,19,23,21,22),
"Name" = c("John", "Emily", "Sam", "Eleanor", "Jonathan","Sarah","Ren","Jessie"), stringsAsFactor

str(df1)
```

```
## 'data.frame':    8 obs. of  3 variables:
##  $ Sex : int  1 2 1 2 1 2 1 2
##  $ Age : num  21 15 18 22 19 23 21 22
##  $ Name: chr  "John" "Emily" "Sam" "Eleanor" ...
```

```
is.character(df1$Name)
```

```
## [1] TRUE
```

```
# Can also use is.character() function to tell if something has type "character"
```

8.2 Cleaning and Processing Text

You can match and replace patterns using `str_replace()`, which searches for matches to the argument (assigned object 'pattern') within character vectors.

For larger text mining tasks, such as web scraping, you might use regular expressions, which is often shorted to regex. Regex allows you to construct string searching algorithms to find and replace very specific strings patterns that are not limited to literal characters (e.g. all four letter words in a data set).

Here's a regex cheat sheet: http://www.rstudio.com/wp-content/uploads/2016/09/RegExCheatsheet.pdf. Additionally, you can search for and test regex text patterns here: https://regexr.com/

For our example, let's create a string object that includes some unwanted noise.

```
sometext <- "I pulled /t/t/t/n/t/t some  weirdly /n/n formatted and encoded data from t
sometext
```

```
## [1] "I pulled /t/t/t/n/t/t some  weirdly /n/n formatted and encoded data from the in
```

The `str_detect()` function searches for a character pattern in a string: `str_detect(pattern, string)`.

```
str_detect(sometext, "I pushed")
```

```
## [1] FALSE
```

```
str_detect(sometext, "I pulled")
```

```
## [1] TRUE
```

The `str_replace_all()` function finds and replaces a character pattern: gsub(pattern, replacement, string). By running the following lines of code, we will be able to replace instances of /t and /n with no data, effectively deleting these. We will also delete a few instances of double spaces with single space replacements.

```
sometext <- str_replace_all(sometext, "/t", "")
sometext <- str_replace_all(sometext, "/n", "")
sometext <- str_replace_all(sometext, "  ", " ")
sometext
```

```
## [1] "I pulled some weirdly formatted and encoded data from the internet and I want i
```

You can also use `gsub()` to include several replacements in the same function separated by `|`. This code follows the same process as the previous chunk.

```
sometext <- "I pulled /t/t/t/n/t/t some  weirdly /n/n formatted and encoded data from the interne
spacingisoff <- gsub("/t|/n", "", sometext)
finaltext <- gsub("  ", " ", spacingisoff)
finaltext
```

```
## [1] "I pulled some weirdly formatted and encoded data from the internet and I want it to look
```

8.2.1 Case Study 1

For the first case study, we are going to use a collection of scraped Donald Trump tweets. Run each line in this chunk to load the data set from an OSF static link. Choose your own way to examine the data before we work with it.

```
load(url("https://osf.io/dvrhc/download"))
head(trumptweets$text)
```

```
## [1] "Just met with UN Secretary-General António Guterres who is working hard to "Make the Unit
## [2] "America is a Nation that believes in the power of redemption. America is a Nation that be
## [3] "RT @SteveForbesCEO: .@realDonaldTrump speech on drug costs pays immediate dividends. New
## [4] "We grieve for the terrible loss of life, and send our support and love to everyone affect
we are with you in this tragic hour, and we will be with you forever... https://t.co/LtJ0D29Hsv"
## [5] "School shooting in Texas. Early reports not looking good. God bless all!"
## [6] "Reports are there was indeed at least one FBI representative implanted, for political pur
```

The `tidytext` `unnest()` function pulls out **tokens**, or particular words in a data set, from the column "text" and distributes them into individual rows with accompanying metadata. Tokens represent individual units of meaning, thus the process of dividing text data into individual units is called 'tokenization.'

In reading the code below, we are assigning a new object `tidy_trumps` from the trumptweets dataset where `%>%` the columns created_at, text, is_retweet, source, and hastags are isolated from the rest of the data using `select()`, where then `%>%` the `unnest_tokens()` function is deployed on the 'text' column at the `word` level. This argument can also take `sentence`, `ngram`, and a few other options for dividing your text data.

```
tidy_trumps <- trumptweets %>%
  select(created_at, text, is_retweet, source, hashtags) %>%
  unnest_tokens("word", text)

head(tidy_trumps)
```

```
## # A tibble: 6 x 5
```

```
##   created_at          is_retweet source             hashtags  word
##   <dttm>              <lgl>      <chr>              <list>    <chr>
## 1 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> just
## 2 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> met
## 3 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> with
## 4 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> un
## 5 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> secretary
## 6 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> general
```

Stop words are a set of commonly used words in a language. In natural language processing, we want to filter these words from our data sets. First, let's examine the list of stop words from the `tidytext` package. The lexicon refers to the source of the stop word, as this data set was aggregated from three separate lists. (For more information, see the link: https://juliasilge.github.io/tidytext/reference/stop_words.html)

```
data("stop_words")
head(stop_words)
```

```
## # A tibble: 6 x 2
##   word      lexicon
##   <chr>     <chr>
## 1 a         SMART
## 2 a's       SMART
## 3 able      SMART
## 4 about     SMART
## 5 above     SMART
## 6 according SMART
```

We can use the `anti_join()` function to tell R to find the stop words in our list of tokens and remove them. This action is predicated on the logic of joins, where data sets can be combined in a number of ways. The removal of one token because it exists in another data set is a common NLP task.

```
tidy_trumps <- tidy_trumps %>% anti_join(stop_words)
```

```
## Joining, by = "word"
```

```
head(tidy_trumps)
```

```
## # A tibble: 6 x 5
##   created_at          is_retweet source             hashtags  word
##   <dttm>              <lgl>      <chr>              <list>    <chr>
```

```
## 1 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> met
## 2 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> secretary
## 3 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> antónio
## 4 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> guterres
## 5 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> hard
## 6 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> united
```

Use the `count()` function to see a count of all tokens. The output is not optimal, however. Sorting as TRUE doesn't help much either.

```
head(tidy_trumps %>% count(word, sort = TRUE))
```

```
## # A tibble: 6 x 2
##   word       n
##   <chr>  <int>
## 1 https   1281
## 2 t.co    1258
## 3 amp      562
## 4 rt       351
## 5 people   302
## 6 news     271
```

If prevalent words in the data do not have practical meaning, we need to eliminate their influence. You can create a custom table for the rest of the words you'd like to omit using the code below using the `data.frame()` function.

```
new_stop_words <- data.frame("word" = c("https", "t.co", "rt", "amp"), stringsAsFactors = FALSE)
str(new_stop_words)
```

```
## 'data.frame':    4 obs. of  1 variable:
##  $ word: chr  "https" "t.co" "rt" "amp"
```

Now use the `anti_join()` to remove your new list of words and examine readability of the output.

```
tidy_trumps <- tidy_trumps %>% anti_join(new_stop_words)
```

```
## Joining, by = "word"
```

```
head(tidy_trumps %>% count(word, sort = FALSE))
```

```
## # A tibble: 6 x 2
##   word           n
##   <chr>      <int>
## 1 0             10
## 2 00            33
## 3 00am           1
## 4 00ame          1
## 5 00mao6vk7r     1
## 6 00pm           2
```

As we recently saw, there are several numbers we need to get rid of in the data. Let's use grep and the regular expression for all digits for this task. By the way, when we tokenized the text all punctuation was removed thanks to `tidytext`.

```
tidy_trumps <- tidy_trumps[-grep("\\(?[0-9,.]+\\)?", tidy_trumps$word),]
tidy_trumps %>% count(word, sort = TRUE)
```

```
## # A tibble: 6,695 x 2
##    word          n
##    <chr>     <int>
##  1 people      302
##  2 news        271
##  3 president   235
##  4 fake        234
##  5 trump       218
##  6 country     213
##  7 america     204
##  8 tax         190
##  9 time        173
## 10 american    171
## # ... with 6,685 more rows
```

Finally, tokens should be **stemmed**, which includes the reduction of words to their word stem, base or root form. Use the `wordStem()` function as an argument in the `mutate_at()` function on the column word using the `SnowballC` package to stem similar words (e.g., dancing and dance).

```
tidy_trump_stems <- tidy_trumps %>%
  mutate_at("word", funs(wordStem((.), language="en")))
```

```
## Warning: `funs()` was deprecated in dplyr 0.8.0.
## Please use a list of either functions or lambdas:
##
##   # Simple named list:
```

```
##   list(mean = mean, median = median)
##
##   # Auto named with `tibble::lst()`:
##   tibble::lst(mean, median)
##
##   # Using lambdas
##   list(~ mean(., trim = .2), ~ median(., na.rm = TRUE))
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was generated.
```

```
head(tidy_trump_stems)
```

```
## # A tibble: 6 x 5
##   created_at          is_retweet source             hashtags  word
##   <dttm>              <lgl>      <chr>              <list>    <chr>
## 1 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> met
## 2 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> secretari
## 3 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> antónio
## 4 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> guterr
## 5 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> hard
## 6 2018-05-18 20:41:21 FALSE      Twitter for iPhone <chr [1]> unit
```

Much better!

8.3 Quantitative Text Analysis

We are going to continue to use the cleaned trump tweet data set for some basic quantitative text analysis techniques using R. First, let's make a wordcloud using the `worldcloud()` function. The `with()` function applies the expression to the data set. The `min.freq` argument sets the threshold for which a word must exceed to be placed on the wordcloud. Use lower numbers to increase the granularity of vocabulary used.

```
tidy_trumps %>%
  count(word) %>%
  with(wordcloud(word, n, min.freq = 50, colors = brewer.pal(8, "Dark2")))
```



The **Term Frequency - Inverse Document Frequency** is a commonly used metric for text data where the frequency of terms is considered alongside the uniqueness of terms to produce a measurement of word importance. The tf–idf value increases proportionally to the number of times a word appears in the document and is offset by the number of documents in the corpus that contain the word, which helps to adjust for the fact that some words appear more frequently in general. Use the `bind_tf_idf()` function for this calculation.

```
trump_tfidf <- tidy_trumps %>%
  count(word, created_at) %>%
  bind_tf_idf(word, created_at, n) %>%
  arrange(desc(tf_idf))

head(trump_tfidf)
```

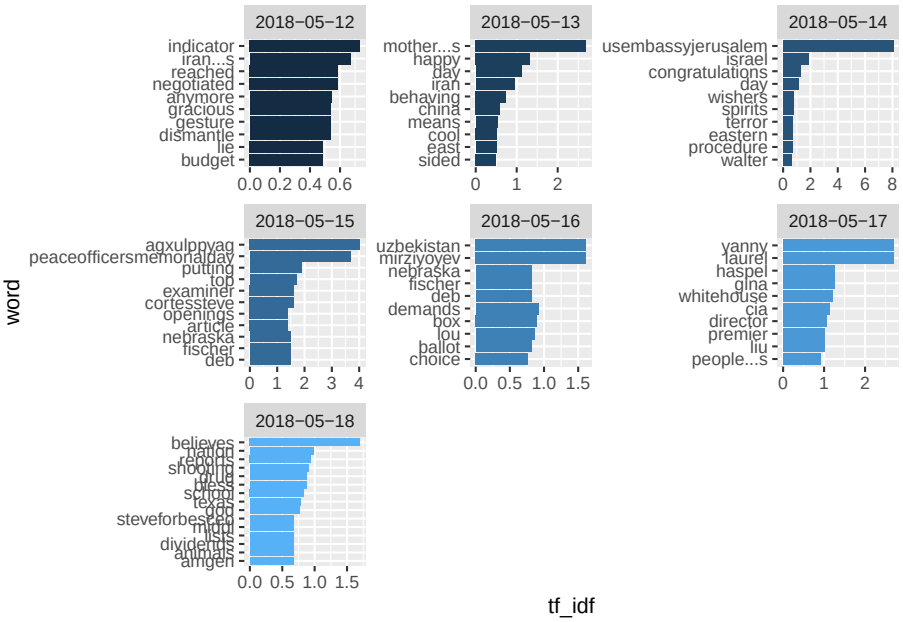
```
## # A tibble: 6 x 6
##   word                    created_at              n    tf   idf tf_idf
##   <chr>                   <dttm>              <int> <dbl> <dbl>  <dbl>
## 1 brazoriacounty          2017-08-29 16:06:57     1     1  8.07   8.07
## 2 congressionalbaseballgame 2017-06-15 23:49:24   1     1  8.07   8.07
## 3 luisriveramarin         2017-10-31 15:48:22     1     1  8.07   8.07
## 4 nicole                  2017-08-05 23:01:05     1     1  8.07   8.07
## 5 senatordole             2017-07-09 00:29:21     1     1  8.07   8.07
## 6 usembassyjerusalem      2018-05-14 15:16:38     1     1  8.07   8.07
```

Let's visualize some tweets over time using the tfidf scores. First, let's convert the created_at column to a Date class using the POSIXct class converter. This helps with time-based analysis as date becomes a more manipulable data type.

```
trump_tfidf$created_at <- as.Date(as.POSIXct(trump_tfidf$created_at))
```

The following code will produce seven plots that include counts of the top ten tokens used on a given day in terms of tfidf. See the comments for how to read each line of code.

```
trump_tfidf %>% #update the object trump_tfidf where
  filter(created_at >= as.Date("2018-05-12")) %>% #the date must be greater than or equal to May
  group_by(created_at) %>% #and the data set is grouped by the same column
  top_n(10, tf_idf) %>% #with only the top 10 tfidf scores
  ungroup() %>% #we then ungroup the date
  mutate(word = reorder(word, tf_idf)) %>% #so that words can be ordered by tfidf
  ggplot(aes(word, tf_idf, fill = created_at)) + #and plotted with words on the x axis and tf_idf
  geom_col(show.legend = FALSE) + #and legends removed
  facet_wrap(~created_at, scales = "free") + #with multiple plots free to expand to the maximum
  coord_flip() #with x and y coordinates then flipped so that text data are more readable.
```



8.4 Sentiment Analysis

Sentiment analysis is the process of computationally identifying and categorizing opinions expressed in a piece of text. The `sentimentr` package makes 'opinion mining' easy. For instance, the `sentiment()` function was deployed below on four separate strings, one including two sentences. Compare the output of these four chunks.

```
sentiment('Sentiment analysis is super fun.')
```

```
##    element_id sentence_id word_count sentiment
## 1:          1           1          5 0.6708204
```

```
sentiment('I hate sentiment analysis.')
```

```
##    element_id sentence_id word_count sentiment
## 1:          1           1          4    -0.375
```

```
sentiment('Sentiment analysis is okay.')
```

```
##    element_id sentence_id word_count sentiment
## 1:          1           1          4         0
```

```
sentiment('Sentiment analysis is super boring. I do love working in R though.')
```

```
##    element_id sentence_id word_count  sentiment
## 1:          1           1          5 -0.1118034
## 2:          1           2          7  0.3779645
```

The sentiment column reflects a number from -1 to +1 that is associated with the negative or positive valence of a particular comment. This works as the `sentimentR` package includes labelled lists of words which are then calculated alongside each other as part of word patterns. Therefore, the package is able to distinguish neutral, positive, and negatively phrased strings.

8.4.1 Case Study Two

For our second case study we are going to be using the harrypotter dataset, which includes the full text from the entire Harry Potter series. For the source code see https://github.com/bradleyboehmke/harrypotter.

First, set books as an array where each value in the array is a chapter. Each book is an array in which each value in the array is a chapter. In the code below, `unnest_tokens()` is deployed on each chapter at the word level for subsequent analysis.

```
books <- list(philosophers_stone, chamber_of_secrets, prisoner_of_azkaban,
              goblet_of_fire, order_of_the_phoenix, half_blood_prince,
              deathly_hallows)

titles <- c("Philosopher's Stone", "Chamber of Secrets", "Prisoner of Azkaban",
            "Goblet of Fire", "Order of the Phoenix", "Half-Blood Prince",
            "Deathly Hallows")

series <- tibble()
for(i in seq_along(titles)) {

  temp <- tibble(chapter = seq_along(books[[i]]), text = books[[i]]) %>%
    unnest_tokens(word, text) %>%
    mutate(book = titles[i]) %>%
    select(book, everything())

  series <- rbind(series, temp)
}
```

Set a factor to keep books ordered by publication date and chapter.

```
series$book <- factor(series$book, levels = rev(titles))
series
```

```
## # A tibble: 1,089,386 x 3
##    book                chapter word
##    <fct>                 <int> <chr>
##  1 Philosopher's Stone       1 the
##  2 Philosopher's Stone       1 boy
##  3 Philosopher's Stone       1 who
##  4 Philosopher's Stone       1 lived
##  5 Philosopher's Stone       1 mr
##  6 Philosopher's Stone       1 and
##  7 Philosopher's Stone       1 mrs
##  8 Philosopher's Stone       1 dursley
##  9 Philosopher's Stone       1 of
## 10 Philosopher's Stone       1 number
## # ... with 1,089,376 more rows
```

```
head(series)
```

```
## # A tibble: 6 x 3
##   book                chapter word
##   <fct>                 <int> <chr>
## 1 Philosopher's Stone       1 the
## 2 Philosopher's Stone       1 boy
## 3 Philosopher's Stone       1 who
## 4 Philosopher's Stone       1 lived
## 5 Philosopher's Stone       1 mr
## 6 Philosopher's Stone       1 and
```

This code will produce the most frequently appearing words in the series.

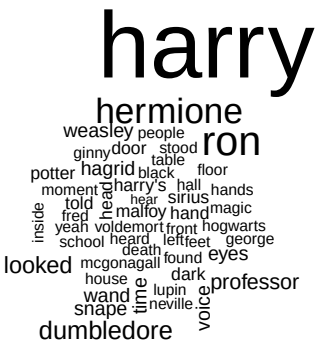
```
head(series %>% count(word, sort = TRUE))
```

```
## # A tibble: 6 x 2
##   word      n
##   <chr> <int>
## 1 the   51593
## 2 and   27430
## 3 to    26985
## 4 of    21802
## 5 a     20966
## 6 he    20322
```

Let's see how removing stop words impacts that list and display the results with a wordcloud.

```
series %>%
  anti_join(stop_words) %>%
  count(word, sort = TRUE) %>%
  with(wordcloud(word, n, max.words = 50))
```

```
## Joining, by = "word"
```



As previously mentioned, there are datasets which are labelled for sentiment. Below we right join the list of sentiment tokens to our harrypotter corpus word list.

```
series %>%
  right_join(get_sentiments("nrc")) %>% # nrc is the name of data set
  filter(!is.na(sentiment)) %>%  # filter
  count(sentiment, sort = TRUE)
```

```
## Joining, by = "word"
```

```
## # A tibble: 10 x 2
##    sentiment        n
##    <chr>        <int>
##  1 negative     55093
##  2 positive     37758
##  3 sadness      34878
##  4 anger        32743
##  5 trust        23154
##  6 fear         21536
##  7 anticipation 20625
##  8 joy          13800
##  9 disgust      12861
## 10 surprise     12817
```

`nrc` filters for positive/negative valence, and has predetermined emotional classifiers built in, while `bing` only uses positive/negative valence.

```
series %>%
  right_join(get_sentiments("bing")) %>%
  filter(!is.na(sentiment)) %>%
  count(sentiment, sort = TRUE)
```

```
## Joining, by = "word"
```

```
## # A tibble: 2 x 2
##   sentiment     n
##   <chr>     <int>
## 1 negative  39502
## 2 positive  29065
```

This example brings in the `bing` word list which is joined to the words in the harrypotter corpus alongside the sentiment scores, and then plots the top positive and negative words in the same wordcloud.

```
series %>%
  inner_join(get_sentiments("bing")) %>%
  count(word, sentiment, sort = TRUE) %>%
  acast(word ~ sentiment, value.var = "n", fill = 0) %>%
  comparison.cloud(colors = c("#F8766D", "#00BFC4"),
                   max.words = 50)
```

```
## Joining, by = "word"
```



Did you notice that example left stopwords in? Let's see how it looks with a stopword `anti_join()`.

```
series %>%
  anti_join(stop_words) %>%
  inner_join(get_sentiments("bing")) %>%
  count(word, sentiment, sort = TRUE) %>%
  acast(word ~ sentiment, value.var = "n", fill = 0) %>%
  comparison.cloud(colors = c("#F8766D", "#00BFC4"),
                   max.words = 50)
```

```
## Joining, by = "word"
## Joining, by = "word"
```



In this next bit of code, we are going to group by words in each book, bring in bing sentiments, count words and sentiment before plotting a geom including the sentiment of words used in each book.

```
series %>%
  group_by(book) %>%
  mutate(word_count = 1:n(),
         index = word_count %/% 500 + 1) %>%
  inner_join(get_sentiments("bing")) %>%
  count(book, index = index , sentiment) %>%
  spread(sentiment, n, fill = 0) %>%
  mutate(sentiment = positive - negative,
         book = factor(book, levels = titles)) %>%
  ggplot(aes(index, sentiment, fill = book)) +
  geom_bar(alpha = 0.5, stat = "identity", show.legend = FALSE) +
  facet_wrap(~ book, ncol = 2, scales = "free_x")
```

```
## Joining, by = "word"
```



To investigate bigrams, use the unnest() function where token = ngrams, and n = 2, and mutate our dataframe accordingly. We then need to order the books appropriately by setting them as a factor once more.

```
series <- tibble()
for(i in seq_along(titles)) {

  temp <- tibble(chapter = seq_along(books[[i]]),
                 text = books[[i]]) %>%
    unnest_tokens(bigram, text, token = "ngrams", n = 2) %>%
    mutate(book = titles[i]) %>%
    select(book, everything())

  series <- rbind(series, temp)
}
series$book <- factor(series$book, levels = rev(titles))
```

Much like word frequencies, this code will count the top bigrams in the dataset.

```
series %>%
  count(bigram, sort = TRUE)
```

```
## # A tibble: 340,021 x 2
##    bigram          n
```

```
##    <chr>      <int>
##  1 of the      4895
##  2 in the      3571
##  3 said harry  2626
##  4 he was      2490
##  5 at the      2435
##  6 to the      2386
##  7 on the      2359
##  8 he had      2138
##  9 it was      2123
## 10 out of      1911
## # ... with 340,011 more rows
```

Bigrams, like words, can be filtered and cleaned to remove stop words and other noise.

```
bigrams_separated <- series %>%
  separate(bigram, c("word1", "word2"), sep = " ")
bigrams_filtered <- bigrams_separated %>%
  filter(!word1 %in% stop_words$word) %>%
  filter(!word2 %in% stop_words$word)

head(bigrams_separated)
```

```
## # A tibble: 6 x 4
##   book                chapter word1 word2
##   <fct>                 <int> <chr> <chr>
## 1 Philosopher's Stone       1 the   boy
## 2 Philosopher's Stone       1 boy   who
## 3 Philosopher's Stone       1 who   lived
## 4 Philosopher's Stone       1 lived mr
## 5 Philosopher's Stone       1 mr    and
## 6 Philosopher's Stone       1 and   mrs
```

```
head(bigrams_filtered)
```

```
## # A tibble: 6 x 4
##   book                chapter word1     word2
##   <fct>                 <int> <chr>     <chr>
## 1 Philosopher's Stone       1 privet    drive
## 2 Philosopher's Stone       1 perfectly normal
## 3 Philosopher's Stone       1 firm      called
## 4 Philosopher's Stone       1 called    grunnings
## 5 Philosopher's Stone       1 usual     amount
## 6 Philosopher's Stone       1 time      craning
```

Here are our updated bigram counts with clean data.

```
bigrams_united <- bigrams_filtered %>%
  unite(bigram, word1, word2, sep = " ")
head(bigrams_united %>%
    count(bigram, sort = TRUE))
```

```
## # A tibble: 6 x 2
##   bigram                   n
##   <chr>                <int>
## 1 professor mcgonagall   578
## 2 uncle vernon           386
## 3 harry potter           349
## 4 death eaters           346
## 5 harry looked           316
## 6 harry ron              302
```

`bind_tf_idf()` works for bigrams too.

```
bigram_tf_idf <- bigrams_united %>%
  count(book, bigram) %>%
  bind_tf_idf(bigram, book, n) %>%
  arrange(desc(tf_idf))
head(bigram_tf_idf)
```

```
## # A tibble: 6 x 6
##   book                 bigram                n      tf   idf  tf_idf
##   <fct>                <chr>             <int>   <dbl> <dbl>   <dbl>
## 1 Order of the Phoenix professor umbridge   173 0.00533 1.25  0.00667
## 2 Prisoner of Azkaban  professor lupin      107 0.00738 0.847 0.00625
## 3 Deathly Hallows      elder wand            58 0.00243 1.95  0.00473
## 4 Goblet of Fire       ludo bagman           49 0.00201 1.95  0.00391
## 5 Prisoner of Azkaban  aunt marge            42 0.00290 1.25  0.00363
## 6 Deathly Hallows      death eaters         139 0.00582 0.560 0.00326
```

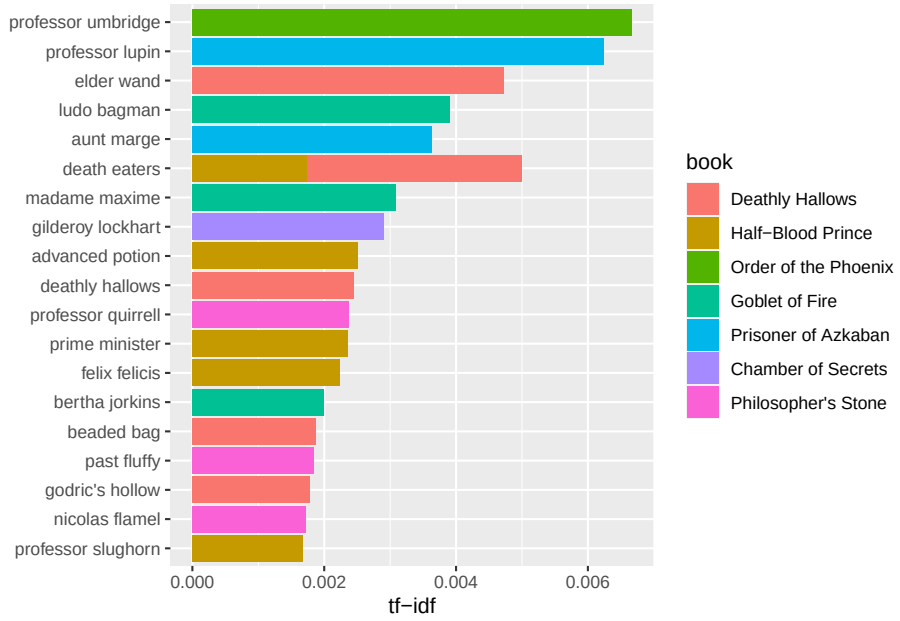
Use this code to display the bigrams with the highest TF-IDF scores across the series.

```
plot_potter<- bigram_tf_idf %>%
  arrange(desc(tf_idf)) %>%
  mutate(bigram = factor(bigram, levels = rev(unique(bigram))))

plot_potter %>%
  top_n(20) %>%
```

```
  ggplot(aes(bigram, tf_idf, fill = book)) +
  geom_col() +
  labs(x = NULL, y = "tf-idf") +
  coord_flip()
```

```
## Selecting by tf_idf
```



We may be overestimating the negative sentiment in the data set due to negatives. Deal with negatives by filtering and removing words that are associated with 'not.'

```
bigrams_separated %>%
  filter(word1 == "not") %>%
  count(word1, word2, sort = TRUE)
```

```
## # A tibble: 1,085 x 3
##    word1 word2     n
##    <chr> <chr> <int>
##  1 not   to      383
##  2 not   be      157
##  3 not   a       131
##  4 not   have    116
##  5 not   the     103
```

```
##  6 not   know     98
##  7 not   going    92
##  8 not   want     81
##  9 not   said     76
## 10 not   been     75
## # ... with 1,075 more rows
```

```
bigrams_separated <- bigrams_separated %>%
  filter(word1 == "not") %>%
  filter(!word2 %in% stop_words$word)%>%
  count(word1, word2, sort = TRUE)

head(bigrams_separated)
```

```
## # A tibble: 6 x 3
##   word1 word2          n
##   <chr> <chr>      <int>
## 1 not   answer        41
## 2 not   speak         41
## 3 not   understand    35
## 4 not   harry         32
## 5 not   supposed      25
## 6 not   dare          24
```

8.5 Networks with Hamilton Lyrics

See this code for an application of network analysis on the corpus from Hamilton. This code was taken from this tutorial: https://cfss.uchicago.edu/notes/hamilton/

```
library(widyr)
library(ggraph)
library(tidyverse)
library(tidytext)
library(ggtext)
library(here)
```

```
## here() starts at /Users/jbh6331/Desktop/R/IDSR
```

```
set.seed(123)
theme_set(theme_minimal())

hamilton <- read_csv(file = here("hamilton.csv")) %>%
  mutate(song_name = parse_factor(song_name))
```

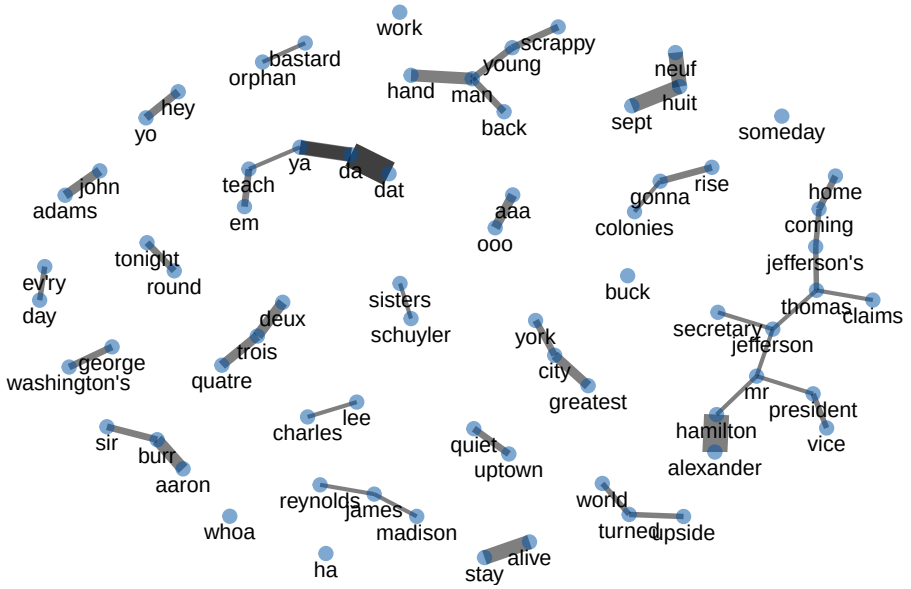
```
## Rows: 3532 Columns: 5
```

```
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (3): song_name, line, speaker
## dbl (2): song_number, line_num
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# calculate all pairs of words in the musical
hamilton_pair <- hamilton %>%
  unnest_tokens(output = word, input = line, token = "ngrams", n = 2) %>%
  separate(col = word, into = c("word1", "word2"), sep = " ") %>%
  filter(!word1 %in% get_stopwords(source = "smart")$word,
         !word2 %in% get_stopwords(source = "smart")$word) %>%
  drop_na(word1, word2) %>%
  count(word1, word2, sort = TRUE)

# filter for only relatively common combinations
bigram_graph <- hamilton_pair %>%
  filter(n > 3) %>%
  igraph::graph_from_data_frame()

# draw a network graph
set.seed(1776) # New York City
ggraph(bigram_graph, layout = "fr") +
  geom_edge_link(aes(edge_alpha = n, edge_width = n), show.legend = FALSE, alpha = .5)
  geom_node_point(color = "#0052A5", size = 3, alpha = .5) +
  geom_node_text(aes(label = name), vjust = 1.5) +
  ggtitle("Word Network in Lin-Manuel Miranda's *Hamilton*") +
  theme_void() +
  theme(plot.title = element_markdown())
```



8.6 Topic Modeling

A traditional content analysis might utilize keyword counting, thematic coding, and other human-driven data categorization to contextualize text data. Instead, we can use a Latent Dirichlet Allocation (LDA) process on the corpora, a machine learning technique that emerged as part of developments in the artificial intelligence field (Gropp & Herzog, 2016). In essence, LDA identifies latent constructs within a corpus through the identification of co-occurring word patterns and semantically similar clustered word combinations. These methods have increased in popularity across the social sciences in the past decade, as robust algorithms allow researchers to investigate the inner workings of text-based data while limiting their bias. LDA algorithms examine entire corpora in minutes, illuminating "statistical regularities in word co-occurrence that often correspond to recognizable themes, events, or discourses" (Baumer, et al., 2017, p. 1398). The output of these algorithms is referred to as a topic model.

Topic model data typically includes a numerical set of topic labels (e.g. topic 0, topic 1, topic 2), a set of keywords that are associated with each topic, and a list of representative sentences for each topic. The algorithm assigns these sentences to each topic based on the prevalence of the topic keywords within the text. In the LDA process, unique keywords are assigned numerical values (e.g. Young = 0, People's = 1, Concerts = 2, and so on). Using these values, the representativeness of each sentence is measured with a logistic distribution and is then assigned a document-topic probability score between 0 and 1. The higher

the score, the better the probability that the topic of the sentence provides represents a larger subset of a corpus.

To get started, let's convert the tidy_ham word column to a corpus from the `tm` library.

```
tidy_ham <- hamilton %>%
  select(song_name, line) %>%
  unnest_tokens("word", line) %>%
  anti_join(stop_words) #%>%
```

```
## Joining, by = "word"
```

```
  #mutate_at("word", funs(wordStem((.), language="en")))

ham_corpus <-iconv(tidy_ham$word)
corpus <- Corpus(VectorSource(ham_corpus))
corpus
```

```
## <<SimpleCorpus>>
## Metadata:  corpus specific: 1, document level (indexed): 0
## Content:  documents: 7338
```

We now transform all upper case to lower case, as well as remove punctuation, stop words, numbers, and whitespace.

```
corpus <- tm_map(corpus, content_transformer(tolower))
```

```
## Warning in tm_map.SimpleCorpus(corpus, content_transformer(tolower)):
## transformation drops documents
```

```
corpus <- tm_map(corpus, removePunctuation)
```

```
## Warning in tm_map.SimpleCorpus(corpus, removePunctuation): transformation drops
## documents
```

```
corpus <- tm_map(corpus,removeWords,stopwords("english"))
```

```
## Warning in tm_map.SimpleCorpus(corpus, removeWords, stopwords("english")):
## transformation drops documents
```

```
corpus <- tm_map(corpus, removeNumbers)
```

```
## Warning in tm_map.SimpleCorpus(corpus, removeNumbers): transformation drops
## documents
```

```
corpus <- tm_map(corpus, stripWhitespace)
```

```
## Warning in tm_map.SimpleCorpus(corpus, stripWhitespace): transformation drops
## documents
```

```
corpus
```

```
## <<SimpleCorpus>>
## Metadata:  corpus specific: 1, document level (indexed): 0
## Content:  documents: 7338
```

The **document term matrix** (dtm) is our new dataframe. Topic model packages interact with dtm's to construct possible models

```
dtm <- DocumentTermMatrix(corpus)
dtm
```

```
## <<DocumentTermMatrix (documents: 7338, terms: 2447)>>
## Non-/sparse entries: 7103/17948983
## Sparsity           : 100%
## Maximal term length: 18
## Weighting          : term frequency (tf)
```

More cleaning. This code will create an index for each token.

```
rowTotals <- apply(dtm,1,sum) #running this line takes time
empty.rows <- dtm[rowTotals==0,]$dimnames[1][[1]]
corpus <- corpus[-as.numeric(empty.rows)]
dtm <- DocumentTermMatrix(corpus)
dtm
```

```
## <<DocumentTermMatrix (documents: 7103, terms: 2447)>>
## Non-/sparse entries: 7103/17373938
## Sparsity           : 100%
## Maximal term length: 18
## Weighting          : term frequency (tf)
```

Use the `inspect()` function to see how the dtm counts text by index.

```
inspect(dtm[1:5, 1:5])
```

```
## <<DocumentTermMatrix (documents: 5, terms: 5)>>
## Non-/sparse entries: 5/20
## Sparsity           : 80%
## Maximal term length: 8
## Weighting          : term frequency (tf)
## Sample             :
##     Terms
## Docs bastard orphan scotsman son whore
##    1       1      0        0   0     0
##    2       0      1        0   0     0
##    3       0      0        0   1     0
##    4       0      0        0   0     1
##    5       0      0        1   0     0
```

Sum and sort these word counts.

```
dtm.mx <- as.matrix(dtm)
frequency <- colSums(dtm.mx)
frequency <- sort(frequency, decreasing=TRUE)
frequency[1:25]
```

```
##      wait      time  hamilton       hey      burr      shot       sir alexander
##        81        79        77        73        65        58        56        51
##      whoa president     gonna      rise     world     story satisfied     alive
##        42        39        38        37        36        35        35        34
##      york      home  helpless  angelica jefferson     fight       dat      eyes
##        33        32        32        31        30        29        27        27
##     write
##        26
```

Topic models must have the number of potential topics specified. There are many ways to go about this, though some researchers depend on trial and error in an iterative process while they check the appropriateness of model output. Four alternative automated approaches are included in the plot below. These models predict the latent distribution of document topic probabilities to produce the best model. Read this output by seeing where the models intersect on each graph.

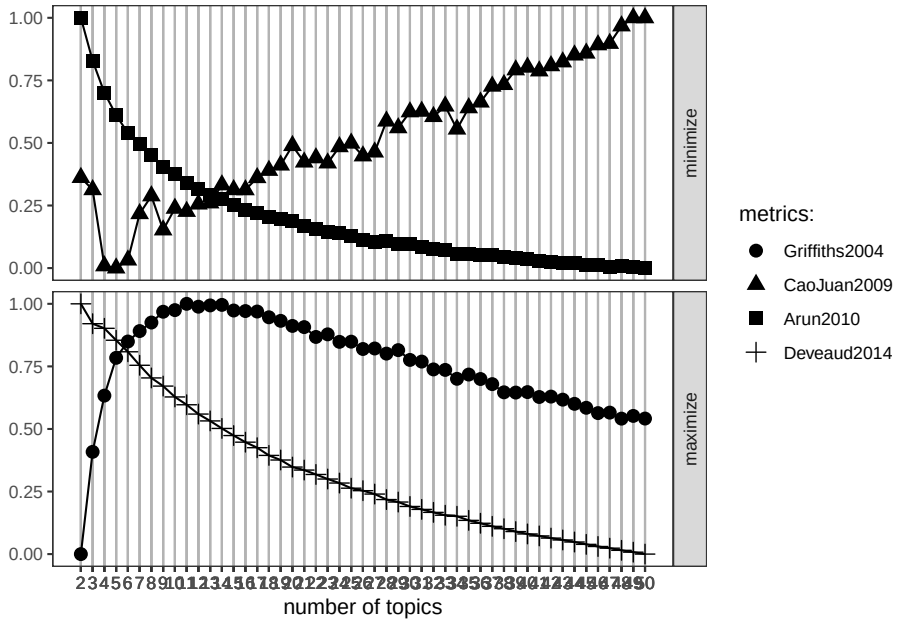
```
result <- FindTopicsNumber(
  dtm,
  topics = seq(from = 2, to = 50, by = 1),
```

```
  metrics = c("Griffiths2004", "CaoJuan2009", "Arun2010", "Deveaud2014"),
  method = "Gibbs",
  control = list(seed = 77),
  mc.cores = 2L,
  verbose = TRUE
)
```

```
## fit models... done.
## calculate metrics:
##   Griffiths2004... done.
##   CaoJuan2009... done.
##   Arun2010... done.
##   Deveaud2014... done.
```

```
FindTopicsNumber_plot(result)
```

```
## Warning: `guides(<scale> = FALSE)` is deprecated. Please use `guides(<scale> =
## "none")` instead.
```



For a deep dive on each of the parameters in the topic models package, see the documentation here: https://cran.r-project.org/web/packages/topicmodels/topicmodels.pdf

For now, all we should focus on is k, which represents the number of topics you are feeding into your model. Based on the previous graph, we should let the model assign 9 topics based on a general interpretation of the previous plot.

```
#set model parameters
burnin <- 4000
iter <- 2000
thin <- 500
seed <-list(2003,5,63,100001,765)
nstart <- 5
best <- TRUE
k <- 9
```

Convert your LDA model to be readable by the `LDAvis` package.

```
ldaOut <-LDA(dtm, k, method="Gibbs", control=list(nstart=nstart, seed = seed, best=best
ldaOut.topics <- as.matrix(topics(ldaOut))
```

See the top six terms in each topic.

```
ldaOut.terms <- as.matrix(terms(ldaOut, 6))
ldaOut.terms
```

```
##      Topic 1    Topic 2     Topic 3    Topic 4 Topic 5     Topic 6
## [1,] "sir"      "hey"       "shot"     "wait"  "hey"       "time"
## [2,] "york"     "president" "gonna"    "gotta" "alive"     "son"
## [3,] "helpless" "whoa"      "hamilton" "life"  "alexander" "war"
## [4,] "wrote"    "story"     "rise"     "ooh"   "choose"    "gon"
## [5,] "happened" "wanna"     "city"     "time"  "sister"    "mind"
## [6,] "love"     "day"       "father"   "boom"  "goodbye"   "history"
##      Topic 7     Topic 8      Topic 9
## [1,] "hamilton"  "home"       "burr"
## [2,] "world"     "jefferson"  "time"
## [3,] "fight"     "alexander"  "angelica"
## [4,] "throwing"  "eyes"       "eliza"
## [5,] "lafayette" "satisfied"  "hamilton"
## [6,] "evry"      "revolution" "stand"
```

This code prepares your LDA visualization.

```
topicmodels2LDAvis <- function(x, ...){
  post <- topicmodels::posterior(x)
  if (ncol(post[["topics"]]) < 3) stop("The model must contain > 2 topics")
  mat <- x@wordassignments
```

```
  LDAvis::createJSON(
    phi = post[["terms"]],
    theta = post[["topics"]],
    vocab = colnames(post[["terms"]]),
    doc.length = slam::row_sums(mat, na.rm = TRUE),
    term.frequency = slam::col_sums(mat, na.rm = TRUE)
  )
}
```

Finally, display your LDA using `LDAvis`.

```
serVis(topicmodels2LDAvis(ldaOut))
```

8.6.1 A Note on Word Embeddings

Word embeddings allow researchers to investigate semantically similar words that do not have the same stem (e.g. anxious and confused). The text below is from a tutorial (linked below) that visualizes text data in the word embedding space along different dimensions such as rating scores. This allows for more quantitative analysis of text data. Link to article here:

https://ocean.sagepub.com/blog/tools-and-tech/text-an-r-package-for-analyzing-human-language?utm_medium=email&utm_content=1P0003B&utm_campaign=not+tracked&utm_term=&em=2bbc1236b0b88e21ec6d31aeeb956250f8f33e8bec73c04fe3b02294f79734bd&source=adestra

```
install.packages('text')
library(text)
# Use data (DP_projections_HILS_SWLS_100) that have been pre-processed with the textProjectionDat
plot_projection <- textProjectionPlot(
  word_data = DP_projections_HILS_SWLS_100,
  y_axes = TRUE,
  title_top = " Supervised Bicentroid Projection of Harmony in life words",
  x_axes_label = "Low vs. High HILS score",
  y_axes_label = "Low vs. High SWLS score",
  position_jitter_hight = 0.5,
  position_jitter_width = 0.8
)
plot_projection
#> $final_plot
```

8.7 Review

In this chapter we introduced text analysis concepts and methods. To make sure you understand this material, there is a practice assessment to go along with this chapter at <https://jayholster.shinyapps.io/Text>AnalysisinRAssessment.

8.8 References

Csardi, G., Nepusz, T. (2006). "The igraph software package for complex network research." *InterJournal*, Complex Systems, 1695. https://igraph.org.

Grun, B., Hornik, K., Blei, D. M., Lafferty, J. D., Phan, X. H., Matsumoto, M., Nishimura, T., & Cobus, S. (2021). Topic models. https://cran.r-project.org/web/packages/topicmodels/index.html

Fellows, I. (2018). wordcloud. https://cran.r-project.org/web/packages/wordcloud/index.html

Feinerer, I., Hornik, K., & Meyer, D. (2008). "Text Mining Infrastructure in R." *Journal of Statistical Software*, **25**(5), 1–54. https://www.jstatsoft.org/v25/i05/.

Gogolewski, M., Tarantus, B., & others (2021). Stringi. https://cran.r-project.org/web/packages/stringi/index.html

Grolemund, G., &Wickham, H. (2011). Dates and times made easy with lubridate. *Journal of Statistical Software*, *40*(3), 1–25. https://www.jstatsoft.org/v40/i03/.

Nikita, M., & Chaney, N. (2020). Ldatuning. https://cran.r-project.org/web/packages/ldatuning/index.html

Hvitfeldt, E., & Silge, J. (2022). textdata. https://cran.r-project.org/web/packages/textdata/index.html

Porter, M., Boulton, R., & Bouchet-Valat, M. (2013). SnowballC. https://cran.r-project.org/web/packages/SnowballC/index.html

Rinker, T.W. (2021). *sentimentr: Calculate Text Polarity Sentiment*. version 2.9.0, <https://github.com/trinker/sentimentr>.

Robinson, D. (2021). gutenbergr. https://cran.r-project.org/web/packages/gutenbergr/index.html

Sievert, C., & Shirley, K. (2015). LDAvis. https://cran.r-project.org/web/packages/LDAvis/index.html

Silge, J., Robinson, D. (2016). tidytext: Text mining and analysis using tidy data principles in R." *JOSS*, *1*(3). https://[doi:10.21105/joss.00037](https://%5Bdoi:10.21105/joss.00037)

Tyner, S., Briatte, F., & Hofmann, H. (2017). Network Visualization with `ggplot2`, *The R Journal* 9(1): 27–59. https://briatte.github.io/ggnetwork/

Wickham, H. (2022). reshape. https://cran.r-project.org/web/packages/reshape/index.html

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L.D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T.L., Miller, E., Bache, S.M., Müller, K., Ooms, J., Robinson, D., Seidel, D.P., Spinu, V., Takahashi, K., Vaughan, D., Wilke, C., Woo, K., & Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software, 4*(43), 1686. https://doi.org/10.21105/joss.01686.

Xie, Y., Sievert, C., Anderson, J., Vaidyanathan, R., & Lesur, R. (2021). servr. https://cran.r-project.org/web/packages/servr/index.html

8.8.1 R Short Course Series

Video lectures of each guidebook chapter can be found at https://osf.io/6jb9t/. For this chapter, find the follow the folder path Text Analysis in R -> AY 2021-2022 Spring and access the video files, r markdown documents, and other materials for each short course.

8.8.2 Acknowledgements

This guidebook was created with support from the Center for Research Data and Digital Scholarship and the Laboratory for Interdisciplinary Statistical Analysis at the University of Colorado Boulder, as well as the U.S. Agency for International Development under cooperative agreement #7200AA18CA00022. Individuals who contributed to materials related to this project include Jacob Holster, Eric Vance, Michael Ramsey, Nicholas Varberg, and Nickoal Eichmann-Kalwara.